

Rust by Example

[Rust](#) 是一门现代系统编程语言，专注于安全性、速度和并发性。通过内存安全而不使用垃圾回收来实现这些目标。

《通过例子学 Rust》(Rust By Example, RBE) 是一系列可运行的示例，它们展示了各种 Rust 概念和标准库。为了更好地利用这些示例，请不要忘记[本地安装 Rust](#)并查看[官方文档](#)。此外，好奇的话，你也可以[查看这个网站的源代码](#)。

现在让我们开始吧！

- [Hello World](#) - 从一个经典的 Hello World 程序开始。
- [Primitives](#) - 学习有符号整数、无符号整数和其他原生类型。
- [自定义类型](#) - 结构体 `struct` 和枚举 `enum`。
- [变量绑定](#) - 可变绑定、作用域、遮蔽。
- [类型](#) - 学习如何改变和定义类型。
- [转换](#) - 在不同类型的数据之间进行转换，如字符串、整数和浮点数。
- [表达式](#) - 学习表达式及其使用方法。
- [控制流](#) - `if/else`、`for` 等。
- [函数](#) - 学习方法、闭包和高阶函数。
- [模块](#) - 使用模块组织代码
- [Crates](#) - Crate 是 Rust 中的编译单元。学习如何创建库。
- [Cargo](#) - 了解官方 Rust 包管理工具的一些基本功能。
- [属性](#) - 属性是应用于某些模块、crate 或项的元数据。
- [泛型](#) - 学习编写可以适用于多种类型参数的函数或数据类型。
- [作用域规则](#) - 作用域在所有权 (ownership)、借用 (borrowing) 和生命周期 (lifetime) 中扮演重要角色。
- [特质](#) - 特质 (trait) 是为未知类型 `self` 定义的一组方法。
- [宏](#) - 宏是一种编写代码以生成其他代码的方式，也被称为元编程。
- [错误处理](#) - 学习 Rust 处理失败的方式。
- [标准库类型](#) - 学习 `std` 标准库提供的一些自定义类型。

- [标准库中的其他内容](#) - 更多关于文件处理、线程的自定义类型。
- [测试](#) - Rust 中的各种测试方法。
- [不安全操作](#) - 学习如何编写和使用不安全代码块。
- [兼容性](#) - 应对 Rust 语言的演进及可能出现的兼容性问题。
- [补充](#) - 文档，基准测试。

Hello World

这是经典的 Hello World 程序的源代码。

```
1 // 这是一行注释，编译器会忽略它。
2 // 你可以点击右侧的 "Run" 按钮来测试这段代码 ->
3 // 如果你更习惯使用键盘，也可以使用 "Ctrl + Enter" 快捷键来运行。
4
5 // 这段代码是可以编辑的，请随意修改！
6 // 你可以随时点击 "Reset" 按钮恢复到初始代码 ->
7
8 // 这是主函数。
9 fn main() {
10     // 当运行编译后的程序时，这里的语句会被执行。
11
12     // 在控制台打印文本。
13     println!("Hello World!");
14 }
```

`println!` 是一个用于在控制台打印文本的[宏](#)。

可以使用 Rust 编译器 `rustc` 来生成可执行文件。

```
$ rustc hello.rs
```

`rustc` 将生成一个名为 `hello` 的可执行文件。

```
$ ./hello
Hello World!
```

练习

点击上方的“运行”按钮查看预期输出。接下来，添加一行新代码，再次使用 `println!` 宏，输出显示如下内容：

```
Hello World!
I'm a Rustacean!
```

注释

每个程序都需要注释，Rust 支持几种不同类型的注释：

Regular Comments

These are ignored by the compiler:

- **Line comments:** Start with `//` and continue to the end of the line
- **Block comments:** Enclosed in `/* ... */` and can span multiple lines

Documentation Comments (Doc Comments) which are parsed into HTML library [documentation](#):

- `///` - Generates docs for the item that follows it
- `//!` - Generates docs for the enclosing item (typically used at the top of a file or module)


```
1 |  
2 fn main() {  
3     // Line comments start with two slashes.  
4     // Everything after the slashes is ignored by the compiler.  
5  
6     // Example: This line won't execute  
7     // println!("Hello, world!");  
8  
9     // Try removing the slashes above and running the code again.  
10  
11     /*  
12         Block comments are useful for temporarily disabling code.  
13         They can also be nested: /* like this */ which makes it easy  
14         to comment out large sections quickly.  
15     */  
16  
17     /*  
18     * Note: The asterisk column on the left is just for style -  
19     * it's not required by the language.  
20     */  
21  
22     // Block comments make it easy to toggle code on/off by adding  
23     // or removing just one slash:  
24  
25     /* <- Add a '/' here to uncomment the entire block below  
26  
27     println!("Now");  
28     println!("everything");  
29     println!("executes!");  
30     // Line comments inside remain unaffected  
31  
32     // */  
33  
34     // Block comments can also be used within expressions:  
35     let x = 5 + /* 90 + */ 5;  
36     println!("`x` 是 10 还是 100? x = {}", x);  
37 }
```

另请参阅：

[库文档](#)

格式化打印

打印功能由 `std::fmt` 中定义的一系列宏处理，其中包括：

- `format!`：将格式化文本写入 `String`
- `print!`：与 `format!` 类似，但文本会打印到控制台 (`io::stdout`)
- `println!`：与 `print!` 类似，但会在末尾添加换行符
- `eprint!`：与 `print!` 类似，但文本会打印到标准错误输出 (`io::stderr`)
- `eprintln!`：与 `eprint!` 类似，但会在末尾添加换行符

所有这些宏都以相同的方式解析文本。此外，Rust 会在编译时检查格式化的正确性。

```

1 fn main() {
2     // 通常, `{}` 会被自动替换为任何参数。
3     // 这些参数会被转换为字符串。
4     println!("{}", 31);
5
6     // 可以使用位置参数。在 `{}` 中指定一个整数
7     // 来决定替换哪个额外的参数。参数编号
8     // 从格式字符串后立即开始, 从 0 开始。
9     println!("{0}, 这是 {1}。{1}, 这是 {0}", "Alice", "Bob");
10
11    // 还可以使用命名参数。
12    println!("{subject} {verb} {object}",
13             object="那只懒惰的狗",
14             subject="那只敏捷的棕色狐狸",
15             verb="跳过");
16
17    // 在 `:` 后指定格式字符,
18    // 可以调用不同的格式化方式。
19    println!("{}", 69420); // 69420
20    println!("{:b}", 69420); // 10000111100101100
21    println!("{:o}", 69420); // 207454
22    println!("{:x}", 69420); // 10f2c
23
24    // 可以指定宽度来右对齐文本。这将输出
25    // "    1"。(四个空格和一个 "1", 总宽度为 5。)
26    println!("{number:>5}", number=1);
27
28    // 可以用额外的零来填充数字,
29    println!("{number:0>5}", number=1); // 00001
30    // 通过翻转符号来左对齐。这将输出 "10000"。
31    println!("{number:0<5}", number=1); // 10000
32
33    // 在格式说明符后添加 `$` 可以使用命名参数。
34    println!("{number:0>width$}", number=1, width=5);
35
36    // Rust 甚至会检查使用的参数数量是否正确。
37    println!("我的名字是 {0}, {1} {0}", "Bond");
38    // FIXME ^ 添加缺失的参数: "James"
39
40    // 只有实现了 fmt::Display 的类型才能用 `{}` 格式化。
41    // 用户定义的类型默认不实现 fmt::Display。
42
43    #[allow(dead_code)] // 禁用 `dead_code`, 它会警告未使用的模块
44    struct Structure(i32);
45
46    // 这无法编译, 因为 `Structure` 没有实现 fmt::Display。
47    // println!("这个结构体 `{}` 无法打印...", Structure(3));
48    // TODO ^ 尝试取消注释这一行
49
50    // 在 Rust 1.58 及以上版本, 你可以直接从周围的变量捕获参数。
51    // 就像上面一样, 这将输出 "    1", 4 个空格和一个 "1"。
52    let number: f64 = 1.0;
53    let width: usize = 5;
54    println!("{number:>width$}");
55 }

```

`std::fmt` 包含许多控制文本显示的 `traits`。下面列出了两个重要的基本形式:

- `fmt::Debug`: 使用 `{:?}` 标记。用于调试目的的文本格式化。
- `fmt::Display`: 使用 `{}` 标记。以更优雅、用户友好的方式格式化文本。

这里我们使用 `fmt::Display`，因为标准库为这些类型提供了实现。要打印自定义类型的文本，需要额外的步骤。

实现 `fmt::Display` 特性会自动实现 `ToString` 特性，这允许我们将该类型转换为 `String`。

In line 43, `#[allow(dead_code)]` is an `attribute` which only applies to the item after it.

练习

- 修复上述代码中的问题（参见 `FIXME` 注释），使其能够正常运行。
- 尝试取消注释那行尝试格式化 `Structure` 结构体的代码（参见 `TODO` 注释）
- 添加一个 `println!` 宏调用，打印：`Pi` 约等于 `3.142`，通过控制显示的小数位数来实现。在本练习中，使用 `let pi = 3.141592` 作为 `pi` 的近似值。（提示：你可能需要查阅 `std::fmt` 文档来了解如何设置显示的小数位数）

另请参阅：

`std::fmt`、`macros`、`struct`、`traits` 和 `dead_code`

调试 Debug

所有想要使用 `std::fmt` 格式化 `traits` 的类型都需要实现才能打印。自动实现仅为 `std` 库中的类型提供。所有其他类型都**必须**以某种方式手动实现。

`fmt::Debug` trait 使这变得非常简单。**所有类型都可以** `derive`（自动创建） `fmt::Debug` 实现。但这对 `fmt::Display` 不适用，后者必须手动实现。

```
// 这个结构体无法通过 `fmt::Display` 或 `fmt::Debug` 打印。
struct UnPrintable(i32);

// `derive` 属性自动创建使这个 `struct` 可以用 `fmt::Debug` 打印的实现。
#[derive(Debug)]
struct DebugPrintable(i32);
```

所有 `std` 库类型也可以自动使用 `{:?}` 打印：

```
1 // 为 `Structure` 派生 `fmt::Debug` 实现。`Structure` 是一个包含单个 `i32`
2 #[derive(Debug)]
3 struct Structure(i32);
4
5 // 在 `Deep` 结构体中放入一个 `Structure`。使其也可打印。
6 #[derive(Debug)]
7 struct Deep(Structure);
8
9 fn main() {
10     // 使用 `{:?}` 打印类似于使用 `{}`。
11     println!("{:?} 个月在一年中。", 12);
12     println!("{1:?} {0:?} 是这个 {actor:?} 的名字。",
13              "Slater",
14              "Christian",
15              actor="演员");
16
17     // `Structure` 现在可以打印了！
18     println!("现在 {:?} 将会打印！", Structure(3));
19
20     // `derive` 的问题是无法控制输出的样式。
21     // 如果我只想显示一个 `7` 怎么办？
22     println!("现在 {:?} 将会打印！", Deep(Structure(7)));
23 }
```

所以 `fmt::Debug` 确实使其可打印，但牺牲了一些优雅。Rust 还提供了使用 `{:#?}` 进行“美化打印”的功能。

```
1 #[derive(Debug)]
2 struct Person<'a> {
3     name: &'a str,
4     age: u8
5 }
6
7 fn main() {
8     let name = "Peter";
9     let age = 27;
10    let peter = Person { name, age };
11
12    // 美化打印
13    println!("{:#?}", peter);
14 }
```

可以手动实现 `fmt::Display` 来控制显示方式。

另请参阅：

[attributes](#)、[derive](#)、[std::fmt](#) 和 [struct](#)

显示 Display

`fmt::Debug` 的输出往往不够简洁清晰，因此自定义输出外观通常更有优势。这可以通过手动实现 `fmt::Display` 来完成，它使用 `{}` 打印标记。实现方式如下：

```
// 通过 `use` 导入 `fmt` 模块使其可用。
use std::fmt;

// 定义一个结构体，我们将为其实现 `fmt::Display`。
// 这是一个名为 `Structure` 的元组结构体，包含一个 `i32`。
struct Structure(i32);

// 要使用 `{}` 标记，必须为该类型手动实现 `fmt::Display` trait。
impl fmt::Display for Structure {
    // 这个 trait 要求 `fmt` 方法具有确切的签名。
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        // 将第一个元素严格写入提供的输出流 `f`。
        // 返回 `fmt::Result`，表示操作是否成功。
        // 注意 `write!` 的语法与 `println!` 非常相似。
        write!(f, "{}", self.0)
    }
}
```

`fmt::Display` 可能比 `fmt::Debug` 更简洁，但这给 `std` 库带来了一个问题：如何显示歧义类型？例如，如果 `std` 库为所有 `Vec<T>` 实现统一的样式，应该采用哪种样式？是以下两种之一吗？

- `Vec<path>`： `./etc:/home/username:/bin` （以 `:` 分隔）
- `Vec<number>`： `1,2,3` （以 `,` 分隔）

答案是否定的。因为不存在适用于所有类型的理想样式，`std` 库也不应擅自规定一种。因此，`fmt::Display` 并未为 `Vec<T>` 或其他泛型容器实现。在这些泛型情况下，必须使用 `fmt::Debug`。

不过，这不是问题。对于任何新的非泛型容器类型，都可以实现 `fmt::Display`。

```

1 use std::fmt; // 导入 `fmt`
2
3 // 定义一个包含两个数字的结构体。派生 `Debug` 特性，
4 // 以便与 `Display` 的结果进行对比。
5 #[derive(Debug)]
6 struct MinMax(i64, i64);
7
8 // 为 `MinMax` 实现 `Display` 特性。
9 impl fmt::Display for MinMax {
10     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
11         // 使用 `self.number` 引用每个位置的数据点。
12         write!(f, "({}, {})", self.0, self.1)
13     }
14 }
15
16 // 定义一个结构体，其字段可命名以便比较。
17 #[derive(Debug)]
18 struct Point2D {
19     x: f64,
20     y: f64,
21 }
22
23 // 同样，为 `Point2D` 实现 `Display` 特性。
24 impl fmt::Display for Point2D {
25     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
26         // 自定义实现，只显示 `x` 和 `y`。
27         write!(f, "x: {}, y: {}", self.x, self.y)
28     }
29 }
30
31 fn main() {
32     let minmax = MinMax(0, 14);
33
34     println!("比较结构体: ");
35     println!("Display: {}", minmax);
36     println!("Debug: {:?}", minmax);
37
38     let big_range = MinMax(-300, 300);
39     let small_range = MinMax(-3, 3);
40
41     println!("大范围是 {big}, 小范围是 {small}",
42             small = small_range,
43             big = big_range);
44
45     let point = Point2D { x: 3.3, y: 7.2 };
46
47     println!("比较点: ");
48     println!("Display: {}", point);
49     println!("Debug: {:?}", point);
50
51     // The following line would not compile: both `Debug` and `Display`
52     // were implemented, but `{:b}` requires `fmt::Binary` to be
53     // implemented, which it hasn't been for `Point2D`.
54     // println!("What does Point2D look like in binary: {:b}?", point);
55 }

```

因此，虽然实现了 `fmt::Display`，但未实现 `fmt::Binary`，所以无法使用。`std::fmt` 包含许多这样的 `traits`，每个都需要单独实现。更多详情请参阅 [std::fmt](#)。

练习

查看上述示例的输出后，参考 `Point2D` 结构体，向示例中添加一个 `Complex` 结构体。以相同方式打印时，输出应为：

```
Display: 3.3 +7.2i
Debug: Complex { real: 3.3, imag: 7.2 }

Display: 4.7 -2.3i
Debug: Complex { real: 4.7, imag: -2.3 }
```

Bonus: Add a space after the `+` / `-` signs.

如果你遇到困难，这里有三个提示：

- 你可以使用 `:0>2` 用零填充到宽度为 2。
- 额外练习：查看 `if-else` 分支语句以及 `abs` 函数。

另请参阅：

`derive`、`std::fmt`、`macros`、`struct`、`trait` 和 `use`

测试实例：列表

为一个需要顺序处理元素的结构体实现 `fmt::Display` 是棘手的。问题在于每个 `write!` 都会生成一个 `fmt::Result`。正确处理这种情况需要处理所有的结果。Rust 提供了 `?` 运算符专门用于此目的。

在 `write!` 上使用 `?` 的示例如下：

```
// 尝试执行 `write!`，检查是否出错。如果出错，返回错误。
// 否则继续执行。
write!(f, "{}", value)?;
```

有了 `?` 运算符，为 `Vec` 实现 `fmt::Display` 就变得简单明了：

```
1 use std::fmt; // 导入 `fmt` 模块。
2
3 // 定义一个名为 `List` 的结构体，包含一个 `Vec`。
4 struct List(Vec<i32>);
5
6 impl fmt::Display for List {
7     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
8         // Create a reference to the Vec<i32> stored in the List struct.
9         let vec = &self.0;
10
11         write!(f, "[")?;
12
13         // 遍历 `vec` 中的 `v`，同时用 `index` 枚举迭代索引。
14         for (index, v) in vec.iter().enumerate() {
15             // 除第一个元素外，为每个元素添加逗号。
16             // 使用 `?` 运算符在出错时返回。
17             if index != 0 { write!(f, ",")?; }
18             write!(f, "{}", v)?;
19         }
20
21         // 闭合左括号并返回 `fmt::Result` 值
22         write!(f, "]")
23     }
24 }
25
26 fn main() {
27     let v = List(vec![1, 2, 3]);
28     println!("{}", v);
29 }
```

练习

尝试修改程序，使向量中每个元素的索引也被打印出来。新的输出应该如下所示：

```
[0: 1, 1: 2, 2: 3]
```

另请参阅：

`for`、`ref`、`Result`、`struct`、`?` 和 `vec!`

格式化

我们已经看到，格式化是通过一个_格式字符串_来指定的：

- `format!("{}", foo) -> "3735928559"`
- `format!("{:X}", foo) -> "0xDEADBEEF"`
- `format!("{:o}", foo) -> "0o33653337357"`

同一个变量（`foo`）可以根据使用的**参数类型**而有不同的格式化方式：`x`、`o` 或**未指定**。

这种格式化功能是通过 `trait` 实现的，每种参数类型都对应一个 `trait`。最常用的格式化 `trait` 是 `Display`，它处理参数类型未指定的情况，例如 `{}`。

```

1 use std::fmt::{self, Formatter, Display};
2
3 struct City {
4     name: &'static str,
5     // 纬度
6     lat: f32,
7     // 经度
8     lon: f32,
9 }
10
11 impl Display for City {
12     // `f` 是一个缓冲区，此方法必须将格式化的字符串写入其中。
13     fn fmt(&self, f: &mut Formatter) -> fmt::Result {
14         let lat_c = if self.lat >= 0.0 { 'N' } else { 'S' };
15         let lon_c = if self.lon >= 0.0 { 'E' } else { 'W' };
16
17         // `write!` 类似于 `format!`，但它会将格式化后的字符串
18         // 写入一个缓冲区（第一个参数）。
19         write!(f, "{}: {:.3}°{} {:.3}°{}",
20             self.name, self.lat.abs(), lat_c, self.lon.abs(), lon_c)
21     }
22 }
23
24 #[derive(Debug)]
25 struct Color {
26     red: u8,
27     green: u8,
28     blue: u8,
29 }
30
31 fn main() {
32     for city in [
33         City { name: "Dublin", lat: 53.347778, lon: -6.259722 },
34         City { name: "Oslo", lat: 59.95, lon: 10.75 },
35         City { name: "Vancouver", lat: 49.25, lon: -123.1 },
36     ] {
37         println!("{}", city);
38     }
39     for color in [
40         Color { red: 128, green: 255, blue: 90 },
41         Color { red: 0, green: 3, blue: 254 },
42         Color { red: 0, green: 0, blue: 0 },
43     ] {
44         // 一旦你为 fmt::Display 添加了实现，就把这里改成使用 {}。
45         println!("{:?}", color);
46     }
47 }

```

你可以在 `std::fmt` 文档中查看[格式化 trait 的完整列表](#)及其参数类型。

练习

为上面的 `Color` 结构体实现 `fmt::Display` trait，使输出显示如下：

```
RGB (128, 255, 90) 0x80FF5A
```

```
RGB (0, 3, 254) 0x0003FE
```

```
RGB (0, 0, 0) 0x000000
```

Two hints if you get stuck:

- 你可能需要多次列出每种颜色。
- You can [pad with zeros to a width of 2](#) with `:0>2`. For hexadecimals, you can use `:02X`.

Bonus:

- If you would like to experiment with [type casting](#) in advance, the formula for [calculating a color in the RGB color space](#) is $RGB = (R * 65_536) + (G * 256) + B$, where `R` is RED, `G` is GREEN, and `B` is BLUE. An unsigned 8-bit integer (`u8`) can only hold numbers up to 255. To cast `u8` to `u32`, you can write `variable_name` as `u32`.

另请参阅：

[std::fmt](#)

原生类型

Rust 提供了多种 原生类型 。以下是一些示例：

标量类型

- 有符号整数： `i8` 、 `i16` 、 `i32` 、 `i64` 、 `i128` 和 `isize` （指针大小）
- 无符号整数： `u8` 、 `u16` 、 `u32` 、 `u64` 、 `u128` 和 `usize` （指针大小）
- 浮点数： `f32` 、 `f64`
- `char` Unicode 标量值，如 `'a'` 、 `'α'` 和 `'∞'` （每个都是 4 字节）
- `bool` 值为 `true` 或 `false`
- 单元类型 `()` ，其唯一可能的值是空元组： `()`

尽管单元类型的值是一个元组，但它不被视为复合类型，因为它不包含多个值。

复合类型

- 数组，如 `[1, 2, 3]`
- 元组，如 `(1, true)`

变量总是可以进行**类型标注**。数字还可以通过**后缀或默认方式**来标注。整数默认为 `i32` 类型，浮点数默认为 `f64` 类型。请注意，Rust 也可以从上下文中推断类型。

```
1 fn main() {
2     // 变量可以被类型标注。
3     let logical: bool = true;
4
5     let a_float: f64 = 1.0; // 常规标注
6     let an_integer = 5i32; // 后缀标注
7
8     // 或者使用默认类型。
9     let default_float = 3.0; // `f64`
10    let default_integer = 7; // `i32`
11
12    // 类型也可以从上下文中推断。
13    let mut inferred_type = 12; // 从另一行推断出类型为 i64。
14    inferred_type = 4294967296i64;
15
16    // 可变变量的值可以改变。
17    let mut mutable = 12; // 可变的 `i32`
18    mutable = 21;
19
20    // 报错！变量的类型不能改变。
21    mutable = true;
22
23    // 变量可以通过遮蔽 (shadowing) 来覆盖。
24    let mutable = true;
25
26    /* 复合类型 - 数组和元组 */
27
28    // 数组的签名由类型 T 和长度组成，表示为 [T; length]。
29    let my_array: [i32; 5] = [1, 2, 3, 4, 5];
30
31    // 元组是不同类型值的集合，
32    // 使用圆括号 () 构造。
33    let my_tuple = (5u32, 1u8, true, -5.04f32);
34 }
```

另请参阅：

[std 库](#)、[mut](#)、[inference](#) 和 [shadowing](#)

字面量和运算符

整数 `1`、浮点数 `1.2`、字符 `'a'`、字符串 `"abc"`、布尔值 `true` 和单元类型 `()` 可以用字面值表示。

整数也可以使用十六进制、八进制或二进制表示法，分别使用这些前缀：`0x`、`0o` 或 `0b`。

可以在数字字面值中插入下划线以提高可读性，例如 `1_000` 与 `1000` 相同，`0.000_001` 与 `0.000001` 相同。

Rust 还支持科学计数法 [E-notation](#)，例如 `1e6`、`7.6e-4`。相关类型是 `f64`。

我们需要告诉编译器我们使用的字面值的类型。现在，我们将使用 `u32` 后缀来表示该字面值是一个无符号 32 位整数，使用 `i32` 后缀来表示它是一个有符号 32 位整数。

Rust 中可用的运算符及其优先级与其他 [类 C 语言](#) 类似。

```
1 fn main() {
2     // 整数加法
3     println!("1 + 2 = {}", 1u32 + 2);
4
5     // 整数减法
6     println!("1 - 2 = {}", 1i32 - 2);
7     // TODO ^ 尝试将 `1i32` 改为 `1u32`，看看为什么类型很重要
8
9     // 科学记数法
10    println!("1e4 is {}, -2.5e-3 is {}", 1e4, -2.5e-3);
11
12    // 短路布尔逻辑
13    println!("true AND false is {}", true && false);
14    println!("true OR false is {}", true || false);
15    println!("NOT true is {}", !true);
16
17    // 位运算
18    println!("0011 AND 0101 is {:04b}", 0b0011u32 & 0b0101);
19    println!("0011 OR 0101 is {:04b}", 0b0011u32 | 0b0101);
20    println!("0011 XOR 0101 is {:04b}", 0b0011u32 ^ 0b0101);
21    println!("1 << 5 is {}", 1u32 << 5);
22    println!("0x80 >> 2 is 0x{:x}", 0x80u32 >> 2);
23
24    // 使用下划线来提高可读性！
25    println!("One million is written as {}", 1_000_000u32);
26 }
```

元组

元组是一个可以包含各种类型的值的集合。元组使用圆括号 `()` 来构造，而且每个元组本身是一个类型标记为 `(T1, T2, ...)` 的值，其中 `T1`、`T2` 是其成员的类型。函数可以使用元组来返回多个值，因为元组可以存储任意数量的值。

```
1 // 元组可以用作函数参数和返回值。
2 fn reverse(pair: (i32, bool)) -> (bool, i32) {
3     // `let` 可以用来把元组的成员绑定到变量。
4     let (int_param, bool_param) = pair;
5
6     (bool_param, int_param)
7 }
8
9 // 以下结构体将用于后续练习。
10 #[derive(Debug)]
11 struct Matrix(f32, f32, f32, f32);
12
13 fn main() {
14     // 一个包含多种不同类型的元组。
15     let long_tuple = (1u8, 2u16, 3u32, 4u64,
16                     -1i8, -2i16, -3i32, -4i64,
17                     0.1f32, 0.2f64,
18                     'a', true);
19
20     // 可以使用元组索引来提取元组中的值。
21     println!("长元组的第一个值: {}", long_tuple.0);
22     println!("长元组的第二个值: {}", long_tuple.1);
23
24     // 元组可以作为元组的成员。
25     let tuple_of_tuples = ((1u8, 2u16, 2u32), (4u64, -1i8), -2i16);
26
27     // 元组是可打印的。
28     println!("元组的元组: {:?}", tuple_of_tuples);
29
30     // 但长元组（超过 12 个元素）无法打印。
31     //let too_long_tuple = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13);
32     //println!("过长的元组: {:?}", too_long_tuple);
33     // TODO ^ 取消上面两行的注释以查看编译器错误
34
35     let pair = (1, true);
36     println!("对组是 {:?}", pair);
37
38     println!("反转后的对组是 {:?}", reverse(pair));
39
40     // 创建单元素元组时，需要使用逗号来区分它们
41     // 与被括号包围的字面量。
42     println!("单元素元组: {:?}", (5u32,));
43     println!("仅是一个整数: {:?}", (5u32));
44
45     // 可以通过解构元组来创建绑定。
46     let tuple = (1, "hello", 4.5, true);
47
48     let (a, b, c, d) = tuple;
49     println!("{:?}, {:?}", a, b, c, d);
50
51     let matrix = Matrix(1.1, 1.2, 2.1, 2.2);
52     println!("{:?}", matrix);
53 }
```

练习

1. **复习**：为上面示例中的 `Matrix` 结构体添加 `fmt::Display` trait，这样当你从打印调试格式 `{:?}` 切换到显示格式 `{}` 时，你会看到以下输出：

```
( 1.1 1.2 )
( 2.1 2.2 )
```

你可能需要回顾一下 [打印显示](#) 的示例。

2. 参考 `reverse` 函数的模板，添加一个 `transpose` 函数。该函数接受一个矩阵作为参数，并返回一个交换了两个元素的矩阵。例如：

```
println!("矩阵: \n{}", matrix);
println!("转置: \n{}", transpose(matrix));
```

输出结果为：

```
Matrix:
( 1.1 1.2 )
( 2.1 2.2 )
Transpose:
( 1.1 2.1 )
( 1.2 2.2 )
```

数组和切片

数组是一种存储在连续内存中的相同类型 `T` 的对象集合。数组使用方括号 `[]` 创建，其长度在编译时已知，是其类型签名 `[T; length]` 的一部分。

切片类似于数组，但其长度在编译时未知。切片是一个双字对象：第一个字是指向数据的指针，第二个字是切片的长度。字的大小与 `usize` 相同，由处理器架构决定，例如在 `x86-64` 上是 64 位。切片可用于借用数组的一部分，其类型签名为 `&[T]`。

```
1 use std::mem;
2
3 // 此函数借用一个切片。
4 fn analyze_slice(slice: &[i32]) {
5     println!("切片的第一个元素: {}", slice[0]);
6     println!("切片有 {} 个元素", slice.len());
7 }
8
9 fn main() {
10     // 固定大小的数组（类型签名是多余的）。
11     let xs: [i32; 5] = [1, 2, 3, 4, 5];
12
13     // 所有元素可以初始化为相同的值。
14     let ys: [i32; 500] = [0; 500];
15
16     // 索引从 0 开始。
17     println!("数组的第一个元素: {}", xs[0]);
18     println!("数组的第二个元素: {}", xs[1]);
19
20     // `len` 返回数组中元素的数量。
21     println!("数组中的元素数量: {}", xs.len());
22
23     // 数组是栈分配的。
24     println!("数组占用 {} 字节", mem::size_of_val(&xs));
25
26     // 数组可以自动借用为切片。
27     println!("将整个数组借用为切片。");
28     analyze_slice(&xs);
29
30     // 切片可以指向数组的一部分。
31     // 它们的形式是 [起始索引..结束索引]。
32     // `起始索引` 是切片中的第一个位置。
33     // `结束索引` 是切片中最后一个位置的后一个位置。
34     println!("借用数组的一部分作为切片。");
35     analyze_slice(&ys[1 .. 4]);
36
37     // 空切片 `&[]` 的示例：
38     let empty_array: [u32; 0] = [];
39     assert_eq!(&empty_array, &[]);
40     assert_eq!(&empty_array, &[][..]); // 同样的操作，但更详细
41
42     // 可以使用 `.get` 安全地访问数组，它返回一个 `Option`。
43     // 可以像下面这样对其进行匹配，或者使用 `.expect()`。
44     // 如果你希望程序在访问越界时优雅地退出而不是继续执行，
45     // 可以使用 `.expect()`。
46     for i in 0..xs.len() + 1 { // 糟糕，访问超出了数组范围！
47         match xs.get(i) {
48             Some(xval) => println!("{}", i, xval),
49             None => println!("慢着！{} 超出范围了！", i),
50         }
51     }
52
53     // 使用常量值对数组的越界索引会导致编译时错误。
54     //println!("{}", xs[5]);
55     // 对切片的越界索引会导致运行时错误。
56     //println!("{}", xs[..][5]);
57 }
```

自定义类型

Rust 的自定义数据类型主要通过以下两个关键字来创建：

- `struct`：定义结构体
- `enum`：定义枚举

常量也可以通过 `const` 和 `static` 关键字来创建。

结构体

使用 `struct` 关键字可以创建三种类型的结构体（“structs”）：

- 元组结构体：本质上是具名元组。
- 经典的 [C 语言风格结构体](#)
- 单元结构体：没有字段，在泛型中很有用。


```
1  // 这个属性用于隐藏未使用代码的警告。
2  #![allow(dead_code)]
3
4  #[derive(Debug)]
5  struct Person {
6      name: String,
7      age: u8,
8  }
9
10 // 单元结构体
11 struct Unit;
12
13 // 元组结构体
14 struct Pair(i32, f32);
15
16 // 带有两个字段的结构体
17 struct Point {
18     x: f32,
19     y: f32,
20 }
21
22 // 结构体可以作为另一个结构体的字段
23 struct Rectangle {
24     // 可以通过指定左上角和右下角的位置
25     // 来定义一个矩形。
26     top_left: Point,
27     bottom_right: Point,
28 }
29
30 fn main() {
31     // 使用字段初始化简写语法创建结构体
32     let name = String::from("Peter");
33     let age = 27;
34     let peter = Person { name, age };
35
36     // 打印结构体的调试信息
37     println!("{:?}", peter);
38
39     // 实例化一个 `Point`
40     let point: Point = Point { x: 5.2, y: 0.4 };
41     let another_point: Point = Point { x: 10.3, y: 0.2 };
42
43     // 访问点的字段
44     println!("点的坐标: ({{}}, {{})", point.x, point.y);
45
46     // 使用结构体更新语法创建新的点，
47     // 复用之前创建的点的字段
48     let bottom_right = Point { x: 10.3, ..another_point };
49
50     // `bottom_right.y` 与 `another_point.y` 相同，
51     // 因为我们使用了 `another_point` 的该字段
52     println!("第二个点: ({{}}, {{})", bottom_right.x, bottom_right.y);
53
54     // 使用 `let` 绑定解构点
55     let Point { x: left_edge, y: top_edge } = point;
56
57     let _rectangle = Rectangle {
58         // 结构体实例化也是一个表达式
59         top_left: Point { x: left_edge, y: top_edge },
60         bottom_right: bottom_right
```

```
60         bottom_right: bottom_right,  
61     };  
62  
63     // 实例化一个单元结构体  
64     let _unit = Unit;  
65  
66     // 实例化一个元组结构体  
67     let pair = Pair(1, 0.1);  
68  
69     // 访问元组结构体的字段  
70     println!("pair 包含 {:?} 和 {:?}", pair.0, pair.1);  
71  
72     // 解构一个元组结构体  
73     let Pair(integer, decimal) = pair;  
74  
75     println!("pair 包含 {:?} 和 {:?}", integer, decimal);  
76 }
```

练习

1. 添加一个 `rect_area` 函数来计算 `Rectangle` 的面积（尝试使用嵌套解构）。
2. 添加一个 `square` 函数，它接受一个 `Point` 和一个 `f32` 作为参数，返回一个 `Rectangle`，其左上角在该点上，宽度和高度都等于 `f32` 参数。

另请参阅

[属性](#)、[原始标识符和解构](#)

枚举

`enum` 关键字允许创建一个可能是几种不同变体之一的类型。任何作为 `struct` 有效的变体在 `enum` 中也是有效的。

```
1 // 创建一个 `enum` 来分类网页事件。注意名称和类型信息如何共同指定变体：
2 // `PageLoad != PageUnload` 且 `KeyPress(char) != Paste(String)`。
3 // 每个变体都是不同且独立的。
4 enum WebEvent {
5     // `enum` 变体可以类似单元结构体（`unit-like`），
6     PageLoad,
7     PageUnload,
8     // 类似元组结构体，
9     KeyPress(char),
10    Paste(String),
11    // 或类似 C 语言的结构体。
12    Click { x: i64, y: i64 },
13 }
14
15 // 一个接受 `WebEvent` 枚举作为参数且不返回任何值的函数。
16 fn inspect(event: WebEvent) {
17     match event {
18         WebEvent::PageLoad => println!("页面已加载"),
19         WebEvent::PageUnload => println!("页面已卸载"),
20         // 从 `enum` 变体内部解构 `c`。
21         WebEvent::KeyPress(c) => println!("按下了 '{}' 键。", c),
22         WebEvent::Paste(s) => println!("粘贴了 '{}'。", s),
23         // 将 `Click` 解构为 `x` 和 `y`。
24         WebEvent::Click { x, y } => {
25             println!("点击坐标: x={}, y={}。", x, y);
26         },
27     }
28 }
29
30 fn main() {
31     let pressed = WebEvent::KeyPress('x');
32     // `to_owned()` 从字符串切片创建一个拥有所有权的 `String`。
33     let pasted = WebEvent::Paste("我的文本".to_owned());
34     let click = WebEvent::Click { x: 20, y: 80 };
35     let load = WebEvent::PageLoad;
36     let unload = WebEvent::PageUnload;
37
38     inspect(pressed);
39     inspect(pasted);
40     inspect(click);
41     inspect(load);
42     inspect(unload);
43 }
44
```

类型别名

使用类型别名可以通过别名引用每个枚举变体。当枚举名称过长或过于泛化，而你想重命名它时，这会很有用。

```
1 enum VeryVerboseEnumOfThingsToDoWithNumbers {
2     Add,
3     Subtract,
4 }
5
6 // 创建类型别名
7 type Operations = VeryVerboseEnumOfThingsToDoWithNumbers;
8
9 fn main() {
10     // 我们可以通过别名引用每个变体，而不是使用又长又不便的名称。
11     let x = Operations::Add;
12 }
```

你最常见到这种用法的地方是在使用 `Self` 别名的 `impl` 块中。

```
1 enum VeryVerboseEnumOfThingsToDoWithNumbers {
2     Add,
3     Subtract,
4 }
5
6 impl VeryVerboseEnumOfThingsToDoWithNumbers {
7     fn run(&self, x: i32, y: i32) -> i32 {
8         match self {
9             Self::Add => x + y,
10            Self::Subtract => x - y,
11        }
12    }
13 }
```

要了解更多关于枚举和类型别名的信息，你可以阅读这个特性被稳定到 Rust 时的[稳定化报告](#)。

另请参阅：

[match](#)、[fn](#)、[String](#) 和“类型别名枚举变体”RFC

use

The `use` declaration can be used to avoid typing the full module path to access a name:

```
1  // 这个属性用于隐藏未使用代码的警告。
2  #![allow(dead_code)]
3
4  enum Stage {
5      Beginner,
6      Advanced,
7  }
8
9  enum Role {
10     Student,
11     Teacher,
12 }
13
14 fn main() {
15     // 显式 `use` 每个名称，使它们可以不需要
16     // 手动作用域限定就能使用。
17     use Stage::{Beginner, Advanced};
18     // 自动 `use` `Role` 内的每个名称。
19     use Role::*;
20
21     // 等同于 `Stage::Beginner`。
22     let stage = Beginner;
23     // 等同于 `Role::Student`。
24     let role = Student;
25
26     match stage {
27         // 注意由于上面的显式 `use`，这里不需要作用域限定。
28         Beginner => println!("初学者正在开始他们的学习之旅！"),
29         Advanced => println!("高级学习者正在掌握他们的科目..."),
30     }
31
32     match role {
33         // 再次注意这里不需要作用域限定。
34         Student => println!("学生正在获取知识！"),
35         Teacher => println!("教师正在传播知识！"),
36     }
37 }
```

另请参阅：

`match` 和 `use`

C 风格用法

`enum` 也可以像 C 语言那样使用。

```
1  // 这个属性用于隐藏未使用代码的警告。
2  #![allow(dead_code)]
3
4  // 带隐式判别值的枚举（从 0 开始）
5  enum Number {
6      Zero,
7      One,
8      Two,
9  }
10
11 // 带显式判别值的枚举
12 enum Color {
13     Red = 0xff0000,
14     Green = 0x00ff00,
15     Blue = 0x0000ff,
16 }
17
18 fn main() {
19     // `enum` 可以转换为整数。
20     println!("zero 的值是 {}", Number::Zero as i32);
21     println!("one 的值是 {}", Number::One as i32);
22
23     println!("玫瑰的颜色是 #{:06x}", Color::Red as u32);
24     println!("紫罗兰的颜色是 #{:06x}", Color::Blue as u32);
25 }
```

另请参阅：

[类型转换](#)

测试实例：链表

使用 `enum` 是实现链表的常见方法：

```

1 use crate::List::*;
2
3 enum List {
4     // Cons: 包含一个元素和指向下一节点指针的元组结构体
5     Cons(u32, Box<List>),
6     // Nil: 表示链表末尾的节点
7     Nil,
8 }
9
10 // 可以为枚举实现方法
11 impl List {
12     // 创建空链表
13     fn new() -> List {
14         // `Nil` 的类型是 `List`
15         Nil
16     }
17
18     // 消耗一个链表，并返回在其头部添加新元素后的链表
19     fn prepend(self, elem: u32) -> List {
20         // `Cons` 的类型也是 List
21         Cons(elem, Box::new(self))
22     }
23
24     // 返回链表长度
25     fn len(&self) -> u32 {
26         // 需要对 `self` 进行匹配，因为方法的行为取决于 `self` 的变体
27         // `self` 的类型是 `&List`，而 `*self` 的类型是 `List`
28         // 匹配具体类型 `T` 优于匹配引用 `&T`
29         // 在 Rust 2018 版本后，这里可以直接使用 self，下面可以使用 tail
30         // Rust 会自动推断为 &s 和 ref tail
31         // 参见 https://doc.rust-lang.org/edition-guide/rust-2018/ownership
32         match *self {
33             // Can't take ownership of the tail, because `self` is borrowed
34             // instead take a reference to the tail
35             // And it's a non-tail recursive call which may cause stack overflow
36             Cons(_, ref tail) => 1 + tail.len(),
37             // 基本情况：空链表长度为零
38             Nil => 0
39         }
40     }
41
42     // 返回链表的字符串表示（堆分配）
43     fn stringify(&self) -> String {
44         match *self {
45             Cons(head, ref tail) => {
46                 // `format!` 类似于 `print!`，但返回堆分配的字符串，
47                 // 而不是打印到控制台
48                 format!("{}", head, tail.stringify())
49             },
50             Nil => {
51                 format!("Nil")
52             },
53         }
54     }
55 }
56
57 fn main() {
58     // 创建空链表
59     let mut list = List::new();
60

```



```
61 // 在头部添加一些元素
62 list = list.prepend(1);
63 list = list.prepend(2);
64 list = list.prepend(3);
65
66 // 显示链表的最终状态
67 println!("链表长度为: {}", list.len());
68 println!("{}", list.stringify());
69 }
```

另请参阅：

[Box](#) 和 [方法](#)

常量

Rust 有两种常量类型，可以在任何作用域（包括全局作用域）中声明。两者都需要显式类型标注：

- `const`：不可变值（常见用法）。
- `static`：具有 `'static` 生命周期的可能可变变量。静态生命周期会被自动推断，无需明确指定。访问或修改可变静态变量是 `unsafe` 的。

```
1 // 全局变量在所有其他作用域之外声明。
2 static LANGUAGE: &str = "Rust";
3 const THRESHOLD: i32 = 10;
4
5 fn is_big(n: i32) -> bool {
6     // 在函数中访问常量
7     n > THRESHOLD
8 }
9
10 fn main() {
11     let n = 16;
12
13     // 在主线程中访问常量
14     println!("这是 {}", LANGUAGE);
15     println!("阈值为 {}", THRESHOLD);
16     println!("{}", n, if is_big(n) { "大的" } else { "小的" });
17
18     // 错误！不能修改 `const`。
19     THRESHOLD = 5;
20     // 修复：^ 注释掉此行
21 }
```

另请参阅：

[const / static RFC](#)，`'static` 生命周期

变量绑定

Rust 通过静态类型提供类型安全。变量绑定在声明时可以添加类型注解。然而，在大多数情况下，编译器能够从上下文推断出变量类型，大大减少了类型注解的负担。

可以使用 `let` 关键字将值（如字面量）绑定到变量。

```
1 fn main() {
2     let an_integer = 1u32;
3     let a_boolean = true;
4     let unit = ();
5
6     // 将 `an_integer` 复制到 `copied_integer`
7     let copied_integer = an_integer;
8
9     println!("An integer: {}", copied_integer);
10    println!("A boolean: {}", a_boolean);
11    println!("单元值: {:?}", unit);
12
13    // 编译器会对未使用的变量绑定发出警告
14    // 可以通过在变量名前加下划线来消除这些警告
15    let _unused_variable = 3u32;
16
17    let noisy_unused_variable = 2u32;
18    // 修复: ^ 在变量名前加下划线以消除警告
19    // 注意: 在浏览器中可能不会显示警告
20 }
```

可变性

变量绑定默认是不可变的，但可以使用 `mut` 修饰符来改变这一行为。

```
1 fn main() {  
2     let _immutable_binding = 1;  
3     let mut mutable_binding = 1;  
4  
5     println!("修改前: {}", mutable_binding);  
6  
7     // 正确  
8     mutable_binding += 1;  
9  
10    println!("修改后: {}", mutable_binding);  
11  
12    // 错误! 不能给不可变变量赋新值  
13    _immutable_binding += 1;  
14 }
```

编译器会对可变性错误给出详细的诊断信息。

作用域和遮蔽

变量绑定有作用域，它们被限制在一个**代码块**中生存。代码块是由花括号 `{}` 包围的一系列语句。

```
1 fn main() {
2     // 这个绑定存在于 main 函数中
3     let long_lived_binding = 1;
4
5     // 这是一个代码块，它的作用域比 main 函数小
6     {
7         // 这个绑定只存在于此代码块中
8         let short_lived_binding = 2;
9
10        println!("内部 short: {}", short_lived_binding);
11    }
12    // 代码块结束
13
14    // 错误! `short_lived_binding` 在此作用域中不存在
15    println!("外部 short: {}", short_lived_binding);
16    // 修复: ^ 注释掉此行
17
18    println!("外部 long: {}", long_lived_binding);
19 }
```

此外，Rust 允许**变量遮蔽**。

```
1 fn main() {
2     let shadowed_binding = 1;
3
4     {
5         println!("被遮蔽前: {}", shadowed_binding);
6
7         // 这个绑定*遮蔽*了外部的绑定
8         let shadowed_binding = "abc";
9
10        println!("内部代码块中被遮蔽: {}", shadowed_binding);
11    }
12    println!("内部代码块外: {}", shadowed_binding);
13
14    // 这个绑定*遮蔽*了之前的绑定
15    let shadowed_binding = 2;
16    println!("外部代码块中被遮蔽: {}", shadowed_binding);
17 }
```

先声明

可以先声明变量绑定然后再初始化，但所有变量绑定在使用前必须先初始化：编译器禁止使用未初始化的变量绑定，因为这会导致未定义行为。

在函数中先声明变量绑定而稍后再初始化的做法并不常见。当初始化与声明分离时，读者更难找到初始化的位置。更常见的做法是在变量即将使用的地方附近声明并初始化变量绑定。

```
1 fn main() {
2     // 声明一个变量绑定
3     let a_binding;
4
5     {
6         let x = 2;
7
8         // 初始化绑定
9         a_binding = x * x;
10    }
11
12    println!("绑定: {}", a_binding);
13
14    let another_binding;
15
16    // 错误！使用未初始化的绑定
17    println!("另一个绑定: {}", another_binding);
18    // 修复：^ 注释掉此行
19
20    another_binding = 1;
21
22    println!("另一个绑定: {}", another_binding);
23 }
```

冻结

当数据以相同名称被不可变地绑定时，它也会**冻结**。被冻结的数据在不可变绑定离开作用域之前不能被修改：

```
1 fn main() {
2     let mut _mutable_integer = 7i32;
3
4     {
5         // 通过不可变的 `_mutable_integer` 进行遮蔽
6         let _mutable_integer = _mutable_integer;
7
8         // 错误! `_mutable_integer` 在此作用域中被冻结
9         _mutable_integer = 50;
10        // 修复: ^ 注释掉此行
11
12        // `_mutable_integer` 离开作用域
13    }
14
15    // 正确! `_mutable_integer` 在此作用域中未被冻结
16    _mutable_integer = 3;
17 }
```

类型

Rust 提供了几种机制来更改或定义原生类型和用户定义类型。以下部分将介绍：

- 原生类型之间的[类型转换](#)
- 指定[字面量](#)的类型
- 使用[类型推断](#)
- [类型别名](#)

类型转换

Rust 不支持原始类型之间的隐式类型转换（强制转换）。但可以使用 `as` 关键字进行显式类型转换（转型）。

整数类型之间的转换规则通常遵循 C 语言惯例，但 C 中存在未定义行为的情况除外。在 Rust 中，所有整数类型之间的转换行为都有明确定义。

```

1  // Suppress all errors from casts which overflow.
2  #![allow(overflowing_literals)]
3
4  fn main() {
5      let decimal = 65.4321_f32;
6
7      // 错误！不允许隐式转换
8      let integer: u8 = decimal;
9      // 修复：^ 注释掉此行
10
11     // 显式转换
12     let integer = decimal as u8;
13     let character = integer as char;
14
15     // 错误！转换规则有限制。
16     // 浮点数不能直接转换为字符。
17     let character = decimal as char;
18     // 修复：^ 注释掉此行
19
20     println!("类型转换: {} -> {} -> {}", decimal, integer, character);
21
22     // when casting any value to an unsigned type, T,
23     // T::MAX + 1 is added or subtracted until the value
24     // fits into the new type ONLY when the #![allow(overflowing_literals,
25     // lint is specified like above. Otherwise there will be a compiler e
26
27     // 1000 已经适合 u16
28     println!("1000 转换为 u16 是: {}", 1000 as u16);
29
30     // 1000 - 256 - 256 - 256 = 232
31     // 实际上，保留了最低有效位（LSB）的前 8 位，
32     // 而朝最高有效位（MSB）方向的其余位被截断。
33     println!("1000 转换为 u8 是: {}", 1000 as u8);
34     // -1 + 256 = 255
35     println!(" -1 转换为 u8 是: {}", (-1i8) as u8);
36
37     // 对于正数，这等同于取模运算
38     println!("1000 对 256 取模是: {}", 1000 % 256);
39
40     // 当转换为有符号类型时，（按位）结果等同于
41     // 先转换为对应的无符号类型。如果该值的最高有效位
42     // 为 1，则该值为负数。
43
44     // 当然，如果已经适合的话就不需要转换。
45     println!(" 128 转换为 i16 是: {}", 128 as i16);
46
47     // 边界情况：128 在 8 位二进制补码表示中为 -128
48     println!(" 128 转换为 i8 是: {}", 128 as i8);
49
50     // 重复上面的例子
51     // 1000 转换为 u8 -> 232
52     println!("1000 转换为 u8 是: {}", 1000 as u8);
53     // 而 232 在 8 位二进制补码表示中为 -24
54     println!(" 232 转换为 i8 是: {}", 232 as i8);
55
56     // 从 Rust 1.45 开始，`as` 关键字在浮点数转整数时执行*饱和转换*
57     // 如果浮点值超出上界或低于下界，返回值
58     // 将等于所越过的边界值。
59
60     // 200.0 转换为 u8 且 255

```

```
60 // 300.0 转换为 u8 是 255
61 println!(" 300.0 转换为 u8 是: {}", 300.0_f32 as u8);
62 // -100.0 转换为 u8 是 0
63 println!("-100.0 转换为 u8 是: {}", -100.0_f32 as u8);
64 // NaN 转换为 u8 是 0
65 println!("   NaN 转换为 u8 是: {}", f32::NAN as u8);
66
67 // 这种行为会产生少量运行时开销，可以通过不安全方法避免，
68 // 但结果可能溢出并返回**不可靠的值**。请谨慎使用这些方法：
69 unsafe {
70     // 300.0 转换为 u8 是 44
71     println!(" 300.0 转换为 u8 是: {}", 300.0_f32.to_int_unchecked::<u8>());
72     // -100.0 转换为 u8 是 156
73     println!("-100.0 转换为 u8 是: {}", (-100.0_f32).to_int_unchecked::<u8>());
74     // NaN 转换为 u8 是 0
75     println!("   NaN 转换为 u8 是: {}", f32::NAN.to_int_unchecked::<u8>());
76 }
77 }
```

字面量

数字字面值可以通过添加类型后缀进行类型标注。例如，要指定字面值 `42` 的类型为 `i32`，可以写成 `42i32`。

无后缀数字字面值的类型取决于其使用方式。如果没有约束，编译器将对整数使用 `i32`，对浮点数使用 `f64`。

```
1 fn main() {
2     // 带后缀的字面值，其类型在初始化时确定
3     let x = 1u8;
4     let y = 2u32;
5     let z = 3f32;
6
7     // 无后缀的字面值，其类型取决于使用方式
8     let i = 1;
9     let f = 1.0;
10
11    // `size_of_val` 返回变量的字节大小
12    println!("`x` 的字节大小: {}", std::mem::size_of_val(&x));
13    println!("`y` 的字节大小: {}", std::mem::size_of_val(&y));
14    println!("`z` 的字节大小: {}", std::mem::size_of_val(&z));
15    println!("`i` 的字节大小: {}", std::mem::size_of_val(&i));
16    println!("`f` 的字节大小: {}", std::mem::size_of_val(&f));
17 }
```

前面的代码中使用了一些尚未解释的概念。为了满足迫不及待的读者，这里简要说明如下：

- `std::mem::size_of_val` 是一个函数，这里使用了它的“完整路径”来调用。代码可以被划分为称为“模块”的逻辑单元。在这个例子中，`size_of_val` 函数定义在 `mem` 模块中，而 `mem` 模块则定义在 `std` crate 中。更多详情请参阅[模块](#)和[crate](#)。

类型推断

类型推断引擎相当智能。它不仅在初始化时分析值表达式的类型，还会根据变量后续的使用方式来推断其类型。下面是一个类型推断的高级示例：

```
1 fn main() {
2     // 通过类型注解，编译器得知 `elem` 的类型为 u8
3     let elem = 5u8;
4
5     // 创建一个空向量（可增长的数组）
6     let mut vec = Vec::new();
7     // 此时编译器还不知道 `vec` 的具体类型，
8     // 只知道它是某种类型的向量（`Vec<_>`）。
9
10    // 将 `elem` 插入向量中
11    vec.push(elem);
12    // 啊哈！现在编译器知道 `vec` 是 `u8` 类型的向量（`Vec<u8>`）
13    // TODO ^ 尝试注释掉 `vec.push(elem)` 这一行
14
15    println!("{:?}", vec);
16 }
```

无需为变量添加类型注解，编译器和程序员都很满意！

别名

`type` 语句用于为现有类型创建新名称。类型名必须使用 `UpperCamelCase`（大驼峰）命名，否则编译器会发出警告。此规则的例外是原始类型，如 `usize`、`f32` 等。

```
1  // `NanoSecond`、`Inch` 和 `U64` 都是 `u64` 的新名称。
2  type NanoSecond = u64;
3  type Inch = u64;
4  type U64 = u64;
5
6  fn main() {
7      // `NanoSecond` = `Inch` = `U64` = `u64`。
8      let nanoseconds: NanoSecond = 5 as u64;
9      let inches: Inch = 2 as U64;
10
11     // 注意，类型别名*不会*提供额外的类型安全性，因为别名*不是*新类型
12     println!("{}", 纳秒 + {} 英寸 = {} 单位?",
13               nanoseconds,
14               inches,
15               nanoseconds + inches);
16 }
```

别名的主要用途是减少重复代码。例如，`io::Result<T>` 类型是 `Result<T, io::Error>` 类型的别名。

另请参阅：

属性

转换

原始类型可以通过[类型转换](#)相互转换。

Rust 通过使用[特质](#)来处理自定义类型（如 `struct` 和 `enum`）之间的转换。通用转换使用 [From](#) 和 [Into](#) 特质。然而，对于更常见的情况，特别是与 `String` 相互转换时，还有一些更具体的特质。

From 和 Into

`From` 和 `Into` 特质本质上是相互关联的，这实际上是其实现的一部分。如果你能将类型 A 从类型 B 转换，那么我们也应该能够将类型 B 转换为类型 A。

From

`From` 特质允许一个类型定义如何从另一个类型创建自身，从而提供了一种非常简单的机制来在多种类型之间进行转换。标准库中有许多这个特质的实现，用于原始类型和常见类型的转换。

例如，我们可以轻松地将 `str` 转换为 `String`

```
let my_str = "hello";
let my_string = String::from(my_str);
```

我们可以为自己的类型定义类似的转换。

```
1 use std::convert::From;
2
3 #[derive(Debug)]
4 struct Number {
5     value: i32,
6 }
7
8 impl From<i32> for Number {
9     fn from(item: i32) -> Self {
10         Number { value: item }
11     }
12 }
13
14 fn main() {
15     let num = Number::from(30);
16     println!("我的数字是 {:?}", num);
17 }
```

Into

`Into` 特质简单来说就是 `From` 特质的反向操作。它定义了如何将一个类型转换为另一个类型。

调用 `into()` 通常需要我们指定结果类型，因为编译器大多数时候无法确定这一点。


```
1 use std::convert::Into;
2
3 #[derive(Debug)]
4 struct Number {
5     value: i32,
6 }
7
8 impl Into<Number> for i32 {
9     fn into(self) -> Number {
10         Number { value: self }
11     }
12 }
13
14 fn main() {
15     let int = 5;
16     // 尝试移除类型标注
17     let num: Number = int.into();
18     println!("我的数字是 {:?}", num);
19 }
```

From 和 Into 是可互换的

From 和 Into 被设计为互补的。我们不需要为两个特质都提供实现。如果你为你的类型实现了 From 特质，Into 会在必要时调用它。但请注意，反过来并不成立：为你的类型实现 Into 不会自动为它提供 From 的实现。

```
1 use std::convert::From;
2
3 #[derive(Debug)]
4 struct Number {
5     value: i32,
6 }
7
8 // 定义 `From`
9 impl From<i32> for Number {
10     fn from(item: i32) -> Self {
11         Number { value: item }
12     }
13 }
14
15 fn main() {
16     let int = 5;
17     // 使用 `Into`
18     let num: Number = int.into();
19     println!("我的数字是 {:?}", num);
20 }
```

TryFrom 和 TryInto

与 `From` 和 `Into` 类似，`TryFrom` 和 `TryInto` 是用于类型转换的泛型特质。与 `From` / `Into` 不同，`TryFrom` / `TryInto` 特质用于可能失败的转换，因此返回 `Result`。

```
1 use std::convert::TryFrom;
2 use std::convert::TryInto;
3
4 #[derive(Debug, PartialEq)]
5 struct EvenNumber(i32);
6
7 impl TryFrom<i32> for EvenNumber {
8     type Error = ();
9
10    fn try_from(value: i32) -> Result<Self, Self::Error> {
11        if value % 2 == 0 {
12            Ok(EvenNumber(value))
13        } else {
14            Err(())
15        }
16    }
17 }
18
19 fn main() {
20     // TryFrom
21
22     assert_eq!(EvenNumber::try_from(8), Ok(EvenNumber(8)));
23     assert_eq!(EvenNumber::try_from(5), Err(()));
24
25     // TryInto
26
27     let result: Result<EvenNumber, ()> = 8i32.try_into();
28     assert_eq!(result, Ok(EvenNumber(8)));
29     let result: Result<EvenNumber, ()> = 5i32.try_into();
30     assert_eq!(result, Err(()));
31 }
```

String 类型转换

转换为字符串

要将任何类型转换为 `String`，只需为该类型实现 `ToString` 特质即可。但更好的做法是实现 `fmt::Display` 特质，它不仅会自动提供 `ToString`，还允许打印该类型，就像在 `print!` 部分讨论的那样。

```
1 use std::fmt;
2
3 struct Circle {
4     radius: i32
5 }
6
7 impl fmt::Display for Circle {
8     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
9         write!(f, "半径为 {} 的圆", self.radius)
10    }
11 }
12
13 fn main() {
14     let circle = Circle { radius: 6 };
15     println!("{}", circle.to_string());
16 }
```

解析字符串

将字符串转换为其他类型很有用，其中最常见的操作之一是将字符串转换为数字。惯用的方法是使用 `parse` 函数，可以通过类型推断或使用“涡轮鱼”语法指定要解析的类型。以下示例展示了这两种方法。

只要为目标类型实现了 `FromStr` 特质，就可以将字符串转换为指定的类型。标准库中为许多类型实现了这个特质。

```
1 fn main() {
2     let parsed: i32 = "5".parse().unwrap();
3     let turbo_parsed = "10".parse::<i32>().unwrap();
4
5     let sum = parsed + turbo_parsed;
6     println!("总和: {:?}", sum);
7 }
```

要在自定义类型上获得这个功能，只需为该类型实现 `FromStr` 特质。

```
1 use std::num::ParseIntError;
2 use std::str::FromStr;
3
4 #[derive(Debug)]
5 struct Circle {
6     radius: i32,
7 }
8
9 impl FromStr for Circle {
10     type Err = ParseIntError;
11     fn from_str(s: &str) -> Result<Self, Self::Err> {
12         match s.trim().parse() {
13             Ok(num) => Ok(Circle{ radius: num }),
14             Err(e) => Err(e),
15         }
16     }
17 }
18
19 fn main() {
20     let radius = "  3 ";
21     let circle: Circle = radius.parse().unwrap();
22     println!("{:?}", circle);
23 }
```

表达式

Rust 程序（主要）由一系列语句组成：

```
1 fn main() {  
2     // 语句  
3     // 语句  
4     // 语句  
5 }
```

Rust 中有几种语句。最常见的两种是声明变量绑定，以及在表达式后使用分号 `;`：

```
1 fn main() {  
2     // 变量绑定  
3     let x = 5;  
4  
5     // 表达式;  
6     x;  
7     x + 1;  
8     15;  
9 }
```

代码块也是表达式，因此可以在赋值中作为值使用。代码块中的最后一个表达式会被赋值给左值表达式（如局部变量）。但是，如果代码块的最后一个表达式以分号结尾，返回值将是 `()`。

```
1 fn main() {  
2     let x = 5u32;  
3  
4     let y = {  
5         let x_squared = x * x;  
6         let x_cubed = x_squared * x;  
7  
8         // 这个表达式将被赋值给 `y`  
9         x_cubed + x_squared + x  
10    };  
11  
12    let z = {  
13        // 分号抑制了这个表达式，`() ` 被赋值给 `z`  
14        2 * x;  
15    };  
16  
17    println!("x 是 {:?}", x);  
18    println!("y 是 {:?}", y);  
19    println!("z 是 {:?}", z);  
20 }
```

控制流

控制流是任何编程语言的重要组成部分，如 `if/else`、`for` 等。让我们来讨论 Rust 中的这些内容。

if/else

`if-else` 分支结构与其他语言类似。不同之处在于，布尔条件不需要用括号括起来，每个条件后面都跟着一个代码块。`if-else` 条件是表达式，所有分支必须返回相同的类型。

```
1 fn main() {
2     let n = 5;
3
4     if n < 0 {
5         print!("{}", 是负数", n);
6     } else if n > 0 {
7         print!("{}", 是正数", n);
8     } else {
9         print!("{}", 是零", n);
10    }
11
12    let big_n =
13        if n < 10 && n > -10 {
14            println!("{}", 是一个小数字，扩大十倍");
15
16            // 这个表达式返回 `i32` 类型。
17            10 * n
18        } else {
19            println!("{}", 是一个大数字，将数字减半");
20
21            // 这个表达式也必须返回 `i32` 类型。
22            n / 2
23            // TODO ^ 尝试用分号结束这个表达式。
24        };
25    // ^ 别忘了在这里加分号！所有 `let` 绑定都需要它。
26
27    println!("{}", -> {}", n, big_n);
28 }
```

loop

Rust 提供 `loop` 关键字来表示无限循环。

`break` 语句可以随时退出循环，而 `continue` 语句可以跳过当前迭代的剩余部分并开始下一次迭代。

```
1 fn main() {
2     let mut count = 0u32;
3
4     println!("让我们数到无穷大！");
5
6     // 无限循环
7     loop {
8         count += 1;
9
10        if count == 3 {
11            println!("three");
12
13            // 跳过本次迭代的剩余部分
14            continue;
15        }
16
17        println!("{}", count);
18
19        if count == 5 {
20            println!("好了，够了");
21
22            // 退出这个循环
23            break;
24        }
25    }
26 }
```


嵌套和标签

在处理嵌套循环时，可以 `break` 或 `continue` 外层循环。这种情况下，循环必须用 `'label` 标记，并且必须将标签传递给 `break / continue` 语句。

```
1  #![allow(unreachable_code, unused_labels)]
2
3  fn main() {
4      'outer: loop {
5          println!("进入外层循环");
6
7          'inner: loop {
8              println!("进入内层循环");
9
10             // 这只会中断内层循环
11             //break;
12
13             // 这会中断外层循环
14             break 'outer;
15         }
16
17         println!("这一点永远不会到达");
18     }
19
20     println!("退出外层循环");
21 }
```

在 loop 中返回值

`loop` 的一个用途是重试操作直到成功。如果操作返回一个值，你可能需要将它传递给代码的其余部分：将它放在 `break` 之后，它将被 `loop` 表达式返回。

```
1 fn main() {  
2     let mut counter = 0;  
3  
4     let result = loop {  
5         counter += 1;  
6  
7         if counter == 10 {  
8             break counter * 2;  
9         }  
10    };  
11  
12    assert_eq!(result, 20);  
13 }
```

while

`while` 关键字用于在条件为真时运行循环。

让我们用 `while` 循环来编写著名的 `FizzBuzz` 程序。

```
1 fn main() {
2     // 计数器变量
3     let mut n = 1;
4
5     // 当 `n` 小于 101 时继续循环
6     while n < 101 {
7         if n % 15 == 0 {
8             println!("fizzbuzz");
9         } else if n % 3 == 0 {
10            println!("fizz");
11        } else if n % 5 == 0 {
12            println!("buzz");
13        } else {
14            println!("{}", n);
15        }
16
17        // 计数器递增
18        n += 1;
19    }
20 }
```

for 循环

for 和 range

`for in` 结构可用于遍历 `Iterator`。创建迭代器最简单的方法之一是使用区间表示法 `a..b`。这会生成从 `a`（包含）到 `b`（不包含）的值，步长为 1。

让我们用 `for` 而不是 `while` 来编写 FizzBuzz。

```
1 fn main() {
2     // `n` 在每次迭代中将取值: 1, 2, ..., 100
3     for n in 1..101 {
4         if n % 15 == 0 {
5             println!("fizzbuzz");
6         } else if n % 3 == 0 {
7             println!("fizz");
8         } else if n % 5 == 0 {
9             println!("buzz");
10        } else {
11            println!("{}", n);
12        }
13    }
14 }
```

另外，可以使用 `a..=b` 表示两端都包含的范围。上面的代码可以改写为：

```
1 fn main() {
2     // `n` 在每次迭代中将取值: 1, 2, ..., 100
3     for n in 1..=100 {
4         if n % 15 == 0 {
5             println!("fizzbuzz");
6         } else if n % 3 == 0 {
7             println!("fizz");
8         } else if n % 5 == 0 {
9             println!("buzz");
10        } else {
11            println!("{}", n);
12        }
13    }
14 }
```

Just remember that even though you can compile the code when `a>b`, the loop gets never executed.

```
1 for i in 10..1{
2     println!("fizzbuzz");
3 }
```

If you want to count down, you need to use `.rev()` instead

```
1 for i in (1..10).rev(){
2     println!("fizzbuzz");
3 }
```

for 与迭代器

`for in` 结构能以多种方式与 `Iterator` 交互。正如在 [Iterator](#) 特质一节中讨论的那样，默认情况下 `for` 循环会对集合应用 `into_iter` 函数。然而，这并不是将集合转换为迭代器的唯一方法。

`into_iter`、`iter` 和 `iter_mut` 都以不同的方式处理集合到迭代器的转换，通过提供对数据的不同视图。

- `iter` - 在每次迭代中借用集合的每个元素。因此，集合保持不变，并且在循环之后可以重复使用。

```
1 fn main() {
2     let names = vec!["Bob", "Frank", "Ferris"];
3
4     for name in names.iter() {
5         match name {
6             &"Ferris" => println!("我们中间有一个 Rustacean! "),
7             // TODO ^ 尝试删除 & 并只匹配 "Ferris"
8             _ => println!("你好 {}", name),
9         }
10    }
11
12    println!("names: {:?}", names);
13 }
```

- `into_iter` - 这会消耗集合，使得在每次迭代中提供确切的数据。一旦集合被消耗，它就不再可用于重复使用，因为它已经在循环中被“移动”了。

```
1 fn main() {
2     let names = vec!["Bob", "Frank", "Ferris"];
3
4     for name in names.into_iter() {
5         match name {
6             "Ferris" => println!("我们中间有一个 Rustacean! "),
7             _ => println!("你好 {}", name),
8         }
9     }
10
11    // `names` has been 'moved' and can no longer be used.
12    // Try uncommenting the line below to see the compiler error:
13    // println!("names: {:?}", names);
14 }
```

- `iter_mut` - 这会可变地借用集合的每个元素，允许在原地修改集合。

```
1 fn main() {  
2     let mut names = vec!["Bob", "Frank", "Ferris"];  
3  
4     for name in names.iter_mut() {  
5         *name = match name {  
6             &mut "Ferris" => "我们中间有一个 Rustacean! ",  
7             _ => "你好",  
8         }  
9     }  
10  
11     println!("names: {:?}", names);  
12 }
```

在上面的代码片段中，注意 `match` 分支的类型，这是迭代类型的关键区别。类型的差异意味着可以执行不同的操作。

另请参阅：

[迭代器](#)

match

Rust 通过 `match` 关键字提供模式匹配，类似于 C 语言的 `switch`。第一个匹配的分支会被求值，并且必须覆盖所有可能的值。

```
1 fn main() {
2     let number = 13;
3     // TODO ^ 尝试为 `number` 赋不同的值
4
5     println!("告诉我关于 {} 的信息", number);
6     match number {
7         // 匹配单个值
8         1 => println!("一! "),
9         // 匹配多个值
10        2 | 3 | 5 | 7 | 11 => println!("这是个质数"),
11        // TODO ^ 尝试将 13 添加到质数列表中
12        // 匹配一个闭区间范围
13        13..=19 => println!("一个青少年"),
14        // 处理其余情况
15        _ => println!("没什么特别的"),
16        // TODO ^ 尝试注释掉这个匹配所有情况的分支
17    }
18
19    let boolean = true;
20    // match 也是一个表达式
21    let binary = match boolean {
22        // match 的分支必须覆盖所有可能的值
23        false => 0,
24        true => 1,
25        // TODO ^ 尝试注释掉其中一个分支
26    };
27
28    println!("{}", boolean, binary);
29 }
```

解构

`match` 块可以以多种方式解构项。

- [解构元组](#)
- [解构数组和切片](#)
- [解构枚举](#)
- [解构指针](#)
- [解构结构体](#)

另请参阅：

[The Rust Reference for Destructuring](#)

元组

元组可以在 `match` 中按如下方式解构：

```
1 fn main() {
2     let triple = (0, -2, 3);
3     // TODO ^ 尝试为 `triple` 赋不同的值
4
5     println!("告诉我关于 {:?} 的信息", triple);
6     // match 可用于解构元组
7     match triple {
8         // 解构第二和第三个元素
9         (0, y, z) => println!("第一个是 `0`, `y` 是 {:?}, `z` 是 {:?}", y, z),
10        (1, ..)  => println!("第一个是 `1`, 其余的不重要"),
11        (.., 2)  => println!("最后一个为 `2`, 其余的不重要"),
12        (3, .., 4) => println!("第一个是 `3`, 最后一个为 `4`, 其余的不重要"),
13        // `..` 可用于忽略元组中的其余部分
14        _       => println!("它们是什么并不重要"),
15        // `_` 表示不将值绑定到变量
16    }
17 }
```

另请参阅：

[元组](#)

数组/切片

与元组类似，数组和切片也可以用这种方式解构：

```
1 fn main() {
2     // 尝试改变数组中的值，或将其变成切片！
3     let array = [1, -2, 6];
4
5     match array {
6         // 将第二个和第三个元素分别绑定到相应的变量
7         [0, second, third] =>
8             println!("array[0] = 0, array[1] = {}, array[2] = {}", second,
9
10            // 单个值可以用 _ 忽略
11            [1, _, third] => println!(
12                "array[0] = 1, array[2] = {}, array[1] 被忽略了",
13                third
14            ),
15
16            // 你也可以绑定一部分值并忽略其余的
17            [-1, second, ..] => println!(
18                "array[0] = -1, array[1] = {}, 其他所有的都被忽略了",
19                second
20            ),
21            // 下面的代码无法编译
22            // [-1, second] => ...
23
24            // 或者将它们存储在另一个数组/切片中（类型取决于
25            // 正在匹配的值的类型）
26            [3, second, tail @ ..] => println!(
27                "array[0] = 3, array[1] = {}, 其他元素是 {:?}",
28                second, tail
29            ),
30
31            // 结合这些模式，我们可以，例如，绑定第一个和
32            // 最后一个值，并将其余的存储在一个单独的数组中
33            [first, middle @ .., last] => println!(
34                "array[0] = {}, 中间部分 = {:?}", array[2] = {}",
35                first, middle, last
36            ),
37    }
38 }
```

另请参阅：

关于 @ 符号的更多信息，请参阅[数组和切片](#)和[绑定](#)

枚举

enum 的解构方式类似：

```
1 // 使用 `allow` 来抑制警告，因为只使用了一个变体。
2 #[allow(dead_code)]
3 enum Color {
4     // 这 3 个仅通过名称指定。
5     Red,
6     Blue,
7     Green,
8     // 这些同样将 `u32` 元组与不同的名称（颜色模型）关联。
9     RGB(u32, u32, u32),
10    HSV(u32, u32, u32),
11    HSL(u32, u32, u32),
12    CMY(u32, u32, u32),
13    CMYK(u32, u32, u32, u32),
14 }
15
16 fn main() {
17     let color = Color::RGB(122, 17, 40);
18     // TODO ^ 尝试为 `color` 使用不同的变体
19
20     println!("这是什么颜色？");
21     // 可以使用 `match` 来解构 `enum`。
22     match color {
23         Color::Red => println!("颜色是红色！"),
24         Color::Blue => println!("颜色是蓝色！"),
25         Color::Green => println!("颜色是绿色！"),
26         Color::RGB(r, g, b) =>
27             println!("红：{}, 绿：{}, 蓝：{}!", r, g, b),
28         Color::HSV(h, s, v) =>
29             println!("色相：{}, 饱和度：{}, 明度：{}!", h, s, v),
30         Color::HSL(h, s, l) =>
31             println!("色相：{}, 饱和度：{}, 亮度：{}!", h, s, l),
32         Color::CMY(c, m, y) =>
33             println!("青：{}, 品红：{}, 黄：{}!", c, m, y),
34         Color::CMYK(c, m, y, k) =>
35             println!("青：{}, 品红：{}, 黄：{}, 黑（K）：{}!",
36                 c, m, y, k),
37         // 不需要其他分支，因为所有变体都已检查
38     }
39 }
```

另请参阅：

[#\[allow\(...\)\]](#)、[颜色模型](#)和 [enum](#)

指针/引用

对于指针，需要区分解构和解引用，因为它们是不同的概念，其用法与 C/C++ 等语言不同。

- 解引用使用 `*`
- 解构使用 `&`、`ref` 和 `ref mut`

```
1 fn main() {
2     // 分配一个 `i32` 类型的引用。`&` 表示
3     // 正在分配一个引用。
4     let reference = &4;
5
6     match reference {
7         // 如果 `reference` 与 `&val` 进行模式匹配，结果
8         // 就像这样的比较：
9         // `&i32`
10        // `&val`
11        // ^ 我们可以看到，如果去掉匹配的 `&`，那么 `i32`
12        // 应该被赋值给 `val`。
13        &val => println!("通过解构获得的值：{:?}", val),
14    }
15
16    // 为了避免 `&`，你可以在匹配前解引用。
17    match *reference {
18        val => println!("通过解引用获得的值：{:?}", val),
19    }
20
21    // 如果你一开始没有引用怎么办？`reference` 是一个 `&`
22    // 因为右侧已经是一个引用。这不是
23    // 一个引用，因为右侧不是引用。
24    let _not_a_reference = 3;
25
26    // Rust 提供 `ref` 正是为了这个目的。它修改了
27    // 赋值，为元素创建一个引用；
28    // 这个引用被赋值。
29    let ref _is_a_reference = 3;
30
31    // 相应地，通过定义两个没有引用的值，
32    // 可以通过 `ref` 和 `ref mut` 获取引用。
33    let value = 5;
34    let mut mut_value = 6;
35
36    // 使用 `ref` 关键字创建引用。
37    match value {
38        ref r => println!("获得了一个值的引用：{:?}", r),
39    }
40
41    // 类似地使用 `ref mut`。
42    match mut_value {
43        ref mut m => {
44            // 获得了一个引用。在我们能够
45            // 对其进行任何添加操作之前，必须先解引用。
46            *m += 10;
47            println!("我们加了 10。`mut_value`：{:?}", m);
48        },
49    }
50 }
```

另请参阅：

ref 模式

结构体

同样，`struct` 可以按如下方式解构：

```

1 fn main() {
2     struct Foo {
3         x: (u32, u32),
4         y: u32,
5     }
6
7     // 尝试更改结构体中的值，看看会发生什么
8     let foo = Foo { x: (1, 2), y: 3 };
9
10    match foo {
11        Foo { x: (1, b), y } => println!("x 的第一个元素是 1, b = {}, y = ",
12
13        // 你可以解构结构体并重命名变量，
14        // 顺序并不重要
15        Foo { y: 2, x: i } => println!("y 为 2, i = {:?}", i),
16
17        // 你也可以忽略某些变量：
18        Foo { y, .. } => println!("y = {}, 我们不关心 x 的值", y),
19        // 这会导致错误：模式中未提及字段 `x`
20        // Foo { y } => println!("y = {}", y),
21    }
22
23    let faa = Foo { x: (1, 2), y: 3 };
24
25    // 解构结构体不一定需要 match 块：
26    let Foo { x : x0, y: y0 } = faa;
27    println!("外部: x0 = {x0:?}, y0 = {y0:?}");
28
29    // 解构也适用于嵌套结构体：
30    struct Bar {
31        foo: Foo,
32    }
33
34    let bar = Bar { foo: faa };
35    let Bar { foo: Foo { x: nested_x, y: nested_y } } = bar;
36    println!("嵌套: nested_x = {nested_x:?}, nested_y = {nested_y:?}");
37 }

```

另请参阅：

[结构体](#)

守卫

`match` 分支可以使用**守卫**进行额外的筛选。

```
1 #[allow(dead_code)]
2 enum Temperature {
3     Celsius(i32),
4     Fahrenheit(i32),
5 }
6
7 fn main() {
8     let temperature = Temperature::Celsius(35);
9     // ^ TODO: 尝试为 `temperature` 赋予不同的值
10
11     match temperature {
12         Temperature::Celsius(t) if t > 30 => println!("{}", °C 高于 30°C", t),
13         // `if condition` 部分 ^ 就是守卫
14         Temperature::Celsius(t) => println!("{}", °C 不高于 30°C", t),
15
16         Temperature::Fahrenheit(t) if t > 86 => println!("{}", °F 高于 86°F", t),
17         Temperature::Fahrenheit(t) => println!("{}", °F 不高于 86°F", t),
18     }
19 }
```

注意，编译器在检查 `match` 表达式是否涵盖了所有模式时，不会考虑守卫条件。

```
1 fn main() {
2     let number: u8 = 4;
3
4     match number {
5         i if i == 0 => println!("零"),
6         i if i > 0 => println!("大于零"),
7         // _ => unreachable!("不应该发生。"),
8         // TODO ^ 取消注释以修复编译错误
9     }
10 }
```

另请参阅：

[元组 枚举](#)

绑定

间接访问变量时，无法在分支中使用该变量而不重新绑定。match 提供了 @ 符号，用于将值绑定到名称：

```
1 // 一个返回 `u32` 的 `age` 函数。
2 fn age() -> u32 {
3     15
4 }
5
6 fn main() {
7     println!("告诉我你是什么类型的人");
8
9     match age() {
10         0 => println!("我还没有过第一个生日"),
11         // 我们可以直接 `match` 1 ..= 12, 但那样的话如何确定
12         // 孩子的确切年龄呢?
13         // 可以 `match` n 并使用 `if` 守卫, 但这并不是
14         // 穷尽性检查。
15         // (虽然在这种情况下无关紧要, 因为底部有一个
16         // "兜底" 模式)
17         // 因此, 将 1 ..= 12 序列绑定到 `n`。
18         // 现在可以报告年龄了。
19         n @ 1 ..= 12 => println!("我是 {:?} 岁的儿童", n),
20         n @ 13 ..= 19 => println!("我是 {:?} 岁的青少年", n),
21         // A similar binding can be done when matching several values.
22         n @ (1 | 7 | 15 | 13) => println!("我是 {:?} 岁的青少年", n),
23         // 没有绑定。直接返回结果。
24         n => println!("我是 {:?} 岁的成年人", n),
25     }
26 }
```

你也可以使用绑定来“解构” enum 变体，例如 Option：


```
1 fn some_number() -> Option<u32> {  
2     Some(42)  
3 }  
4  
5 fn main() {  
6     match some_number() {  
7         // Got `Some` variant, match if its value, bound to `n`,  
8         // is equal to 42.  
9         // Could also use `Some(42)` and print `"The Answer: 42!"`  
10        // but that would require changing `42` in 2 spots should  
11        // you ever wish to change it.  
12        // Could also use `Some(n) if n == 42` and print `"The Answer: {n}`  
13        // but that would not contribute to exhaustiveness checks.  
14        // (Although in this case that would not matter since  
15        // the next arm is a "catch-all" pattern)  
16        Some(n @ 42) => println!("答案是: {}!", n),  
17        // 匹配任何其他数字  
18        Some(n)      => println!("不感兴趣... {}", n),  
19        // 匹配其他任何情况（`None` 变体）  
20        _            => (),  
21    }  
22 }
```

另请参阅：

[函数](#)、[枚举](#) 和 [Option](#)

if let

在某些情况下，使用 `match` 匹配枚举可能会显得繁琐。例如：

```
// 创建 `Option<i32>` 类型的 `optional`
let optional = Some(7);

match optional {
    Some(i) => println!("这是一个很长的字符串，其中包含 `{:?}`", i),
    _ => {},
    // ^ 这是必需的，因为 `match` 要求穷举所有情况。
    // 是不是觉得有些浪费空间？
};
```

对于这种情况，`if let` 更加简洁，而且还允许指定各种失败时的处理选项：

```
1 fn main() {
2     // 以下都是 `Option<i32>` 类型
3     let number = Some(7);
4     let letter: Option<i32> = None;
5     let emoticon: Option<i32> = None;
6
7     // `if let` 结构的含义是：如果 `let` 能将 `number` 解构为
8     // `Some(i)`，则执行代码块（`{}`）。
9     if let Some(i) = number {
10         println!("匹配到 {:?}!", i);
11     }
12
13     // 如果需要指定匹配失败的情况，可以使用 else：
14     if let Some(i) = letter {
15         println!("匹配到 {:?}!", i);
16     } else {
17         // 解构失败。转到失败处理的情况。
18         println!("没有匹配到数字。那就用一个字母吧！");
19     }
20
21     // 提供一个修改后的失败条件。
22     let i_like_letters = false;
23
24     if let Some(i) = emoticon {
25         println!("匹配到 {:?}!", i);
26         // 解构失败。评估 `else if` 条件，看是否应该执行替代的失败分支：
27     } else if i_like_letters {
28         println!("没有匹配到数字。那就用一个字母吧！");
29     } else {
30         // 条件判断为假。这个分支是默认情况：
31         println!("我不喜欢字母。那就用个表情符号吧 :)!");
32     }
33 }
```

同样地，`if let` 可以用来匹配任何枚举值：

```
1 // 我们的示例枚举
2 enum Foo {
3     Bar,
4     Baz,
5     Qux(u32)
6 }
7
8 fn main() {
9     // 创建示例变量
10    let a = Foo::Bar;
11    let b = Foo::Baz;
12    let c = Foo::Qux(100);
13
14    // 变量 a 匹配 Foo::Bar
15    if let Foo::Bar = a {
16        println!("a 是 foobar");
17    }
18
19    // 变量 b 不匹配 Foo::Bar
20    // 所以这里不会打印任何内容
21    if let Foo::Bar = b {
22        println!("b 是 foobar");
23    }
24
25    // 变量 c 匹配 Foo::Qux, 它包含一个值
26    // 类似于前面例子中的 Some()
27    if let Foo::Qux(value) = c {
28        println!("c 是 {}", value);
29    }
30
31    // `if let` 也可以进行绑定
32    if let Foo::Qux(value @ 100) = c {
33        println!("c 是一百");
34    }
35 }
```

`if let` 的另一个优点是它允许我们匹配非参数化的枚举变体。即使在枚举没有实现或派生 `PartialEq` 的情况下也是如此。在这种情况下, `if Foo::Bar == a` 将无法编译, 因为枚举的实例无法进行相等比较, 但 `if let` 仍然可以正常工作。

想要挑战一下吗? 请修改以下示例, 使用 `if let`:

```
1 // 这个枚举故意既不实现也不派生 PartialEq。
2 // 这就是为什么下面比较 Foo::Bar == a 会失败。
3 enum Foo {Bar}
4
5 fn main() {
6     let a = Foo::Bar;
7
8     // 变量 a 匹配 Foo::Bar
9     if Foo::Bar == a {
10        // ^-- 这会导致编译时错误。请改用 `if let`。
11        println!("a 是 foobar");
12    }
13 }
```

另请参阅：

[枚举](#)、[Option](#) 和 [RFC](#)

let-else

① 自 Rust 1.65 版本起稳定

① 你可以通过这种方式编译来指定特定版本: `rustc --edition=2021 main.rs`

`let-else` 语法允许可能失败的模式匹配像普通 `let` 一样绑定变量到当前作用域, 或在匹配失败时执行中断操作 (如 `break`、`return`、`panic!`)。

```
use std::str::FromStr;

fn get_count_item(s: &str) -> (u64, &str) {
    let mut it = s.split(' ');
    let (Some(count_str), Some(item)) = (it.next(), it.next()) else {
        panic!("无法分割计数项对: '{s}'");
    };
    let Ok(count) = u64::from_str(count_str) else {
        panic!("无法解析整数: '{count_str}'");
    };
    (count, item)
}

fn main() {
    assert_eq!(get_count_item("3 chairs"), (3, "chairs"));
}
```

名称绑定的作用域是使其区别于 `match` 或 `if let-else` 表达式的主要特点。在此之前, 你可能需要通过一些冗余的重复和外部 `let` 来近似实现这些模式:

```
let (count_str, item) = match (it.next(), it.next()) {
    (Some(count_str), Some(item)) => (count_str, item),
    _ => panic!("无法分割计数项对: '{s}'"),
};
let count = if let Ok(count) = u64::from_str(count_str) {
    count
} else {
    panic!("无法解析整数: '{count_str}'");
};
```

另请参阅:

[Option](#)、[match](#)、[if let](#) 和 [let-else RFC](#)。

while let

与 `if let` 类似，`while let` 可以简化繁琐的 `match` 序列。让我们来看一个递增 `i` 的例子：

```
// 创建 `Option<i32>` 类型的 `optional`
let mut optional = Some(0);

// 重复执行此测试。
loop {
    match optional {
        // 如果 `optional` 解构成功，则执行代码块。
        Some(i) => {
            if i > 9 {
                println!("大于 9，退出！");
                optional = None;
            } else {
                println!("`i` 是 `{:?}`。再试一次。", i);
                optional = Some(i + 1);
            }
            // ^ 需要 3 层缩进！
        },
        // 当解构失败时退出循环：
        _ => { break; }
    }
}
```

使用 `while let` 可以让这个序列更加简洁：

```
1 fn main() {
2     // 创建 `Option<i32>` 类型的 `optional`
3     let mut optional = Some(0);
4
5     // 这段代码的含义是：当 `let` 将 `optional` 解构为 `Some(i)` 时，
6     // 执行代码块 `{}`，否则 `break`。
7     while let Some(i) = optional {
8         if i > 9 {
9             println!("大于 9，退出！");
10            optional = None;
11        } else {
12            println!("`i` 是 `{:?}`。再试一次。", i);
13            optional = Some(i + 1);
14        }
15        // ^ 减少了代码缩进右移，无需显式处理失败情况
16    }
17    // ^ `if let` 可以有额外的 `else`/`else if` 子句，`while let` 则没有。
18 }
```

另请参阅：

[enum](#)、[Option](#) 和 [RFC](#)

函数

函数使用 `fn` 关键字声明。函数参数需要标注类型，就像变量一样。如果函数返回值，则必须在箭头 `->` 后指定返回类型。

函数的最后一个表达式将作为返回值。另外，可以使用 `return` 语句在函数内部提前返回值，甚至可以在循环或 `if` 语句内部使用。

让我们用函数重写 FizzBuzz 吧！

```
1 // 与 C/C++ 不同，Rust 中函数定义的顺序没有限制
2 fn main() {
3     // 我们可以在这里使用函数，并在稍后的某处定义它
4     fizzbuzz_to(100);
5 }
6
7 // 返回布尔值的函数
8 fn is_divisible_by(lhs: u32, rhs: u32) -> bool {
9     // 特殊情况，提前返回
10    if rhs == 0 {
11        return false;
12    }
13
14    // 这是一个表达式，此处不需要 `return` 关键字
15    lhs % rhs == 0
16 }
17
18 // "无返回值"的函数实际上返回单元类型 `()`
19 fn fizzbuzz(n: u32) -> () {
20     if is_divisible_by(n, 15) {
21         println!("fizzbuzz");
22     } else if is_divisible_by(n, 3) {
23         println!("fizz");
24     } else if is_divisible_by(n, 5) {
25         println!("buzz");
26     } else {
27         println!("{}", n);
28     }
29 }
30
31 // 当函数返回 `()` 时，可以在函数签名中省略返回类型
32 fn fizzbuzz_to(n: u32) {
33     for n in 1..=n {
34         fizzbuzz(n);
35     }
36 }
```


关联函数和方法

某些函数与特定类型相关联。这些函数有两种形式：关联函数和方法。关联函数是在类型上定义的函数，而方法是在类型的特定实例上调用的关联函数。

```

1 struct Point {
2     x: f64,
3     y: f64,
4 }
5
6 // 实现块，所有 `Point` 的关联函数和方法都在此处定义
7 impl Point {
8     // 这是一个"关联函数"，因为这个函数与特定类型 `Point` 相关联。
9     //
10    // 关联函数不需要通过实例来调用。
11    // 这些函数通常用作构造函数。
12    fn origin() -> Point {
13        Point { x: 0.0, y: 0.0 }
14    }
15
16    // 另一个接受两个参数的关联函数：
17    fn new(x: f64, y: f64) -> Point {
18        Point { x: x, y: y }
19    }
20 }
21
22 struct Rectangle {
23     p1: Point,
24     p2: Point,
25 }
26
27 impl Rectangle {
28     // 这是一个方法
29     // `&self` 是 `self: &Self` 的语法糖，其中 `Self` 是调用者对象的类型
30     // 在这个例子中 `Self` = `Rectangle`
31     fn area(&self) -> f64 {
32        // `self` 通过点运算符访问结构体字段
33        let Point { x: x1, y: y1 } = self.p1;
34        let Point { x: x2, y: y2 } = self.p2;
35
36        // `abs` 是 `f64` 类型的方法，返回调用者的绝对值
37        ((x1 - x2) * (y1 - y2)).abs()
38    }
39
40    fn perimeter(&self) -> f64 {
41        let Point { x: x1, y: y1 } = self.p1;
42        let Point { x: x2, y: y2 } = self.p2;
43
44        2.0 * ((x1 - x2).abs() + (y1 - y2).abs())
45    }
46
47    // 这个方法要求调用对象是可变的
48    // `&mut self` 是 `self: &mut Self` 的语法糖
49    fn translate(&mut self, x: f64, y: f64) {
50        self.p1.x += x;
51        self.p2.x += x;
52
53        self.p1.y += y;
54        self.p2.y += y;
55    }
56 }
57
58 // `Pair` 拥有两个堆分配的整数资源
59 struct Pair(Box<i32>, Box<i32>);
60

```

```
61 impl Pair {
62     // 这个方法会"消耗"调用对象的资源
63     // `self` 是 `self: Self` 的语法糖
64     fn destroy(self) {
65         // 解构 `self`
66         let Pair(first, second) = self;
67
68         println!("正在销毁 Pair({}, {})", first, second);
69
70         // `first` 和 `second` 超出作用域并被释放
71     }
72 }
73
74 fn main() {
75     let rectangle = Rectangle {
76         // 使用双冒号调用关联函数
77         p1: Point::origin(),
78         p2: Point::new(3.0, 4.0),
79     };
80
81     // 使用点运算符调用方法
82     // 注意, 第一个参数 `&self` 是隐式传递的
83     // 即 `rectangle.perimeter()` 等同于 `Rectangle::perimeter(&rectangle)`
84     println!("矩形周长: {}", rectangle.perimeter());
85     println!("矩形面积: {}", rectangle.area());
86
87     let mut square = Rectangle {
88         p1: Point::origin(),
89         p2: Point::new(1.0, 1.0),
90     };
91
92     // 错误! `rectangle` 是不可变的, 但这个方法需要可变对象
93     //rectangle.translate(1.0, 0.0);
94     // TODO ^ 尝试取消这行的注释
95
96     // 正确! 可变对象可以调用可变方法
97     square.translate(1.0, 1.0);
98
99     let pair = Pair(Box::new(1), Box::new(2));
100
101     pair.destroy();
102
103     // 报错! 之前的 `destroy` 调用已经"消耗"了 `pair`
104     //pair.destroy();
105     // TODO ^ 尝试取消注释这一行
106 }
```

闭包

闭包是可以捕获周围环境的函数。例如，下面是一个捕获变量 `x` 的闭包：

```
|val| val + x
```

闭包的语法和功能使其非常适合即时使用。调用闭包与调用函数完全相同。不过，闭包的输入和返回类型**可以被推断**，而输入变量名**必须指定**。

闭包的其他特点包括：

- 使用 `||` 而不是 `()` 来包围输入变量。
- 单行表达式可省略函数体定界符 `{ }`，其他情况则必须使用
- 能够捕获外部环境的变量

```
1 fn main() {
2     let outer_var = 42;
3
4     // 常规函数无法引用外部环境的变量
5     //fn function(i: i32) -> i32 { i + outer_var }
6     // TODO: 取消上面这行的注释，查看编译器错误
7     // 编译器会建议我们定义一个闭包来替代
8
9     // 闭包是匿名的，这里我们将它们绑定到引用
10    // 注解与函数注解相同，但是可选的
11    // 包裹函数体的 `{}` 也是可选的
12    // 这些无名函数被赋值给适当命名的变量
13    let closure_annotated = |i: i32| -> i32 { i + outer_var };
14    let closure_inferred  = |i      |          i + outer_var ;
15
16    // 调用闭包
17    println!("closure_annotated: {}", closure_annotated(1));
18    println!("closure_inferred: {}", closure_inferred(1));
19    // 闭包类型一旦被推断，就不能再用其他类型重新推断。
20    //println!("不能用其他类型重用 closure_inferred: {}", closure_inferred(1));
21    // TODO: 取消上面这行的注释，观察编译器错误。
22
23    // 一个无参数并返回 `i32` 的闭包。
24    // 返回类型是推断的。
25    let one = || 1;
26    println!("返回 1 的闭包: {}", one());
27
28 }
```

捕获

闭包本质上很灵活，无需注解就能根据功能需求自动适应。这使得捕获可以灵活地适应不同场景，有时移动，有时借用。闭包可以通过以下方式捕获变量：

- 通过引用： `&T`
- 通过可变引用： `&mut T`
- 通过值： `T`

闭包优先通过引用捕获变量，仅在必要时才使用更底部的的捕获方式。

```
1 fn main() {
2     use std::mem;
3
4     let color = String::from("green");
5
6     // 打印 `color` 的闭包，立即借用（`&`）`color` 并
7     // 将借用和闭包存储在 `print` 变量中。借用状态
8     // 将持续到 `print` 最后一次使用。
9     //
10    // `println!` 只需要不可变引用参数，所以
11    // 不会施加更多限制。
12    let print = || println!("`color`: {}", color);
13
14    // 使用借用调用闭包。
15    print();
16
17    // `color` 可以再次被不可变借用，因为闭包只持有
18    // `color` 的不可变引用。
19    let _reborrow = &color;
20    print();
21
22    // `print` 最后一次使用后，允许移动或重新借用
23    let _color_moved = color;
24
25
26    let mut count = 0;
27    // 增加 `count` 的闭包可以接受 `&mut count` 或 `count`，
28    // 但 `&mut count` 限制更少，所以选择它。立即
29    // 借用 `count`。
30    //
31    // `inc` 需要 `mut` 因为内部存储了 `&mut`。因此，
32    // 调用闭包会修改 `count`，这需要 `mut`。
33    let mut inc = || {
34        count += 1;
35        println!("`count`: {}", count);
36    };
37
38    // 使用可变借用调用闭包。
39    inc();
40
41    // 闭包仍然可变借用 `count`，因为它稍后会被调用。
42    // 尝试重新借用会导致错误。
43    // let _reborrow = &count;
44    // ^ TODO: 尝试取消注释这行。
45    inc();
46
47    // 闭包不再需要借用 `&mut count`。因此，
48    // 可以在没有错误的情况下重新借用
49    let _count_reborrowed = &mut count;
50
51
52    // 不可复制类型。
53    let movable = Box::new(3);
54
55    // `mem::drop` 需要 `T`，所以这里必须通过值获取。可复制类型
56    // 会被复制到闭包中，原始值保持不变。
57    // 不可复制类型必须移动，所以 `movable` 立即移动到闭包中。
58    let consume = || {
59        println!("`movable`: {:?}", movable);
60        mem::drop(movable);
61    };
62    consume();
63}
```

```
60         mem::drop(movable);
61     };
62
63     // `consume` 消耗了变量，所以只能调用一次。
64     consume();
65     // consume();
66     // ^ TODO: 尝试取消注释这行。
67 }
```

在竖线前使用 `move` 强制闭包获取捕获变量的所有权：

```
1 fn main() {
2     // `Vec` 是非复制语义。
3     let haystack = vec![1, 2, 3];
4
5     let contains = move |needle| haystack.contains(needle);
6
7     println!("{}", contains(&1));
8     println!("{}", contains(&4));
9
10    // println!("vec 中有 {} 个元素", haystack.len());
11    // ^ 取消上面这行的注释会导致编译时错误
12    // 因为借用检查器不允许在变量被移动后重用。
13
14
15    // 从闭包签名中移除 `move` 将导致闭包
16    // 不可变借用 _haystack_ 变量，因此 _haystack_ 仍可用，
17    // 取消注释上面的行不会导致错误。
18 }
```

另请参阅：

`Box` 和 `std::mem::drop`

作为输入参数

Rust 通常能自动选择如何捕获变量，无需类型标注。但在编写函数时，这种模糊性是不允许的。当将闭包作为输入参数时，必须使用特定的 `trait` 来注解闭包的完整类型。这些 `trait` 由闭包对捕获值的处理方式决定。按限制程度从高到低排列如下：

- `Fn`：闭包通过引用使用捕获的值（`&T`）
- `FnMut`：闭包通过可变引用使用捕获的值（`&mut T`）
- `FnOnce`：闭包通过值使用捕获的值（`T`）

编译器会以尽可能最少限制的方式逐个捕获变量。

例如，考虑一个注解为 `FnOnce` 的参数。这表示闭包**可能**通过 `&T`、`&mut T` 或 `T` 进行捕获，但编译器最终会根据捕获变量在闭包中的使用方式来决定。

这是因为如果可以移动，那么任何类型的借用也应该是可能的。注意反过来并不成立。如果参数被注解为 `Fn`，那么通过 `&mut T` 或 `T` 捕获变量是不允许的。但 `&T` 是允许的。

在下面的例子中，尝试交换 `Fn`、`FnMut` 和 `FnOnce` 的用法，看看会发生什么：


```

1  // 这个函数接受一个闭包作为参数并调用它
2  // <F> 表示 F 是一个"泛型类型参数"
3  fn apply<F>(f: F) where
4      // 这个闭包不接受输入也不返回任何值
5      F: FnOnce() {
6      // ^ TODO: 试着将其改为 `Fn` 或 `FnMut`
7
8      f();
9  }
10
11 // 这个函数接受一个闭包并返回 `i32`
12 fn apply_to_3<F>(f: F) -> i32 where
13     // 这个闭包接受一个 `i32` 并返回一个 `i32`
14     F: Fn(i32) -> i32 {
15
16     f(3)
17 }
18
19 fn main() {
20     use std::mem;
21
22     let greeting = "hello";
23     // 一个非复制类型
24     // `to_owned` 从借用的数据创建拥有所有权的数据
25     let mut farewell = "goodbye".to_owned();
26
27     // 捕获两个变量: 通过引用捕获 `greeting`,
28     // 通过值捕获 `farewell`
29     let diary = || {
30         // `greeting` 是通过引用捕获的: 需要 `Fn`
31         println!("我说{}。", greeting);
32
33         // 修改强制 `farewell` 通过可变引用捕获
34         // 现在需要 `FnMut`
35         farewell.push_str("!!!");
36         println!("然后我喊{}。", farewell);
37         println!("现在我可以睡觉了。呼呼");
38
39         // 手动调用 drop 强制 `farewell` 通过值捕获
40         // 现在需要 `FnOnce`
41         mem::drop(farewell);
42     };
43
44     // 调用应用闭包的函数
45     apply(diary);
46
47     // `double` 满足 `apply_to_3` 的 trait 约束
48     let double = |x| 2 * x;
49
50     println!("3 的两倍是: {}", apply_to_3(double));
51 }

```

另请参阅:

`std::mem::drop`、`Fn`、`FnMut`、[泛型](#)、`where` 和 `FnOnce`

类型匿名

闭包能简洁地从外部作用域捕获变量。这会有什么影响吗？当然会有。注意观察如何将闭包作为函数参数使用时需要**泛型**，这是由于闭包的定义方式所决定的：

```
// `F` 必须是泛型
fn apply<F>(f: F) where
    F: FnOnce() {
    f();
}
```

当定义一个闭包时，编译器会隐式创建一个新的匿名结构来存储内部捕获的变量，同时通过 `Fn`、`FnMut` 或 `FnOnce` 这些 `trait` 之一为这个未知类型实现功能。这个类型被赋给变量并存储，直到被调用。

由于这个新类型是未知类型，在函数中使用时就需要泛型。然而，一个无界的类型参数 `<T>` 仍然会是模糊的，不被允许。因此，通过 `Fn`、`FnMut` 或 `FnOnce` 这些 `trait` 之一（它实现的）来约束就足以指定其类型。

```
1 // `F` 必须实现 `Fn` 用于一个不接受输入且不返回任何内容的闭包
2 // - 这正是 `print` 所需要的
3 fn apply<F>(f: F) where
4     F: Fn() {
5     f();
6 }
7
8 fn main() {
9     let x = 7;
10
11     // 将 `x` 捕获到一个匿名类型中并为其实现 `Fn`
12     // 将其存储在 `print` 中
13     let print = || println!("{}", x);
14
15     apply(print);
16 }
```

另请参阅：

[深入分析](#)、[Fn](#)、[FnMut](#) 和 [FnOnce](#)

输入函数

既然闭包可以作为参数使用，你可能会想知道函数是否也可以这样。确实可以！如果你声明一个函数，它接受一个闭包作为参数，那么任何满足该闭包 trait 约束的函数都可以作为参数传递。

```
1 // 定义一个函数，它接受一个由 `Fn` 约束的泛型参数 `F`，并调用它
2 fn call_me<F: Fn()>(f: F) {
3     f();
4 }
5
6 // 定义一个满足 `Fn` 约束的包装函数
7 fn function() {
8     println!("我是函数！");
9 }
10
11 fn main() {
12     // 定义一个满足 `Fn` 约束的闭包
13     let closure = || println!("我是闭包！");
14
15     call_me(closure);
16     call_me(function);
17 }
```

另外需要注意的是，`Fn`、`FnMut` 和 `FnOnce` 这些 trait 决定了闭包如何从外部作用域捕获变量。

另请参阅：

[Fn](#)、[FnMut](#) 和 [FnOnce](#)

作为输出参数

既然闭包可以作为输入参数，那么将闭包作为输出参数返回也应该是可行的。然而，匿名闭包类型本质上是未知的，因此我们必须使用 `impl Trait` 来返回它们。

可用于返回闭包的有效 trait 包括：

- `Fn`
- `FnMut`
- `FnOnce`

此外，必须使用 `move` 关键字，它表示所有捕获都是按值进行的。这是必要的，因为任何通过引用捕获的变量都会在函数退出时被丢弃，从而在闭包中留下无效的引用。

```
1 fn create_fn() -> impl Fn() {
2     let text = "Fn".to_owned();
3
4     move || println!("这是一个: {}", text)
5 }
6
7 fn create_fnmut() -> impl FnMut() {
8     let text = "FnMut".to_owned();
9
10    move || println!("这是一个: {}", text)
11 }
12
13 fn create_fnonce() -> impl FnOnce() {
14     let text = "FnOnce".to_owned();
15
16    move || println!("这是一个: {}", text)
17 }
18
19 fn main() {
20     let fn_plain = create_fn();
21     let mut fn_mut = create_fnmut();
22     let fn_once = create_fnonce();
23
24     fn_plain();
25     fn_mut();
26     fn_once();
27 }
```

另请参阅：

[Fn](#)、[FnMut](#)、[泛型](#)和 [impl Trait](#)

std 中的例子

本节包含一些使用 `std` 库中闭包的示例。

Iterator::any

`Iterator::any` 是一个函数，它接受一个迭代器作为参数。如果任何元素满足给定的条件，则返回 `true`，否则返回 `false`。其签名如下：

```
pub trait Iterator {
    // 被迭代的类型
    type Item;

    // `any` 接受 `&mut self`，意味着调用者可能被借用
    // 和修改，但不会被消耗
    fn any<F>(&mut self, f: F) -> bool where
        // `FnMut` meaning any captured variable may at most be
        // modified, not consumed. `Self::Item` is the closure parameter
        type,
        // which is determined by the iterator (e.g., `&T` for `.iter()`，
        // `T` for `.into_iter()`).
        F: FnMut(Self::Item) -> bool;
}

1 fn main() {
2     let vec1 = vec![1, 2, 3];
3     let vec2 = vec![4, 5, 6];
4
5     // 对 vec 使用 `iter()` 产生 `&i32`，解构为 `i32`
6     println!("2 在 vec1 中: {}", vec1.iter().any(|&x| x == 2));
7     // 对 vec 使用 `into_iter()` 产生 `i32`，无需解构
8     println!("2 在 vec2 中: {}", vec2.into_iter().any(|x| x == 2));
9
10    // `iter()` 只借用 `vec1` 及其元素，所以它们可以再次使用
11    println!("vec1 长度: {}", vec1.len());
12    println!("vec1 的第一个元素是: {}", vec1[0]);
13    // `into_iter()` 会移动 `vec2` 及其元素，所以它们不能再次使用
14    // println!("vec2 的第一个元素是: {}", vec2[0]);
15    // println!("vec2 长度: {}", vec2.len());
16    // TODO: 取消上面两行的注释，观察编译器错误
17
18    let array1 = [1, 2, 3];
19    let array2 = [4, 5, 6];
20
21    // 对数组使用 `iter()` 产生 `&i32`
22    println!("2 在 array1 中: {}", array1.iter().any(|&x| x == 2));
23    // 对数组使用 `into_iter()` 产生 `i32`
24    println!("2 在 array2 中: {}", array2.into_iter().any(|x| x == 2));
25 }
```

另请参阅：

[std::iter::Iterator::any](#)

通过迭代器搜索

`Iterator::find` 是一个函数，它遍历迭代器并搜索满足特定条件的第一个值。如果没有值满足条件，则返回 `None`。其签名如下：

```
pub trait Iterator {
    // 被迭代的类型
    type Item;

    // `find` 接受 `&mut self`，这意味着调用者可能被借用
    // 和修改，但不会被消耗。
    fn find<P>(&mut self, predicate: P) -> Option<Self::Item> where
        // `FnMut` 表示任何捕获的变量最多只能被修改，不能被消耗。
        // `&Self::Item` 表示它通过引用将参数传递给闭包。
        P: FnMut(&Self::Item) -> bool;
}

1 fn main() {
2     let vec1 = vec![1, 2, 3];
3     let vec2 = vec![4, 5, 6];
4
5     // `vec1.iter()` yields `&i32`.
6     let mut iter = vec1.iter();
7     // `vec2.into_iter()` yields `i32`.
8     let mut into_iter = vec2.into_iter();
9
10    // `iter()` yields `&i32`, and `find` passes `&Item` to the predicate
11    // Since `Item = &i32`, the closure argument has type `&&i32`,
12    // which we pattern-match to dereference down to `i32`.
13    println!("在 vec1 中查找 2: {:?}", iter.find(|&x| x == 2));
14
15    // `into_iter()` yields `i32`, and `find` passes `&Item` to the predicate
16    // Since `Item = i32`, the closure argument has type `&i32`,
17    // which we pattern-match to dereference down to `i32`.
18    println!("在 vec2 中查找 2: {:?}", into_iter.find(|&x| x == 2));
19
20    let array1 = [1, 2, 3];
21    let array2 = [4, 5, 6];
22
23    // `array1.iter()` yields `&i32`, and `find` passes `&Item` to the
24    // predicate. Since `Item = &i32`, the closure argument has type `&&i32`.
25    println!("在 array1 中查找 2: {:?}", array1.iter().find(|&x| x == 2));
26    // `array2.into_iter()` yields `i32` (since Rust 2021 edition), and
27    // `find` passes `&Item` to the predicate. Since `Item = i32`, the
28    // closure argument has type `&i32`.
29    println!("在 array2 中查找 2: {:?}", array2.into_iter().find(|&x| x == 2));
30 }
```

`Iterator::find` 返回元素的引用。如果需获取元素的索引，则使用 `Iterator::position`。

```
1 fn main() {  
2     let vec = vec![1, 9, 3, 3, 13, 2];  
3  
4     // `position` passes the iterator's `Item` by value to the predicate.  
5     // `vec.iter()` yields `&i32`, so the predicate receives `&i32`,  
6     // which we pattern-match to dereference to `i32`.  
7     let index_of_first_even_number = vec.iter().position(|&x| x % 2 == 0)  
8     assert_eq!(index_of_first_even_number, Some(5));  
9  
10    // `vec.into_iter()` yields `i32`, so the predicate receives `i32` directly.  
11    let index_of_first_negative_number = vec.into_iter().position(|x| x < 0)  
12    assert_eq!(index_of_first_negative_number, None);  
13 }
```

另请参阅：

`std::iter::Iterator::find`

`std::iter::Iterator::find_map`

`std::iter::Iterator::position`

`std::iter::Iterator::rposition`

高阶函数

Rust 提供了高阶函数（Higher Order Functions, HOF）。这些函数接受一个或多个函数作为参数，并/或产生一个更有用的函数。HOF 和惰性迭代器赋予了 Rust 函数式编程的特性。

```
1 fn is_odd(n: u32) -> bool {
2     n % 2 == 1
3 }
4
5 fn main() {
6     println!("找出所有平方为奇数且小于 1000 的数字之和");
7     let upper = 1000;
8
9     // 命令式方法
10    // 声明累加器变量
11    let mut acc = 0;
12    // 迭代: 从 0, 1, 2, ... 到无穷大
13    for n in 0.. {
14        // 计算数字的平方
15        let n_squared = n * n;
16
17        if n_squared >= upper {
18            // 如果超过上限则跳出循环
19            break;
20        } else if is_odd(n_squared) {
21            // 如果是奇数, 则累加值
22            acc += n;
23        }
24    }
25    println!("命令式风格: {}", acc);
26
27    // 函数式方法
28    let sum: u32 =
29        (0..).take_while(|&n| n * n < upper) // 小于上限
30        .filter(|&n| is_odd(n * n))         // 筛选奇数
31        .sum();                             // 求和
32    println!("函数式风格: {}", sum);
33 }
```

[Option](#) 和 [Iterator](#) 实现了相当多的高阶函数。

发散函数

发散函数永不返回。它们使用 `!` 标记，这是一个空类型。

```
fn foo() -> ! {  
    panic!("此调用永不返回。");  
}
```

与所有其他类型相反，这个类型不能被实例化，因为这个类型可能拥有的所有可能值的集合是空的。注意，它与 `()` 类型不同，后者恰好有一个可能的值。

例如，这个函数像往常一样返回，尽管返回值中没有任何信息。

```
fn some_fn() {  
    ()  
}  
  
fn main() {  
    let _a: () = some_fn();  
    println!("这个函数返回了，你可以看到这一行。");  
}
```

与之相对的是这个函数，它永远不会将控制权返回给调用者。

```
#![feature(never_type)]  
  
fn main() {  
    let x: ! = panic!("此调用永不返回。");  
    println!("你永远不会看到这一行！");  
}
```

虽然这看起来像是一个抽象概念，但它实际上非常有用且经常派上用场。这种类型的主要优势是它可以被转换为任何其他类型，这使得它在需要精确类型的情况下非常灵活，比如在 `match` 分支中。这种灵活性允许我们编写如下代码：

```
fn main() {  
    fn sum_odd_numbers(up_to: u32) -> u32 {  
        let mut acc = 0;  
        for i in 0..up_to {  
            // 注意这个 match 表达式的返回类型必须是 u32,  
            // 因为 "addition" 变量的类型是 u32。  
            let addition: u32 = match i%2 == 1 {  
                // "i" 变量的类型是 u32, 这完全没问题。  
                true => i,  
                // 另一方面, "continue" 表达式不返回 u32,  
                // 但这仍然可以, 因为它永远不会返回,  
                // 因此不违反 match 表达式的类型要求。  
                false => continue,  
            };  
            acc += addition;  
        }  
        acc  
    }  
    println!("9 以下（不包括 9）的奇数之和: {}", sum_odd_numbers(9));  
}
```

它也是永远循环的函数（例如 `loop {}`）的返回类型，比如网络服务器，或终止进程的函数（例如 `exit()`）。

模块

Rust 提供了一个强大的模块系统，可以用来将代码分层地拆分成逻辑单元（模块），并管理它们之间的可见性（公有/私有）。

一个模块是一系列项的集合：函数、结构体、trait、`impl` 块，甚至其他模块。

可见性

默认情况下，模块中的项具有私有可见性，但可以使用 `pub` 修饰符来覆盖这一默认行为。只有模块中的公有项可以从模块作用域外部访问。

```

1  // 一个名为 `my_mod` 的模块
2  mod my_mod {
3      // 模块中的项默认为私有可见性。
4      fn private_function() {
5          println!("调用了 `my_mod::private_function()`");
6      }
7
8      // 使用 `pub` 修饰符来覆盖默认的可见性。
9      pub fn function() {
10         println!("调用了 `my_mod::function()`");
11     }
12
13     // 同一模块中的项可以访问其他项，
14     // 即使是私有的。
15     pub fn indirect_access() {
16         print!("调用了 `my_mod::indirect_access()`，它\n> ");
17         private_function();
18     }
19
20     // 模块也可以嵌套
21     pub mod nested {
22         pub fn function() {
23             println!("调用了 `my_mod::nested::function()`");
24         }
25
26         #[allow(dead_code)]
27         fn private_function() {
28             println!("调用了 `my_mod::nested::private_function()`");
29         }
30
31         // 使用 `pub(in path)` 语法声明的函数只在给定的路径中可见。
32         // `path` 必须是父模块或祖先模块
33         pub(in crate::my_mod) fn public_function_in_my_mod() {
34             print!("调用了 `my_mod::nested::public_function_in_my_mod()`");
35             public_function_in_nested();
36         }
37
38         // 使用 `pub(self)` 语法声明的函数只在当前模块中可见，
39         // 这与将它们保持为私有是一样的
40         pub(self) fn public_function_in_nested() {
41             println!("调用了 `my_mod::nested::public_function_in_nested()`");
42         }
43
44         // 使用 `pub(super)` 语法声明的函数仅在父模块中可见
45
46         pub(super) fn public_function_in_super_mod() {
47             println!("调用了 `my_mod::nested::public_function_in_super_mod()`");
48         }
49     }
50
51     pub fn call_public_function_in_my_mod() {
52         print!("调用了 `my_mod::call_public_function_in_my_mod()`，它\n> ");
53         nested::public_function_in_my_mod();
54         print!("\n> ");
55         nested::public_function_in_super_mod();
56     }
57
58     // pub(crate) 使函数仅在当前 crate 内可见
59     pub(crate) fn public_function_in_crate() {
60         println!("调用了 `my_mod::public_function_in_crate()`");

```

```

50     println!("调用了 `my_mod::public_function_in_crate()`");
61 }
62
63 // 嵌套模块遵循相同的可见性规则
64 mod private_nested {
65     #[allow(dead_code)]
66     pub fn function() {
67         println!("调用了 `my_mod::private_nested::function()`");
68     }
69
70     // 私有父项仍会限制子项的可见性，
71     // 即使子项被声明为在更大范围内可见。
72     #[allow(dead_code)]
73     pub(crate) fn restricted_function() {
74         println!("调用了 `my_mod::private_nested::restricted_function()`");
75     }
76 }
77 }
78
79 fn function() {
80     println!("调用了 `function()`");
81 }
82
83 fn main() {
84     // 模块允许消除同名项之间的歧义。
85     function();
86     my_mod::function();
87
88     // 公有项（包括嵌套模块内的）可以从父模块外部访问。
89     my_mod::indirect_access();
90     my_mod::nested::function();
91     my_mod::call_public_function_in_my_mod();
92
93     // pub(crate) 项可以在同一个 crate 的任何地方调用
94     my_mod::public_function_in_crate();
95
96     // pub(in path) 项只能在指定的模块内调用
97     // 错误！函数 `public_function_in_my_mod` 是私有的
98     //my_mod::nested::public_function_in_my_mod();
99     // TODO ^ 尝试取消此行注释
100
101     // 模块的私有项不能被直接访问，即使它们嵌套在公有模块中：
102
103     // 错误！`private_function` 是私有的
104     //my_mod::private_function();
105     // TODO ^ 尝试取消此行注释
106
107     // 错误！`private_function` 是私有的
108     //my_mod::nested::private_function();
109     // TODO ^ 尝试取消此行注释
110
111     // 错误！`private_nested` 是一个私有模块
112     //my_mod::private_nested::function();
113     // TODO ^ 尝试取消此行注释
114
115     // 错误！`private_nested` 是一个私有模块
116     //my_mod::private_nested::restricted_function();
117     // TODO ^ 尝试取消此行注释
118 }

```

结构体可见性

结构体的字段具有额外的可见性级别。字段默认为私有，可以使用 `pub` 修饰符来覆盖。这种可见性只在从结构体定义模块外部访问时才有意义，其目的是实现信息隐藏（封装）。

```
1 mod my {
2     // 一个具有泛型类型 `T` 公有字段的公有结构体
3     pub struct OpenBox<T> {
4         pub contents: T,
5     }
6
7     // 一个具有泛型类型 `T` 私有字段的公有结构体
8     pub struct ClosedBox<T> {
9         contents: T,
10    }
11
12    impl<T> ClosedBox<T> {
13        // 一个公有的构造方法
14        pub fn new(contents: T) -> ClosedBox<T> {
15            ClosedBox {
16                contents: contents,
17            }
18        }
19    }
20 }
21
22 fn main() {
23     // 具有公有字段的公有结构体可以正常构造
24     let open_box = my::OpenBox { contents: "公开信息" };
25
26     // 且可以正常访问其字段。
27     println!("打开的盒子包含: {}", open_box.contents);
28
29     // 具有私有字段的公有结构体不能使用字段名来构造。
30     // 错误! `ClosedBox` 有私有字段
31     //let closed_box = my::ClosedBox { contents: "机密信息" };
32     // TODO ^ 尝试取消此行注释
33
34     // 然而，具有私有字段的结构体可以使用公有构造函数创建
35     let _closed_box = my::ClosedBox::new("机密信息");
36
37     // 且无法访问公有结构体的私有字段。
38     // 错误! `contents` 字段是私有的
39     //println!("封闭的盒子包含: {}", _closed_box.contents);
40     // TODO ^ 尝试取消此行注释
41 }
```

另请参阅：

[泛型和方法](#)

use 声明

`use` 声明可以将完整路径绑定到新名称，以便更轻松地访问。它通常这样使用：

```
1 use crate::deeply::nested::{
2     my_first_function,
3     my_second_function,
4     AndATraitType
5 };
6
7 fn main() {
8     my_first_function();
9 }
```

你可以使用 `as` 关键字将导入绑定到不同的名称：

```
1 // 将 `deeply::nested::function` 路径绑定到 `other_function`。
2 use deeply::nested::function as other_function;
3
4 fn function() {
5     println!("调用了 `function()`");
6 }
7
8 mod deeply {
9     pub mod nested {
10         pub fn function() {
11             println!("调用了 `deeply::nested::function()`");
12         }
13     }
14 }
15
16 fn main() {
17     // 更方便地访问 `deeply::nested::function`
18     other_function();
19
20     println!("进入代码块");
21     {
22         // 这等同于 `use deeply::nested::function as function`。
23         // 这个 `function()` 将遮蔽外部的同名函数。
24         use crate::deeply::nested::function;
25
26         // `use` 绑定具有局部作用域。在这种情况下，
27         // `function()` 的遮蔽仅在此代码块内有效。
28         function();
29
30         println!("离开代码块");
31     }
32
33     function();
34 }
```

You can also use `pub use` to re-export an item from a module, so it can be accessed through the module's public interface:

```
1 mod deeply {  
2     pub mod nested {  
3         pub fn function() {  
4             println!("调用了 `deeply::nested::function()`");  
5         }  
6     }  
7 }  
8  
9 mod cool {  
10     pub use crate::deeply::nested::function;  
11 }  
12  
13 fn main() {  
14     cool::function();  
15 }
```

super 和 self

`super` 和 `self` 关键字可以在路径中使用，以消除访问项目时的歧义，并避免不必要的路径硬编码。

```
1 fn function() {
2     println!("调用了 `function()`");
3 }
4
5 mod cool {
6     pub fn function() {
7         println!("调用了 `cool::function()`");
8     }
9 }
10
11 mod my {
12     fn function() {
13         println!("调用了 `my::function()`");
14     }
15
16     mod cool {
17         pub fn function() {
18             println!("调用了 `my::cool::function()`");
19         }
20     }
21
22     pub fn indirect_call() {
23         // 让我们从这个作用域访问所有名为 `function` 的函数！
24         print!("调用了 `my::indirect_call()`，它\n> ");
25
26         // `self` 关键字指的是当前模块作用域 - 在这里是 `my`。
27         // 调用 `self::function()` 和直接调用 `function()` 都会
28         // 得到相同的结果，因为它们指向同一个函数。
29         self::function();
30         function();
31
32         // 我们也可以使用 `self` 来访问 `my` 内的另一个模块：
33         self::cool::function();
34
35         // `super` 关键字指的是父作用域（`my` 模块之外）。
36         super::function();
37
38         // 这将绑定到 *crate* 作用域中的 `cool::function`。
39         // 在这种情况下，crate 作用域是最外层作用域。
40         {
41             use crate::cool::function as root_function;
42             root_function();
43         }
44     }
45 }
46
47 fn main() {
48     my::indirect_call();
49 }
```

文件分层

模块可以映射到文件/目录层次结构。让我们将[可见性示例](#)拆分成多个文件：

```
$ tree .
```

```
.
├── my
│   ├── inaccessible.rs
│   └── nested.rs
├── my.rs
└── split.rs
```

在 `split.rs` 文件中：

```
// 这个声明会查找名为 `my.rs` 的文件，并将其内容
// 插入到当前作用域下名为 `my` 的模块中
mod my;

fn function() {
    println!("调用了 `function()`");
}

fn main() {
    my::function();

    function();

    my::indirect_access();

    my::nested::function();
}
```

在 `my.rs` 文件中：

```
// Similarly `mod inaccessible` and `mod nested` will locate the
// `inaccessible.rs` and `nested.rs` files and insert them here under their
// respective modules
mod inaccessible;
pub mod nested;

pub fn function() {
    println!("调用了 `my::function()`");
}

fn private_function() {
    println!("调用了 `my::private_function()`");
}

pub fn indirect_access() {
    print!("调用了 `my::indirect_access()`, 它\n> ");

    private_function();
}
```

在 `my/nested.rs` 文件中:

```
pub fn function() {
    println!("调用了 `my::nested::function()`");
}

#[allow(dead_code)]
fn private_function() {
    println!("调用了 `my::nested::private_function()`");
}
```

在 `my/inaccessible.rs` 文件中:

```
#[allow(dead_code)]
pub fn public_function() {
    println!("调用了 `my::inaccessible::public_function()`");
}
```

让我们检查一下是否一切仍然如之前一样正常运行:

```
$ rustc split.rs && ./split
called `my::function()`
called `function()`
called `my::indirect_access()`, that
> called `my::private_function()`
called `my::nested::function()`
```

Crates

在 Rust 中，crate 是一个编译单元。每当执行 `rustc some_file.rs` 时，`some_file.rs` 就被视为 **crate 文件**。如果 `some_file.rs` 中包含 `mod` 声明，那么模块文件的内容会在编译器处理之前被插入到 crate 文件中 `mod` 声明的位置。换句话说，模块**不会被**单独编译，只有 crate 才会被编译。

一个 crate 可以被编译成二进制文件或库。默认情况下，`rustc` 会将 crate 编译成二进制文件。通过向 `rustc` 传递 `--crate-type` 标志并指定 `lib`，可以改变这一默认行为。

创建库

让我们创建一个库，然后看看如何将它链接到另一个 crate。

在 `rary.rs` 文件中：

```
pub fn public_function() {
    println!("调用了 rary 的 `public_function()`");
}

fn private_function() {
    println!("调用了 rary 的 `private_function()`");
}

pub fn indirect_access() {
    print!("调用了 rary 的 `indirect_access()`，它\n> ");

    private_function();
}

$ rustc --crate-type=lib rary.rs
$ ls lib*
library.rlib
```

库文件名会自动添加 “lib” 前缀，默认使用 crate 文件的名称。但可以通过向 `rustc` 传递 `--crate-name` 选项或使用 `crate_name` 属性来覆盖这个默认名称。

使用库

要将 crate 链接到这个新库，可以使用 `rustc` 的 `--extern` 标志。库中的所有项目都会被导入到一个与库同名的模块下。这个模块的行为通常与其他模块相同。

```
// extern crate rary; // Rust 2015 版本或更早版本可能需要此声明
```

```
fn main() {  
    rary::public_function();  
  
    // 错误! `private_function` 是私有的  
    // rary::private_function();  
  
    rary::indirect_access();  
}
```

```
# Where library.rlib is the path to the compiled library, assumed that it's  
# in the same directory here:  
$ rustc executable.rs --extern rary=library.rlib && ./executable  
called rary's `public_function()`  
called rary's `indirect_access()`, that  
> called rary's `private_function()`
```


Cargo

`cargo` 是 Rust 的官方包管理工具。它提供了许多非常有用的功能，可以提高代码质量和开发效率！这些功能包括：

- 依赖管理和与 crates.io（Rust 官方包注册中心）的集成
- 支持单元测试
- 支持基准测试

本章将快速介绍一些基础知识，更全面的文档可以在 [Cargo 手册](#) 中找到。

依赖

大多数程序都依赖于一些库。如果你曾经手动管理过依赖，你就知道这有多么痛苦。幸运的是，Rust 生态系统标配了 `cargo`！`cargo` 可以为项目管理依赖。

要创建一个新的 Rust 项目，可以使用以下命令：

```
# 创建二进制项目
cargo new foo

# 创建库项目
cargo new --lib bar
```

在本章的剩余部分，我们假设我们正在创建一个二进制项目，而不是一个库项目，但所有的概念都是相同的。

执行上述命令后，你应该会看到如下的文件结构：

```
.
├── bar
│   ├── Cargo.toml
│   └── src
│       └── lib.rs
└── foo
    ├── Cargo.toml
    └── src
        └── main.rs
```

`main.rs` 是你新建的 `foo` 项目的主源文件——这一点没什么新鲜的。`Cargo.toml` 则是这个项目的 `cargo` 配置文件。如果你查看它的内容，应该会看到类似这样的内容：

```
[package]
name = "foo"
version = "0.1.0"
authors = ["mark"]

[dependencies]
```

`[package]` 下的 `name` 字段决定了项目的名称。如果你将 crate 发布到 `crates.io`（稍后会详细介绍），这个名称将被使用。同时，它也是编译时生成的二进制文件的名称。

`version` 字段是使用[语义化版本控制](#)的 crate 版本号。

`authors` 字段是发布 crate 时使用的作者列表。

`[dependencies]` 部分允许你为项目添加依赖。

举个例子，假设我们想让程序拥有一个出色的命令行界面（CLI）。你可以在 crates.io（Rust 官方包注册中心）上找到许多优秀的包。其中，[clap](#) 是一个广受欢迎的选择。在撰写本文时，`clap` 的最新发布版本是 `2.27.1`。要在我们的程序中添加这个依赖，只需在 `Cargo.toml` 的 `[dependencies]` 下添加：`clap = "2.27.1"`。就这么简单！现在你就可以在程序中使用 `clap` 了。

`cargo` 还支持[其他类型的依赖](#)。这里给出一个简单的示例：

```
[package]
name = "foo"
version = "0.1.0"
authors = ["mark"]

[dependencies]
clap = "2.27.1" # 来自 crates.io
rand = { git = "https://github.com/rust-lang-nursery/rand" } # 来自在线仓库
bar = { path = "../bar" } # 来自本地文件系统的路径
```

`cargo` 不仅仅是一个依赖管理器。`Cargo.toml` 的[格式规范](#)中列出了所有可用的配置选项。

我们可以在项目目录的任何位置（包括子目录！）执行 `cargo build` 来构建项目。也可以使用 `cargo run` 来构建并运行。请注意，这些命令会解析所有依赖，必要时下载 crate，并构建所有内容，包括你的 crate。（值得一提的是，它只会重新构建尚未构建的部分，类似于 `make`）。

瞧！就是这么简单！

约定

在上一章中，我们看到了如下目录结构：

```
foo
├── Cargo.toml
└── src
    └── main.rs
```

那么，如果我们想在同一个项目中包含两个二进制文件，该怎么办呢？

`cargo` 实际上支持这种需求。如我们之前所见，默认的二进制文件名是 `main`，但你可以通过在 `bin/` 目录中放置额外的文件来添加其他二进制文件：

```
foo
├── Cargo.toml
└── src
    ├── main.rs
    └── bin
        └── my_other_bin.rs
```

如果要指示 `cargo` 只编译或运行特定的二进制文件，只需传递 `--bin my_other_bin` 标志，其中 `my_other_bin` 是我们想要处理的二进制文件的名称。

除了额外的二进制文件，`cargo` 还支持[更多功能](#)，如基准测试、测试和示例。

在下一章中，我们将更详细地探讨测试。

测试

众所周知，测试是任何软件不可或缺的一部分！Rust 为单元测试和集成测试提供了一流的支持（参见《Rust 程序设计语言》中的[这一章](#)）。

从上面链接的测试章节中，我们了解了如何编写单元测试和集成测试。在组织结构上，我们可以将单元测试放在它们所测试的模块中，而将集成测试放在专门的 `tests/` 目录中：

```
foo
├── Cargo.toml
├── src
│   ├── main.rs
│   └── lib.rs
└── tests
    ├── my_test.rs
    └── my_other_test.rs
```

`tests` 目录中的每个文件都是一个独立的[集成测试](#)，即旨在测试你的库，就像它被依赖的 crate 调用一样。

[测试](#)章节详细阐述了三种不同的测试风格：[单元测试](#)、[文档测试](#)和[集成测试](#)。

`cargo` 自然提供了一种简便的方式来运行所有测试！

```
$ cargo test
```

你将看到类似这样的输出：

```
$ cargo test
  Compiling blah v0.1.0 (file:///nobackup/blah)
  Finished dev [unoptimized + debuginfo] target(s) in 0.89 secs
  Running target/debug/deps/blah-d3b32b97275ec472

running 4 tests
test test_bar ... ok
test test_baz ... ok
test test_foo_bar ... ok
test test_foo ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

你还可以运行名称匹配特定模式的测试：

```
$ cargo test test_foo
```

```
$ cargo test test_foo
Compiling blah v0.1.0 (file:///nobackup/blah)
Finished dev [unoptimized + debuginfo] target(s) in 0.35 secs
Running target/debug/deps/blah-d3b32b97275ec472

running 2 tests
test test_foo ... ok
test test_foo_bar ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 2 filtered out
```

需要注意：Cargo 可能会并发运行多个测试，因此请确保它们之间不会产生竞态条件。

并发可能导致问题的一个例子是，如果两个测试同时输出到同一个文件，如下所示：

```
#[cfg(test)]
mod tests {
    // 导入必要的模块
    use std::fs::OpenOptions;
    use std::io::Write;

    // 这个测试向文件写入内容
    #[test]
    fn test_file() {
        // 打开 ferris.txt 文件，如果不存在则创建一个
        let mut file = OpenOptions::new()
            .append(true)
            .create(true)
            .open("ferris.txt")
            .expect("无法打开 ferris.txt");

        // 循环打印 "Ferris" 5 次
        for _ in 0..5 {
            file.write_all("Ferris\n".as_bytes())
                .expect("无法写入 ferris.txt");
        }
    }

    // 这个测试尝试写入同一个文件
    #[test]
    fn test_file_also() {
        // 打开 ferris.txt 文件，如果不存在则创建一个
        let mut file = OpenOptions::new()
            .append(true)
            .create(true)
            .open("ferris.txt")
            .expect("无法打开 ferris.txt");

        // 循环打印 "Corro" 5 次
        for _ in 0..5 {
            file.write_all("Corro\n".as_bytes())
                .expect("无法写入 ferris.txt");
        }
    }
}
```

尽管预期结果应该是：

```
$ cat ferris.txt
Ferris
Ferris
Ferris
Ferris
Ferris
Corro
Corro
Corro
Corro
Corro
```

但实际写入 `ferris.txt` 的内容可能是这样的：

```
$ cargo test test_file && cat ferris.txt
Corro
Ferris
Corro
Ferris
Corro
Ferris
Corro
Ferris
Corro
Ferris
```

构建脚本

有时候，`cargo` 的常规构建可能不足以满足需求。你的 crate 可能在 `cargo` 成功编译之前需要一些先决条件，比如代码生成，或者需要编译一些本地代码。为了解决这个问题，我们可以使用 `Cargo` 能够运行的构建脚本。

要为你的包添加构建脚本，可以在 `Cargo.toml` 中指定，如下所示：

```
[package]
...
build = "build.rs"
```

如果没有指定，`Cargo` 默认会在项目目录中查找 `build.rs` 文件。

如何使用构建脚本

构建脚本只是另一个 `Rust` 文件，它会在编译包中的其他内容之前被编译和调用。因此，它可以用来满足你的 crate 的先决条件。

`Cargo` 通过环境变量为脚本提供输入，这些环境变量可以被使用。具体参见[这里的说明](#)。

脚本通过标准输出提供输出。所有打印的行都会被写入

`target/debug/build/<pkg>/output`。此外，以 `cargo:` 为前缀的行会被 `Cargo` 直接解释，因此可以用来为包的编译定义参数。

如需了解更多详细规范和示例，请参阅 [Cargo 构建脚本规范](#)。

属性

属性是应用于模块、crate 或条目的元数据。这些元数据可用于以下目的：

- 代码的条件编译
- 设置 crate 的名称、版本和类型（二进制或库）
- 禁用 代码检查（警告）
- 启用编译器特性（如宏、全局导入等）
- 链接外部库
- 将函数标记为单元测试
- 将函数标记为基准测试的一部分
- 类属性宏

属性的形式为 `#[outer_attribute]`（外部属性）或 `#![inner_attribute]`（内部属性），它们的区别在于应用的位置。

- `#[outer_attribute]` 应用于紧随其后的条目。条目的例子包括：函数、模块声明、常量、结构体、枚举等。以下是一个示例，其中属性 `#[derive(Debug)]` 应用于结构体 `Rectangle`：

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}
```

- `#![inner_attribute]` 应用于包含它的条目（通常是模块或 crate）。换句话说，这种属性被解释为应用于它所在的整个作用域。以下是一个示例，其中 `#![allow(unused_variables)]` 应用于整个 crate（如果放置在 `main.rs` 中）：

```
#![allow(unused_variables)]

fn main() {
    let x = 3; // 这通常会警告未使用的变量。
}
```

属性可以使用不同的语法接受参数：

- `#[attribute = "value"]`
- `#[attribute(key = "value")]`
- `#[attribute(value)]`

属性可以有多个值，也可以跨多行分隔：

```
#[attribute(value, value2)]
```

```
#[attribute(value, value2, value3,  
            value4, value5)]
```

dead_code

编译器提供了一个 `dead_code` [lint](#)，用于警告未使用的函数。可以使用**属性**来禁用这个 lint。

```
1 fn used_function() {}
2
3 // `#[allow(dead_code)]` 是一个用于禁用 `dead_code` lint 的属性
4 #[allow(dead_code)]
5 fn unused_function() {}
6
7 fn noisy_unused_function() {}
8 // FIXME ^ 添加一个属性来抑制警告
9
10 fn main() {
11     used_function();
12 }
```

注意，在实际程序中，你应该消除无用代码。在这些示例中，我们会在某些地方允许存在无用代码，这是因为这些示例具有交互性质。

Crates

`crate_type` 属性可用于告诉编译器一个 crate 是二进制文件还是库（甚至是哪种类型的库），而 `crate_name` 属性可用于设置 crate 的名称。

然而，需要注意的是，当使用 Rust 的包管理器 Cargo 时，`crate_type` 和 `crate_name` 属性**完全不起作用**。由于大多数 Rust 项目都使用 Cargo，这意味着 `crate_type` 和 `crate_name` 在实际使用中的应用相对有限。

```
1 // 这个 crate 是一个库
2 #![crate_type = "lib"]
3 // 这个库的名称是 "rary"
4 #![crate_name = "rary"]
5
6 pub fn public_function() {
7     println!("调用了 rary 的 `public_function()`");
8 }
9
10 fn private_function() {
11     println!("调用了 rary 的 `private_function()`");
12 }
13
14 pub fn indirect_access() {
15     print!("调用了 rary 的 `indirect_access()`，它\n> ");
16
17     private_function();
18 }
```

当使用 `crate_type` 属性时，我们就不再需要向 `rustc` 传递 `--crate-type` 标志。

```
$ rustc lib.rs
$ ls lib*
library.rlib
```

cfg

配置条件检查可以通过两种不同的操作符实现：

- `cfg` 属性：在属性位置使用 `#[cfg(...)]`
- `cfg!` 宏：在布尔表达式中使用 `cfg!(...)`

前者启用条件编译，后者在运行时条件性地求值为 `true` 或 `false` 字面量，允许在运行时进行检查。两者使用相同的参数语法。

`cfg!` 与 `#[cfg]` 不同，它不会移除任何代码，只会求值为 `true` 或 `false`。例如，当 `cfg!` 用于条件时，`if/else` 表达式中的所有代码块都需要是有效的，无论 `cfg!` 正在评估什么。

```
1 // 这个函数只有在目标操作系统是 linux 时才会被编译
2 #[cfg(target_os = "linux")]
3 fn are_you_on_linux() {
4     println!("你正在运行 Linux! ");
5 }
6
7 // 而这个函数只有在目标操作系统**不是** Linux 时才会被编译
8 #[cfg(not(target_os = "linux"))]
9 fn are_you_on_linux() {
10     println!("你**不是**在运行 Linux! ");
11 }
12
13 fn main() {
14     are_you_on_linux();
15
16     println!("你确定吗? ");
17     if cfg!(target_os = "linux") {
18         println!("是的，这绝对是 Linux! ");
19     } else {
20         println!("是的，这绝对**不是** Linux! ");
21     }
22 }
```

另请参阅：

[参考文档](#)、[cfg!](#) 宏和宏。

自定义

一些条件（如 `target_os`）是由 `rustc` 隐式提供的，但自定义条件必须通过 `--cfg` 标志传递给 `rustc`。

```
1 #[cfg(some_condition)]
2 fn conditional_function() {
3     println!("条件满足!");
4 }
5
6 fn main() {
7     conditional_function();
8 }
```

尝试运行这段代码，看看没有自定义 `cfg` 标志会发生什么。

使用自定义 `cfg` 标志：

```
$ rustc --cfg some_condition custom.rs && ./custom
condition met!
```

泛型

泛型是一个关于将类型和功能泛化以适用于更广泛情况的主题。这在多方面都非常有用，可以大大减少代码重复，但可能需要相对复杂的语法。具体来说，使用泛型需要非常谨慎地指定泛型类型在哪些类型上是有效的。泛型最简单和最常见的用途是类型参数。

类型参数通过使用尖括号和大写**驼峰命名法**来指定为泛型：<Aaa, Bbb, ...>。“泛型类型参数”通常表示为 <T>。在 Rust 中，“泛型”也用来描述任何接受一个或多个泛型类型参数 <T> 的东西。任何被指定为泛型类型参数的类型都是泛型的，而其他所有类型都是具体的（非泛型）。

例如，定义一个名为 `foo` 的**泛型函数**，它接受一个任意类型的参数 `T`：

```
fn foo<T>(arg: T) { ... }
```

因为 `T` 已经使用 <T> 指定为泛型类型参数，所以在这里用作 `(arg: T)` 时被视为泛型。即使 `T` 之前已被定义为一个 `struct`，这种情况也成立。

下面的例子展示了一些泛型语法的实际应用：

```
1 // 具体类型 `A`
2 struct A;
3
4 // 定义 `Single` 类型时，首次使用 `A` 前没有 ``
5 // 因此，`Single` 是具体类型，`A` 即上面定义的类型
6 struct Single(A);
7 //           ^ 这里是 `Single` 首次使用 `A` 类型
8
9 // 这里 `` 在首次使用 `T` 之前，所以 `SingleGen` 是泛型
10 // 由于类型参数 `T` 是泛型，它可以是任何类型
11 // 包括上面定义的具体类型 `A`
12 struct SingleGen<T>(T);
13
14 fn main() {
15     // `Single` 是具体类型，明确接受 `A`
16     let _s = Single(A);
17
18     // 创建 `SingleGen<char>` 类型的变量 `_char`
19     // 并赋值为 `SingleGen('a')`
20     // 这里 `SingleGen` 明确指定了类型参数
21     let _char: SingleGen<char> = SingleGen('a');
22
23     // `SingleGen` 也可以隐式指定类型参数：
24     let _t = SingleGen(A); // 使用上面定义的 `A`
25     let _i32 = SingleGen(6); // 使用 `i32`
26     let _char = SingleGen('a'); // 使用 `char`
27 }
```

另请参阅：

[结构体](#)



函数

同样的规则也适用于函数：当类型 `T` 前面加上 `<T>` 时，它就变成了泛型。

使用泛型函数有时需要明确指定类型参数。这种情况可能出现在函数返回类型是泛型时，或者编译器没有足够信息推断必要的类型参数时。

明确指定类型参数的函数调用看起来像这样： `fun::<A, B, ...>()`。

```

1 struct A;           // 具体类型 `A`
2 struct S(A);        // 具体类型 `S`
3 struct SGen<T>(T);  // 泛型 `SGen`
4
5 // 以下函数都获取传入变量的所有权
6 // 并立即离开作用域，释放该变量
7
8 // 定义函数 `reg_fn`，接受一个 `S` 类型的参数 `_s`
9 // 由于没有 ``，所以这不是泛型函数
10 fn reg_fn(_s: S) {}
11
12 // 定义函数 `gen_spec_t`，接受一个 `SGen<T>` 类型的参数 `_s`
13 // 虽然明确给定了类型参数 `A`，但因为 `A` 并未被指定为
14 // `gen_spec_t` 的泛型类型参数，所以这个函数不是泛型的
15 fn gen_spec_t(_s: SGen<A>) {}
16
17 // 定义函数 `gen_spec_i32`，接受一个 `SGen<i32>` 类型的参数 `_s`
18 // 明确指定了类型参数 `i32`，这是一个具体类型
19 // 由于 `i32` 不是泛型类型，所以这个函数也不是泛型的
20 fn gen_spec_i32(_s: SGen<i32>) {}
21
22 // 定义函数 `generic`，接受一个 `SGen<T>` 类型的参数 `_s`
23 // 由于 `SGen<T>` 前面有 ``，所以这个函数是关于 `T` 的泛型函数
24 fn generic<T>(_s: SGen<T>) {}
25
26 fn main() {
27     // 使用非泛型函数
28     reg_fn(S(A));           // 具体类型
29     gen_spec_t(SGen(A));    // 隐式指定类型参数 `A`
30     gen_spec_i32(SGen(6));  // 隐式指定类型参数 `i32`
31
32     // 为 `generic()` 显式指定类型参数 `char`
33     generic::<char>(SGen('a'));
34
35     // 为 `generic()` 隐式指定类型参数 `char`
36     generic(SGen('c'));
37 }
```

另请参阅：

[函数和结构体](#)

实现

与函数类似，实现（`impl`）在涉及泛型时也需要谨慎处理。

```
struct S; // 具体类型 `S`
struct GenericVal<T>(T); // 泛型类型 `GenericVal`
```

```
// GenericVal 的实现，这里我们显式指定类型参数：
impl GenericVal<f32> {} // 指定 `f32`
impl GenericVal<S> {} // 指定上面定义的 `S`
```

```
// `` 必须放在类型前面以保持泛型
impl<T> GenericVal<T> {}
```

```
1 struct Val {
2     val: f64,
3 }
4
5 struct GenVal<T> {
6     gen_val: T,
7 }
8
9 // Val 的实现
10 impl Val {
11     fn value(&self) -> &f64 {
12         &self.val
13     }
14 }
15
16 // 为泛型类型 `T` 实现 GenVal
17 impl<T> GenVal<T> {
18     fn value(&self) -> &T {
19         &self.gen_val
20     }
21 }
22
23 fn main() {
24     let x = Val { val: 3.0 };
25     let y = GenVal { gen_val: 3i32 };
26
27     println!("{}", x.value(), y.value());
28 }
```

另请参阅：

[返回引用的函数](#)、[impl](#) 和 [结构体](#)

特质

当然，`trait` 也可以是泛型的。这里我们定义了一个泛型 `trait`，它重新实现了 `Drop` `trait`，用于释放自身和一个输入参数。

```
1  // 不可复制的类型。
2  struct Empty;
3  struct Null;
4
5  // 一个泛型 trait，使用类型参数 `T`。
6  trait DoubleDrop<T> {
7      // 在调用者类型上定义一个方法，该方法接受一个
8      // 额外的类型为 `T` 的参数，但不对其进行任何操作。
9      fn double_drop(self, _: T);
10 }
11
12 // 为任意泛型参数 `T` 和调用者 `U` 实现 `DoubleDrop<T>`。
13 impl<T, U> DoubleDrop<T> for U {
14     // 此方法获取两个传入参数的所有权，
15     // 并释放它们的内存。
16     fn double_drop(self, _: T) {}
17 }
18
19 fn main() {
20     let empty = Empty;
21     let null = Null;
22
23     // 释放 `empty` 和 `null` 的内存。
24     empty.double_drop(null);
25
26     //empty;
27     //null;
28     // ^ TODO: 尝试取消这些行的注释。
29 }
```

另请参阅：

`Drop`、`struct` 和 `trait`

约束

在使用泛型时，类型参数通常需要使用 trait 作为**约束**，以规定类型应实现哪些功能。例如，下面的示例使用 `Display` trait 来打印，因此它要求 `T` 必须受 `Display` 约束；换句话说，`T` **必须实现** `Display`。

```
// 定义一个函数 `printer`，它接受一个泛型类型 `T`，  
// 该类型必须实现 `Display` trait。  
fn printer<T: Display>(t: T) {  
    println!("{}", t);  
}
```

约束将泛型限制为符合约束条件的类型。也就是说：

```
struct S<T: Display>(T);  
  
// 错误！`Vec<T>` 没有实现 `Display`。  
// 这个特化将会失败。  
let s = S(vec![1]);
```

约束的另一个作用是允许泛型实例访问约束中指定的 trait 的[方法](#)。例如：

```
1 // 实现打印标记 `{:?}` 的 trait。
2 use std::fmt::Debug;
3
4 trait HasArea {
5     fn area(&self) -> f64;
6 }
7
8 impl HasArea for Rectangle {
9     fn area(&self) -> f64 { self.length * self.height }
10 }
11
12 #[derive(Debug)]
13 struct Rectangle { length: f64, height: f64 }
14 #[allow(dead_code)]
15 struct Triangle { length: f64, height: f64 }
16
17 // 泛型 `T` 必须实现 `Debug`。无论 `T` 是什么类型，
18 // 这个函数都能正常工作。
19 fn print_debug<T: Debug>(t: &T) {
20     println!("{:?}", t);
21 }
22
23 // `T` 必须实现 `HasArea`。任何满足这个约束的类型
24 // 都可以访问 `HasArea` 的 `area` 方法。
25 fn area<T: HasArea>(t: &T) -> f64 { t.area() }
26
27 fn main() {
28     let rectangle = Rectangle { length: 3.0, height: 4.0 };
29     let _triangle = Triangle { length: 3.0, height: 4.0 };
30
31     print_debug(&rectangle);
32     println!("面积: {}", area(&rectangle));
33
34     //print_debug(&_triangle);
35     //println!("面积: {}", area(&_triangle));
36     // ^ TODO: 尝试取消这些注释。
37     // | 错误: 未实现 `Debug` 或 `HasArea`。
38 }
```

另外值得注意的是，在某些情况下可以使用 `where` 子句来应用约束，以使表达更加清晰。

另请参阅：

`std::fmt`、`struct` 和 `trait`

测试实例：空约束

约束的工作机制导致即使一个 `trait` 不包含任何功能，你仍然可以将其用作约束。`std` 库中的 `Eq` 和 `Copy` 就是这种 `trait` 的例子。

```
1 struct Cardinal;
2 struct BlueJay;
3 struct Turkey;
4
5 trait Red {}
6 trait Blue {}
7
8 impl Red for Cardinal {}
9 impl Blue for BlueJay {}
10
11 // 这些函数只对实现了这些 trait 的类型有效。
12 // 这些 trait 是否为空并不重要。
13 fn red<T: Red>(_: &T) -> &'static str { "红色" }
14 fn blue<T: Blue>(_: &T) -> &'static str { "蓝色" }
15
16 fn main() {
17     let cardinal = Cardinal;
18     let blue_jay = BlueJay;
19     let _turkey = Turkey;
20
21     // 由于约束的存在，`red()` 不能用于蓝松鸟，反之亦然。
22     println!("红雀是{}", red(&cardinal));
23     println!("蓝松鸟是{}", blue(&blue_jay));
24     //println!("火鸡是{}", red(&_turkey));
25     // ^ TODO: 尝试取消这行的注释。
26 }
```

另请参阅：

`std::cmp::Eq`、`std::marker::Copy` 和 `trait`

多重约束

可以使用 `+` 为单个类型指定多个约束。按照惯例，不同的类型用 `,` 分隔。

```
1 use std::fmt::{Debug, Display};
2
3 fn compare_prints<T: Debug + Display>(t: &T) {
4     println!("Debug: `{:?}`", t);
5     println!("Display: `{}`", t);
6 }
7
8 fn compare_types<T: Debug, U: Debug>(t: &T, u: &U) {
9     println!("t: `{:?}`", t);
10    println!("u: `{:?}`", u);
11 }
12
13 fn main() {
14     let string = "words";
15     let array = [1, 2, 3];
16     let vec = vec![1, 2, 3];
17
18     compare_prints(&string);
19     //compare_prints(&array);
20     // TODO: 尝试取消此行注释。
21
22     compare_types(&array, &vec);
23 }
```

另请参阅：

`std::fmt` 和 `trait`

Where 分句

约束也可以使用 `where` 子句来表达，它位于开括号 `{` 之前，而不是在类型首次提及时。此外，`where` 子句可以将约束应用于任意类型，而不仅限于类型参数。

`where` 子句在以下情况下特别有用：

- 当单独指定泛型类型和约束更清晰时：

```
impl <A: TraitB + TraitC, D: TraitE + TraitF> MyTrait<A, D> for YourType {}
```

// 使用 ``where`` 子句表达约束

```
impl <A, D> MyTrait<A, D> for YourType where
    A: TraitB + TraitC,
    D: TraitE + TraitF {}
```

- 当使用 `where` 子句比使用普通语法更具表现力时。这个例子中的 `impl` 如果不使用 `where` 子句就无法直接表达：

```
1 use std::fmt::Debug;
2
3 trait PrintInOption {
4     fn print_in_option(self);
5 }
6
7 // 这里需要一个 `where` 子句：否则就必须将其表达为 `T: Debug` 或
8 // 使用另一种间接方法，
9 impl<T> PrintInOption for T where
10     Option<T>: Debug {
11     // 我们需要 `Option<T>: Debug` 作为我们的约束，因为这是
12     // 正在被打印的内容。否则就会使用错误的约束。
13     fn print_in_option(self) {
14         println!("{:?}", Some(self));
15     }
16 }
17
18 fn main() {
19     let vec = vec![1, 2, 3];
20
21     vec.print_in_option();
22 }
```

另请参阅：

[RFC](#)、[struct](#) 和 [trait](#)

新类型惯用法

`newtype` 模式在编译时保证了程序接收到正确类型的值。

For example, a function that measures distance in miles, *must* be given a value of type `Miles`.

```

1 struct Miles(f64);
2
3 struct Kilometers(f64);
4
5 impl Miles {
6     pub fn to_kilometers(&self) -> Kilometers {
7         Kilometers(self.0 * 1.609344)
8     }
9 }
10
11 impl Kilometers {
12     pub fn to_miles(&self) -> Miles {
13         Miles(self.0 / 1.609344)
14     }
15 }
16
17 fn is_a_marathon(distance: &Miles) -> bool {
18     distance.0 >= 26.2
19 }
20
21 fn main() {
22     let distance = Miles(30.0);
23     let distance_km = distance.to_kilometers();
24     println!("Is a marathon? {}", is_a_marathon(&distance));
25     println!("Is a marathon? {}", is_a_marathon(&distance_km.to_miles()));
26     // println!("Is a marathon? {}", is_a_marathon(&distance_km));
27 }

```

Uncomment the last print statement to observe that the type supplied must be `Miles`.

要获取 `newtype` 的基本类型值，你可以使用元组语法或解构语法，如下所示：

```

1 struct Miles(f64);
2
3 fn main() {
4     let distance = Miles(42.0);
5     let distance_as_primitive_1: f64 = distance.0; // 元组语法
6     let Miles(distance_as_primitive_2) = distance; // 解构语法
7 }

```

另请参阅：

`struct`

关联项

“关联项”是指与各种类型的 [项](#) 相关的一组规则。它是 `trait` 泛型的扩展，允许 `trait` 在内部定义新的项。

其中一种项被称为**关联类型**，当 `trait` 对其容器类型是泛型时，它提供了更简洁的使用模式。

另请参阅：

[RFC](#)

问题

对于容器类型是泛型的 `trait`，有类型规范要求——`trait` 的使用者**必须**指定所有的泛型类型。

在下面的例子中，`Contains` `trait` 允许使用泛型类型 `A` 和 `B`。然后为 `Container` 类型实现该 `trait`，将 `A` 和 `B` 指定为 `i32`，以便与 `fn difference()` 一起使用。

由于 `Contains` 是泛型的，我们不得不为 `fn difference()` 显式声明**所有**泛型类型。实际上，我们希望有一种方法来表达 `A` 和 `B` 是由**输入** `C` 决定的。正如你将在下一节中看到的，关联类型恰好提供了这种能力。

```
1 struct Container(i32, i32);
2
3 // 一个检查容器内是否存储了两个项的 trait。
4 // 同时可以检索第一个或最后一个值。
5 trait Contains<A, B> {
6     fn contains(&self, _: &A, _: &B) -> bool; // 显式要求 `A` 和 `B`
7     fn first(&self) -> i32; // 不需要显式指定 `A` 或 `B`
8     fn last(&self) -> i32; // 不需要显式指定 `A` 或 `B`
9 }
10
11 impl Contains<i32, i32> for Container {
12     // 如果存储的数字相等则返回 true
13     fn contains(&self, number_1: &i32, number_2: &i32) -> bool {
14         (&self.0 == number_1) && (&self.1 == number_2)
15     }
16
17     // 获取第一个数字
18     fn first(&self) -> i32 { self.0 }
19
20     // 获取最后一个数字
21     fn last(&self) -> i32 { self.1 }
22 }
23
24 // `C` 已经包含了 `A` 和 `B`。考虑到这一点，
25 // 再次指定 `A` 和 `B` 就显得多余且麻烦。
26 fn difference<A, B, C>(container: &C) -> i32 where
27     C: Contains<A, B> {
28     container.last() - container.first()
29 }
30
31 fn main() {
32     let number_1 = 3;
33     let number_2 = 10;
34
35     let container = Container(number_1, number_2);
36
37     println!("容器是否包含 {} 和 {}: {}",
38         &number_1, &number_2,
39         container.contains(&number_1, &number_2));
40     println!("第一个数字: {}", container.first());
41     println!("最后一个数字: {}", container.last());
42
43     println!("差值为: {}", difference(&container));
44 }
```

另请参阅:

`struct` 和 `trait`

关联类型

使用“关联类型”通过将内部类型局部移动到 trait 中作为**输出**类型，提高了代码的整体可读性。trait 定义的语法如下：

```
// `A` 和 `B` 在 trait 中通过 `type` 关键字定义。  
// （注意：这里的 `type` 与用于类型别名的 `type` 不同）  
trait Contains {  
    type A;  
    type B;  
  
    // 更新后的语法，用于泛型地引用这些新类型  
    fn contains(&self, _: &Self::A, _: &Self::B) -> bool;  
}
```

注意，使用 Contains trait 的函数不再需要显式指定 A 或 B：

```
// 不使用关联类型  
fn difference<A, B, C>(container: &C) -> i32 where  
    C: Contains<A, B> { ... }  
  
// 使用关联类型  
fn difference<C: Contains>(container: &C) -> i32 { ... }
```

让我们使用关联类型重写上一节的示例：

```
1 struct Container(i32, i32);
2
3 // 一个检查容器内是否存储了两个项的 trait。
4 // 同时可以检索第一个或最后一个值。
5 trait Contains {
6     // 在此定义泛型类型，方法将能够使用这些类型
7     type A;
8     type B;
9
10    fn contains(&self, _: &Self::A, _: &Self::B) -> bool;
11    fn first(&self) -> i32;
12    fn last(&self) -> i32;
13 }
14
15 impl Contains for Container {
16     // 指定 `A` 和 `B` 的具体类型。如果 `input` 类型
17     // 是 `Container(i32, i32)`，那么 `output` 类型
18     // 就被确定为 `i32` 和 `i32`
19     type A = i32;
20     type B = i32;
21
22     // 在这里使用 `&Self::A` 和 `&Self::B` 也是有效的。
23     fn contains(&self, number_1: &i32, number_2: &i32) -> bool {
24         (&self.0 == number_1) && (&self.1 == number_2)
25     }
26     // 获取第一个数字
27     fn first(&self) -> i32 { self.0 }
28
29     // 获取最后一个数字
30     fn last(&self) -> i32 { self.1 }
31 }
32
33 fn difference<C: Contains>(container: &C) -> i32 {
34     container.last() - container.first()
35 }
36
37 fn main() {
38     let number_1 = 3;
39     let number_2 = 10;
40
41     let container = Container(number_1, number_2);
42
43     println!("容器是否包含 {} 和 {}: {}",
44         &number_1, &number_2,
45         container.contains(&number_1, &number_2));
46     println!("第一个数字: {}", container.first());
47     println!("最后一个数字: {}", container.last());
48
49     println!("差值为: {}", difference(&container));
50 }
```

虚类型参数

虚类型参数是一种在运行时不会出现，但在编译时会进行静态检查的类型参数。

数据类型可以使用额外的泛型类型参数作为标记，或在编译时进行类型检查。这些额外的参数不占用存储空间，也没有运行时行为。

在下面的示例中，我们将 `std::marker::PhantomData` 与虚类型参数的概念结合，创建包含不同数据类型的元组。

```
1 use std::marker::PhantomData;
2
3 // 一个虚元组结构体，它在 `A` 上是泛型的，带有隐藏参数 `B`。
4 #[derive(PartialEq)] // 允许对此类型进行相等性测试。
5 struct PhantomTuple<A, B>(A, PhantomData<B>);
6
7 // 一个虚类型结构体，它在 `A` 上是泛型的，带有隐藏参数 `B`。
8 #[derive(PartialEq)] // 允许对此类型进行相等性测试。
9 struct PhantomStruct<A, B> { first: A, phantom: PhantomData<B> }
10
11 // 注意：为泛型类型 `A` 分配了存储空间，但没有为 `B` 分配。
12 // 因此，`B` 不能用于计算。
13
14 fn main() {
15     // 这里，`f32` 和 `f64` 是隐藏参数。
16     // PhantomTuple 类型指定为 ``。
17     let _tuple1: PhantomTuple<char, f32> = PhantomTuple('Q', PhantomData)
18     // PhantomTuple 类型指定为 ``。
19     let _tuple2: PhantomTuple<char, f64> = PhantomTuple('Q', PhantomData)
20
21     // 类型指定为 ``。
22     let _struct1: PhantomStruct<char, f32> = PhantomStruct {
23         first: 'Q',
24         phantom: PhantomData,
25     };
26     // 类型指定为 ``。
27     let _struct2: PhantomStruct<char, f64> = PhantomStruct {
28         first: 'Q',
29         phantom: PhantomData,
30     };
31
32     // 编译时错误！类型不匹配，无法比较：
33     // println!("_tuple1 == _tuple2 的结果是：{}",
34     //         _tuple1 == _tuple2);
35
36     // 编译时错误！类型不匹配，无法比较：
37     // println!("_struct1 == _struct2 的结果是：{}",
38     //         _struct1 == _struct2);
39 }
```

另请参阅：

[Derive](#), [struct](#), and [tuple](#).

测试实例：单位澄清

通过使用虚类型参数实现 `Add` trait，我们可以探索一种有用的单位转换方法。下面我们来看看 `Add` trait：

```
// 这个结构会强制要求：`Self + RHS = Output`  
// 其中，如果在实现中未指定 RHS，它将默认为 Self。  
pub trait Add<RHS = Self> {  
    type Output;  
  
    fn add(self, rhs: RHS) -> Self::Output;  
}  
  
// `Output` 必须是 `T<U>`，以确保 `T<U> + T<U> = T<U>`。  
impl<U> Add for T<U> {  
    type Output = T<U>;  
    ...  
}
```

完整实现如下：

```
1 use std::ops::Add;
2 use std::marker::PhantomData;
3
4 /// 创建空枚举以定义单位类型。
5 #[derive(Debug, Clone, Copy)]
6 enum Inch {}
7 #[derive(Debug, Clone, Copy)]
8 enum Mm {}
9
10 /// `Length` 是一个带有虚类型参数 `Unit` 的类型，
11 /// 它不是长度类型（即 `f64`）的泛型。
12 ///
13 /// `f64` 已经实现了 `Clone` 和 `Copy` trait。
14 #[derive(Debug, Clone, Copy)]
15 struct Length<Unit>(<f64, PhantomData<Unit>);
16
17 /// `Add` trait 定义了 `+` 运算符的行为。
18 impl<Unit> Add for Length<Unit> {
19     type Output = Length<Unit>;
20
21     // add() 返回一个包含和的新 `Length` 结构体。
22     fn add(self, rhs: Length<Unit>) -> Length<Unit> {
23         // `+` 调用 `f64` 的 `Add` 实现。
24         Length(self.0 + rhs.0, PhantomData)
25     }
26 }
27
28 fn main() {
29     // 指定 `one_foot` 具有虚类型参数 `Inch`。
30     let one_foot: Length<Inch> = Length(12.0, PhantomData);
31     // `one_meter` 具有虚类型参数 `Mm`。
32     let one_meter: Length<Mm> = Length(1000.0, PhantomData);
33
34     // `+` 调用我们为 `Length<Unit>` 实现的 `add()` 方法。
35     //
36     // 由于 `Length` 实现了 `Copy`，`add()` 不会消耗
37     // `one_foot` 和 `one_meter`，而是将它们复制到 `self` 和 `rhs` 中。
38     let two_feet = one_foot + one_foot;
39     let two_meters = one_meter + one_meter;
40
41     // 加法运算正常工作。
42     println!("一英尺 + 一英尺 = {:?} 英寸", two_feet.0);
43     println!("一米 + 一米 = {:?} 毫米", two_meters.0);
44
45     // 无意义的操作会按预期失败：
46     // 编译时错误：类型不匹配。
47     //let one_feter = one_foot + one_meter;
48 }
```

另请参阅：

借用（&）、约束（x: y）、枚举、impl 和 self、运算符重载、ref、trait（x for y）以及元组结构体。

作用域规则

作用域在所有权、借用和生命周期中扮演着重要角色。它们向编译器指示借用何时有效、资源何时可以被释放，以及变量何时被创建或销毁。

RAII

Rust 中的变量不仅仅是在栈上保存数据：它们还**拥有**资源，例如 `Box<T>` 拥有堆上的内存。Rust 强制执行 **RAII**（资源获取即初始化），因此每当一个对象离开作用域时，它的析构函数就会被调用，它拥有的资源也会被释放。

这种行为可以防止**资源泄漏**错误，因此你再也不用手动释放内存或担心内存泄漏了！以下是一个简单示例：

```
1 // raii.rs
2 fn create_box() {
3     // 在堆上分配一个整数
4     let _box1 = Box::new(3i32);
5
6     // `_box1` 在此处被销毁，内存被释放
7 }
8
9 fn main() {
10    // 在堆上分配一个整数
11    let _box2 = Box::new(5i32);
12
13    // 一个嵌套的作用域：
14    {
15        // 在堆上分配一个整数
16        let _box3 = Box::new(4i32);
17
18        // `_box3` 在此处被销毁，内存被释放
19    }
20
21    // 创建大量的 box（仅为演示）
22    // 无需手动释放内存！
23    for _ in 0u32..1_000 {
24        create_box();
25    }
26
27    // `_box2` 在此处被销毁，内存被释放
28 }
```

当然，我们可以使用 `valgrind` 来再次检查内存错误：

```
$ rustc raii.rs && valgrind ./raii
==26873== Memcheck, a memory error detector
==26873== Copyright (C) 2002-2013, and GNU GPL'd, by Julian Seward et al.
==26873== Using Valgrind-3.9.0 and LibVEX; rerun with -h for copyright info
==26873== Command: ./raii
==26873==
==26873==
==26873== HEAP SUMMARY:
==26873==       in use at exit: 0 bytes in 0 blocks
==26873==    total heap usage: 1,013 allocs, 1,013 frees, 8,696 bytes
allocated
==26873==
==26873== All heap blocks were freed -- no leaks are possible
==26873==
==26873== For counts of detected and suppressed errors, rerun with: -v
==26873== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 2 from 2)
```

这里没有内存泄漏！

析构函数

Rust 中的析构函数概念是通过 `Drop` trait 提供的。当资源离开作用域时，析构函数会被调用。并非每种类型都需要实现这个 trait，只有当你需要为自己的类型实现特定的析构逻辑时才需要实现它。

运行下面的示例来了解 `Drop` trait 是如何工作的。当 `main` 函数中的变量离开作用域时，自定义的析构函数将被调用。

```
1 struct ToDrop;
2
3 impl Drop for ToDrop {
4     fn drop(&mut self) {
5         println!("ToDrop 正在被丢弃");
6     }
7 }
8
9 fn main() {
10     let x = ToDrop;
11     println!("创建了一个 ToDrop!");
12 }
```

另请参阅：

[Box](#)

所有权和移动

由于变量负责释放它们自己的资源，**资源只能有一个所有者**。这可以防止资源被多次释放。请注意，并非所有变量都拥有资源（例如[引用](#)）。

当进行赋值（`let x = y`）或按值传递函数参数（`foo(x)`）时，资源的**所有权**会被转移。在 Rust 中，这被称为**移动**（move）。

资源移动后，原所有者将无法再被使用。这避免了悬垂指针的产生。

```
1  // 此函数获取堆分配内存的所有权
2  fn destroy_box(c: Box<i32>) {
3      println!("正在销毁一个包含 {} 的 box", c);
4
5      // `c` 被销毁，内存被释放
6  }
7
8  fn main() {
9      // **栈**分配的整数
10     let x = 5u32;
11
12     // 将 `x` **复制**到 `y` - 没有资源被移动
13     let y = x;
14
15     // 两个值可以独立使用
16     println!("x 是 {}, y 是 {}", x, y);
17
18     // `a` 是指向**堆**分配整数的指针
19     let a = Box::new(5i32);
20
21     println!("a 包含: {}", a);
22
23     // 将 `a` **移动**到 `b`
24     let b = a;
25     // `a` 的指针地址（而非数据）被复制到 `b`
26     // 现在两者都指向同一块堆分配的数据
27     // 但 `b` 现在拥有它
28
29     // 错误! `a` 不再拥有堆内存，因此无法访问数据
30     //println!("a 包含: {}", a);
31     // TODO ^ 尝试取消此行注释
32
33     // 此函数从 `b` 获取堆分配内存的所有权
34     destroy_box(b);
35
36     // 此时堆内存已被释放，这个操作会导致解引用已释放的内存
37     // 但编译器禁止这样做
38     // 错误! 原因与前面的错误相同
39     //println!("b 包含: {}", b);
40     // TODO ^ 尝试取消此行注释
41 }
```

可变性

当所有权转移时，数据的可变性可以改变。

```
1 fn main() {
2     let immutable_box = Box::new(5u32);
3
4     println!("immutable_box 包含 {}", immutable_box);
5
6     // 可变性错误
7     /*immutable_box = 4;
8
9     // *移动*盒子，改变所有权（和可变性）
10    let mut mutable_box = immutable_box;
11
12    println!("mutable_box 包含 {}", mutable_box);
13
14    // 修改盒子的内容
15    *mutable_box = 4;
16
17    println!("mutable_box 现在包含 {}", mutable_box);
18 }
```

部分移动

在对单个变量进行[解构](#)时，可以同时使用 `by-move` 和 `by-reference` 模式绑定。这样做会导致变量的**部分移动**，这意味着变量的一部分会被移动，而其他部分保持不变。在这种情况下，父级变量之后不能作为一个整体使用，但是仍然可以使用只被引用（而不是被移动）的部分。注意，实现了 `Drop` 特质的类型不能被部分移动，因为其 `drop` 方法会在之后将其作为整体使用。

```

1 fn main() {
2     #[derive(Debug)]
3     struct Person {
4         name: String,
5         age: Box<u8>,
6     }
7
8     // 错误！无法从实现了 `Drop` 特质的类型中移动
9     //impl Drop for Person {
10    //    fn drop(&mut self) {
11    //        println!("Dropping the person struct {:?}", self)
12    //    }
13    //}
14    // TODO ^ 尝试取消注释这些行
15
16    let person = Person {
17        name: String::from("Alice"),
18        age: Box::new(20),
19    };
20
21    // `name` 从 person 中被移出，但 `age` 只是被引用
22    let Person { name, ref age } = person;
23
24    println!("此人的年龄是 {}", age);
25
26    println!("此人的姓名是 {}", name);
27
28    // 错误！借用部分移动的值：`person` 发生部分移动
29    //println!("person 结构体是 {:?}", person);
30
31    // `person` 不能使用，但 `person.age` 可以使用，因为它没有被移动
32    println!("从 person 结构体中获取的年龄是 {}", person.age);
33 }
```

（在这个例子中，我们将 `age` 变量存储在堆上以说明部分移动：删除上面代码中的 `ref` 会导致错误，因为 `person.age` 的所有权会被移动到变量 `age`。如果 `Person.age` 存储在栈上，就不需要 `ref`，因为 `age` 的定义会从 `person.age` 复制数据而不是移动它。）

另请参阅：

[解构](#)

借用

大多数情况下，我们希望访问数据而不获取其所有权。为了实现这一点，Rust 使用了**借用**机制。对象可以通过引用（`&T`）传递，而不是按值（`T`）传递。

编译器通过其借用检查器静态地保证引用**始终**指向有效的对象。也就是说，当存在指向一个对象的引用时，该对象就不能被销毁。

```
1  // 此函数获取一个盒子的所有权并销毁它
2  fn eat_box_i32(boxed_i32: Box<i32>) {
3      println!("正在销毁包含 {} 的盒子", boxed_i32);
4  }
5
6  // 此函数借用一个 i32
7  fn borrow_i32(borrowed_i32: &i32) {
8      println!("这个整数是: {}", borrowed_i32);
9  }
10
11 fn main() {
12     // 在堆上创建一个装箱的 i32，在栈上创建一个 i32
13     // 注意：数字可以添加任意下划线以提高可读性
14     // 5_i32 与 5i32 相同
15     let boxed_i32 = Box::new(5_i32);
16     let stacked_i32 = 6_i32;
17
18     // 借用盒子的数据。所有权未被获取，
19     // 所以数据可以再次被借用。
20     borrow_i32(&boxed_i32);
21     borrow_i32(&stacked_i32);
22
23     {
24         // 获取盒子内数据的引用
25         let _ref_to_i32: &i32 = &boxed_i32;
26
27         // 错误！
28         // 当内部值在作用域内稍后被借用时，不能销毁 `boxed_i32`。
29         eat_box_i32(boxed_i32);
30         // 修复：^ 注释掉此行
31
32         // 尝试在内部值被销毁后借用 `_ref_to_i32`
33         borrow_i32(_ref_to_i32);
34         // `_ref_to_i32` 超出作用域，不再被借用。
35     }
36
37     // `boxed_i32` 现在可以将所有权交给 `eat_box_i32` 并被销毁
38     eat_box_i32(boxed_i32);
39 }
```

可变性

可变数据可以使用 `&mut T` 进行可变借用。这被称为**可变引用**，并给予借用者读写访问权。相比之下，`&T` 通过不可变引用借用数据，借用者可以读取数据但不能修改它：

```
1  #[allow(dead_code)]
2  #[derive(Clone, Copy)]
3  struct Book {
4      // '&'static str 是对分配在只读内存中的字符串的引用
5      author: &'static str,
6      title: &'static str,
7      year: u32,
8  }
9
10 // 此函数接受一个对 Book 类型的引用
11 fn borrow_book(book: &Book) {
12     println!("我不可变地借用了《{}》- {} 版", book.title, book.year);
13 }
14
15 // 此函数接受一个对可变 Book 的引用，并将 'year' 改为 2014
16 fn new_edition(book: &mut Book) {
17     book.year = 2014;
18     println!("我可变地借用了《{}》- {} 版", book.title, book.year);
19 }
20
21 fn main() {
22     // 创建一个名为 'immutabook' 的不可变 Book 实例
23     let immutabook = Book {
24         // 字符串字面量的类型是 '&'static str
25         author: "Douglas Hofstadter",
26         title: "哥德尔、埃舍尔、巴赫",
27         year: 1979,
28     };
29
30     // 创建 'immutabook' 的可变副本，并命名为 'mutabook'
31     let mut mutabook = immutabook;
32
33     // 不可变地借用一个不可变对象
34     borrow_book(&immutabook);
35
36     // 不可变地借用一个可变对象
37     borrow_book(&mutabook);
38
39     // 可变地借用一个可变对象
40     new_edition(&mut mutabook);
41
42     // 错误！不能将不可变对象作为可变对象借用
43     new_edition(&mut immutabook);
44     // 修复：^ 注释掉此行
45 }
```

另请参阅：

`static`

别名

数据可以被不可变借用任意次数，但在不可变借用期间，原始数据不能被可变借用。另一方面，同一时间只允许一个可变借用。只有在可变引用最后一次使用之后，原始数据才能再次被借用。

```
1 struct Point { x: i32, y: i32, z: i32 }
2
3 fn main() {
4     let mut point = Point { x: 0, y: 0, z: 0 };
5
6     let borrowed_point = &point;
7     let another_borrow = &point;
8
9     // 可以通过引用和原始所有者访问数据
10    println!("点的坐标为: ( {}, {}, {})",
11             borrowed_point.x, another_borrow.y, point.z);
12
13    // 错误! 不能将 `point` 作为可变借用，因为它当前被不可变借用。
14    // let mutable_borrow = &mut point;
15    // TODO ^ 尝试取消注释此行
16
17    // 这里再次使用了借用的值
18    println!("点的坐标为: ( {}, {}, {})",
19             borrowed_point.x, another_borrow.y, point.z);
20
21    // 不可变引用在代码的剩余部分不再使用
22    // 因此可以用可变引用重新借用。
23    let mutable_borrow = &mut point;
24
25    // 通过可变引用修改数据
26    mutable_borrow.x = 5;
27    mutable_borrow.y = 2;
28    mutable_borrow.z = 1;
29
30    // 错误! 不能将 `point` 作为不可变借用，因为它当前被可变借用。
31    // let y = &point.y;
32    // TODO ^ 尝试取消注释此行
33
34    // 错误! 无法打印，因为 `println!` 需要一个不可变引用。
35    // println!("点的 Z 坐标是 {}", point.z);
36    // TODO ^ 尝试取消注释此行
37
38    // 正确! 可变引用可以作为不可变引用传递给 `println!`
39    println!("点的坐标为: ( {}, {}, {})",
40             mutable_borrow.x, mutable_borrow.y, mutable_borrow.z);
41
42    // 可变引用在代码的剩余部分不再使用，所以可以重新借用
43    let new_borrowed_point = &point;
44    println!("点现在的坐标为: ( {}, {}, {})",
45             new_borrowed_point.x, new_borrowed_point.y, new_borrowed_point.z);
46 }
```

ref 模式

在使用 `let` 绑定进行模式匹配或解构时，可以使用 `ref` 关键字来获取结构体或元组字段的引用。以下示例展示了几个这种用法有用的场景：

```
1  #[derive(Clone, Copy)]
2  struct Point { x: i32, y: i32 }
3
4  fn main() {
5      let c = 'Q';
6
7      // 赋值左侧的 `ref` 借用等同于右侧的 `&` 借用。
8      let ref ref_c1 = c;
9      let ref_c2 = &c;
10
11     println!("ref_c1 等于 ref_c2: {}", *ref_c1 == *ref_c2);
12
13     let point = Point { x: 0, y: 0 };
14
15     // 在解构结构体时，`ref` 也是有效的。
16     let _copy_of_x = {
17         // `ref_to_x` 是 `point` 的 `x` 字段的引用。
18         let Point { x: ref ref_to_x, y: _ } = point;
19
20         // 返回 `point` 的 `x` 字段的副本。
21         *ref_to_x
22     };
23
24     // `point` 的可变副本
25     let mut mutable_point = point;
26
27     {
28         // `ref` 可以与 `mut` 配合使用来获取可变引用。
29         let Point { x: _, y: ref mut mut_ref_to_y } = mutable_point;
30
31         // 通过可变引用修改 `mutable_point` 的 `y` 字段。
32         *mut_ref_to_y = 1;
33     }
34
35     println!("point 的坐标是 ( {}, {} )", point.x, point.y);
36     println!("mutable_point 的坐标是 ( {}, {} )", mutable_point.x, mutable_
37
38     // 包含指针的可变元组
39     let mut mutable_tuple = (Box::new(5u32), 3u32);
40
41     {
42         // 解构 `mutable_tuple` 以修改 `last` 的值。
43         let (_, ref mut last) = mutable_tuple;
44         *last = 2u32;
45     }
46
47     println!("元组是 { :? }", mutable_tuple);
48 }
```

生命周期

生命周期是编译器（更具体地说是其**借用检查器**）用来确保所有借用都有效的一种机制。具体来说，变量的生命周期从创建时开始，到销毁时结束。尽管生命周期和作用域经常被一同提及，但它们并不完全相同。

举个例子，当我们通过 `&` 借用一个变量时，这个借用的生命周期由其声明位置决定。因此，只要借用 在出借者被销毁之前结束，它就是有效的。然而，借用的作用域则由引用使用的位置决定。

在接下来的例子和本节的其余部分中，我们将看到生命周期如何与作用域相关联，以及两者之间的区别。

```
1 // 下面用线条标注了每个变量的创建和销毁，以表示生命周期。
2 // `i` 的生命周期最长，因为它的作用域完全包含了 `borrow1` 和 `borrow2`。
3 // `borrow1` 相对于 `borrow2` 的持续时间是无紧要的，因为它们是不相交的。
4 // （译注：为避免中文的宽度显示问题，下面注释没有翻译）
5 fn main() {
6     let i = 3; // Lifetime for `i` starts.
7     //
8     { //
9         let borrow1 = &i; // `borrow1` lifetime starts.
10        //
11        println!("borrow1: {}", borrow1); //
12    } // `borrow1` ends.
13    //
14    //
15    { //
16        let borrow2 = &i; // `borrow2` lifetime starts.
17        //
18        println!("borrow2: {}", borrow2); //
19    } // `borrow2` ends.
20    //
21 }
```

请注意，生命周期标签没有被赋予名称或类型。这限制了生命周期的使用方式，我们将在后面看到这一点。

显式注解

借用检查器使用显式生命周期注解来确定引用应该有效多长时间。在生命周期没有被省略¹的情况下，Rust 需要显式注解来确定引用的生命周期。显式注解生命周期的语法使用撇号字符，如下所示：

```
foo<'a>
// `foo` 有一个生命周期参数 `a`
```

类似于闭包，使用生命周期需要泛型。此外，这种生命周期语法表示 `foo` 的生命周期不能超过 `'a` 的生命周期。类型的显式注解形式为 `&'a T`，其中 `'a` 已经被引入。

在有多个生命周期的情况下，语法类似：

```
foo<'a, 'b>
// `foo` 有生命周期参数 `a` 和 `b`
```

在这种情况下，`foo` 的生命周期不能超过 `'a` 或 `'b` 的生命周期。

请看下面的例子，展示了显式生命周期注解的使用：

```
1 // `print_refs` 接受两个 `i32` 的引用，它们有不同的
2 // 生命周期 `a` 和 `b`。这两个生命周期都必须
3 // 至少和函数 `print_refs` 一样长。
4 fn print_refs<a, b>(x: &a i32, y: &b i32) {
5     println!("x 是 {}, y 是 {}", x, y);
6 }
7
8 // 一个没有参数但有生命周期参数 `a` 的函数。
9 fn failed_borrow<a>() {
10     let _x = 12;
11
12     // 错误: `_x` 的生命周期不够长
13     let _y: &a i32 = &_x;
14     // 尝试在函数内部使用生命周期 `a` 作为显式类型标注会失败，
15     // 因为 `_x` 的生命周期比 `_y` 的短。
16     // 短生命周期无法强制转换为长生命周期。
17 }
18
19 fn main() {
20     // 创建将要被借用的变量。
21     let (four, nine) = (4, 9);
22
23     // 将两个变量的借用（`&`）传递给函数。
24     print_refs(&four, &nine);
25     // 任何被借用的输入必须比借用者存活更久。
26     // 换句话说，`four` 和 `nine` 的生命周期必须
27     // 比 `print_refs` 的生命周期更长。
28
29     failed_borrow();
30     // `failed_borrow` 不包含任何引用来强制 `a` 比函数的生命周期更长，
31     // 但 `a` 实际上更长。
32     // 因为生命周期从未被约束，它默认为 `static`。
33 }
```

另请参阅：

泛型 (generics) 和闭包 (closures)

1. [省略 \(elision\)](#) 隐式地注解生命周期，因此与显式注解不同。 [↩](#)

函数

排除省略 (elision) 的情况，带有生命周期的函数签名有以下几个约束：

- 任何引用**必须**有一个标注的生命周期。
- 任何被返回的引用**必须**具有与输入相同的生命周期或者是 `static`。

此外，请注意，如果返回引用而没有输入会导致返回指向无效数据的引用，这是被禁止的。以下示例展示了一些带有生命周期的有效函数形式：

```
1 // 一个具有生命周期 'a' 的输入引用，它必须至少与函数存活一样长。
2 fn print_one<'a>(x: &'a i32) {
3     println!("`print_one`: x 是 {}", x);
4 }
5
6 // 可变引用也可以有生命周期。
7 fn add_one<'a>(x: &'a mut i32) {
8     *x += 1;
9 }
10
11 // 多个具有不同生命周期的元素。在这种情况下，
12 // 两者都有相同的生命周期 'a' 是可以的，但
13 // 在更复杂的情况下，可能需要不同的生命周期。
14 fn print_multi<'a, 'b>(x: &'a i32, y: &'b i32) {
15     println!("`print_multi`: x 是 {}, y 是 {}", x, y);
16 }
17
18 // 返回已传入的引用是可以接受的。
19 // 但是，必须返回正确的生命周期。
20 fn pass_x<'a, 'b>(x: &'a i32, _: &'b i32) -> &'a i32 { x }
21
22 //fn invalid_output<'a>() -> &'a String { &String::from("foo") }
23 // 上面的代码是无效的：'a' 必须比函数存活更长。
24 // 这里的 &String::from("foo") 会创建一个 String，然后创建一个引用。
25 // 接着，在退出作用域时数据被丢弃，导致返回一个指向无效数据的引用。
26
27 fn main() {
28     let x = 7;
29     let y = 9;
30
31     print_one(&x);
32     print_multi(&x, &y);
33
34     let z = pass_x(&x, &y);
35     print_one(z);
36
37     let mut t = 3;
38     add_one(&mut t);
39     print_one(&t);
40 }
```

另请参阅：

[函数](#)

方法

方法的生命周期注解与函数类似：

```
1 struct Owner(i32);
2
3 impl Owner {
4     // 像独立函数一样注解生命周期。
5     fn add_one<'a>(&'a mut self) { self.0 += 1; }
6     fn print<'a>(&'a self) {
7         println!("`print`: {}", self.0);
8     }
9 }
10
11 fn main() {
12     let mut owner = Owner(18);
13
14     owner.add_one();
15     owner.print();
16 }
```

另请参阅：

[方法](#)

结构体

结构体中生命周期的注解也与函数类似：

```
1  // `Borrowed` 类型包含一个 `i32` 的引用。
2  // 这个 `i32` 的引用必须比 `Borrowed` 存活更久。
3  #[derive(Debug)]
4  struct Borrowed<'a>(&'a i32);
5
6  // 同样，这里的两个引用都必须比这个结构体存活更久。
7  #[derive(Debug)]
8  struct NamedBorrowed<'a> {
9      x: &'a i32,
10     y: &'a i32,
11 }
12
13 // 一个枚举，可以是 `i32`，或者是 `i32` 的引用。
14 #[derive(Debug)]
15 enum Either<'a> {
16     Num(i32),
17     Ref(&'a i32),
18 }
19
20 fn main() {
21     let x = 18;
22     let y = 15;
23
24     let single = Borrowed(&x);
25     let double = NamedBorrowed { x: &x, y: &y };
26     let reference = Either::Ref(&x);
27     let number    = Either::Num(y);
28
29     println!("x 在 {:?} 中被借用", single);
30     println!("x 和 y 在 {:?} 中被借用", double);
31     println!("x 在 {:?} 中被借用", reference);
32     println!("y 在 {:?} 中**没有**被借用", number);
33 }
```

另请参阅：

`struct` (结构体)

特质

trait 方法中的生命周期注解基本上与函数类似。注意，impl 也可能需要生命周期注解。

```
1 // 一个带有生命周期注解的结构体。
2 #[derive(Debug)]
3 struct Borrowed<'a> {
4     x: &'a i32,
5 }
6
7 // 为 impl 添加生命周期注解。
8 impl<'a> Default for Borrowed<'a> {
9     fn default() -> Self {
10         Self {
11             x: &10,
12         }
13     }
14 }
15
16 fn main() {
17     let b: Borrowed = Default::default();
18     println!("b 是 {:?}", b);
19 }
```

另请参阅：

[trait](#)

约束

正如泛型类型可以被约束一样，生命周期（本身也是泛型）也可以使用约束。这里的 `:` 符号含义略有不同，但 `+` 的用法相同。请注意以下表达的含义：

1. `T: 'a: T` 中的**所有**引用必须比生命周期 `'a` 存活更久。
2. `T: Trait + 'a: 类型 T` 必须实现 `trait Trait`，并且 `T` 中的**所有**引用必须比 `'a` 存活更久。

下面的例子展示了上述语法在 `where` 关键字之后的实际应用：

```
1 use std::fmt::Debug; // 用于约束的 trait。
2
3 #[derive(Debug)]
4 struct Ref<'a, T: 'a>(&'a T);
5 // `Ref` 包含一个指向泛型类型 `T` 的引用，`T` 具有一个 `Ref` 未知的
6 // 生命周期 `a`。`T` 受到约束，使得 `T` 中的任何*引用*必须比 `a` 存活更
7 // 此外，`Ref` 的生命周期不能超过 `a`。
8
9 // 一个使用 `Debug` trait 进行打印的泛型函数。
10 fn print<T>(t: T) where
11     T: Debug {
12     println!("`print`: t 是 {:?}", t);
13 }
14
15 // 这里取了一个 `T` 的引用，其中 `T` 实现了
16 // `Debug` 并且 `T` 中的所有*引用*都比 `a` 存活更久。
17 // 此外，`a` 必须比函数存活更久。
18 fn print_ref<'a, T>(t: &'a T) where
19     T: Debug + 'a {
20     println!("`print_ref`: t 是 {:?}", t);
21 }
22
23 fn main() {
24     let x = 7;
25     let ref_x = Ref(&x);
26
27     print_ref(&ref_x);
28     print(ref_x);
29 }
```

另请参阅：

[泛型、泛型中的约束和泛型中的多重约束](#)

强制转换

一个较长的生命周期可以被强制转换为一个较短的生命周期，使其能在通常无法工作的作用域内工作。这种转换可以通过 Rust 编译器的推断自动完成，也可以通过声明不同的生命周期的形式实现：

```
1 // 在这里，Rust 推断出一个尽可能短的生命周期。
2 // 然后两个引用被强制转换为该生命周期。
3 fn multiply<'a>(first: &'a i32, second: &'a i32) -> i32 {
4     first * second
5 }
6
7 // `<'a: 'b, 'b>` 表示生命周期 `a` 至少和 `b` 一样长。
8 // 这里，我们接收一个 `&'a i32` 并返回一个 `&'b i32` 作为强制转换的结果。
9 fn choose_first<'a: 'b, 'b>(first: &'a i32, _: &'b i32) -> &'b i32 {
10     first
11 }
12
13 fn main() {
14     let first = 2; // 较长的生命周期
15
16     {
17         let second = 3; // 较短的生命周期
18
19         println!("乘积是 {}", multiply(&first, &second));
20         println!("{}", choose_first(&first, &second));
21     };
22 }
```

静态

Rust 有几个保留的生命周期名称。其中之一是 `'static`。你可能在两种情况下遇到它：

```
// 具有 'static 生命周期的引用：  
let s: &'static str = "你好，世界";  
  
// 'static 作为 trait 约束的一部分：  
fn generic<T>(x: T) where T: 'static {}
```

这两种情况虽然相关但有微妙的区别，这也是学习 Rust 时常见的困惑来源。以下是每种情况的一些例子：

引用生命周期

作为引用生命周期，`'static` 表示该引用指向的数据在程序的整个剩余运行期间都有效。它仍然可以被强制转换为更短的生命周期。

有两种常见的方法可以创建具有 `'static` 生命周期的变量，它们都存储在二进制文件的只读内存中：

- 使用 `static` 声明创建一个常量。
- 创建一个字符串字面量，其类型为：`&'static str`。

请看下面的例子，展示了这些方法：


```

1  // 创建一个具有 `static` 生命周期的常量。
2  static NUM: i32 = 18;
3
4  // 返回一个指向 `NUM` 的引用，其中 `static` 生命周期
5  // 被强制转换为输入参数的生命周期。
6  fn coerce_static<'a>(_: &'a i32) -> &'a i32 {
7      &NUM
8  }
9
10 fn main() {
11     {
12         // 创建一个字符串字面量并打印它：
13         let static_string = "我存储在只读内存中";
14         println!("static_string 的值: {}", static_string);
15
16         // 当 `static_string` 离开作用域时，该引用
17         // 不能再被使用，但数据仍然保留在二进制文件中。
18     }
19
20     {
21         // 创建一个整数用于 `coerce_static` 函数：
22         let lifetime_num = 9;
23
24         // 将 `NUM` 的生命周期强制转换为与 `lifetime_num` 一致：
25         let coerced_static = coerce_static(&lifetime_num);
26
27         println!("coerced_static: {}", coerced_static);
28     }
29
30     println!("NUM: {} 仍然可以访问！", NUM);
31 }

```

'static 引用只需在程序生命周期的**剩余部分**有效，因此可以在程序执行过程中创建。为了演示这一点，下面的例子使用 `Box::leak` 动态创建 'static 引用。在这种情况下，它显然不会存在于整个程序生命周期，而只是从泄漏点开始存在。

```

1  extern crate rand;
2  use rand::Fill;
3
4  fn random_vec() -> &'static [u64; 100] {
5      let mut rng = rand::rng();
6      let mut boxed = Box::new([0; 100]);
7      boxed.fill(&mut rng);
8      Box::leak(boxed)
9  }
10
11 fn main() {
12     let first: &'static [u64; 100] = random_vec();
13     let second: &'static [u64; 100] = random_vec();
14     assert_ne!(first, second)
15 }

```

Trait 约束

作为 trait 约束时，它表示该类型不包含任何非静态引用。例如，接收者可以随意持有该类型，直到主动丢弃之前，它都不会变为无效。

理解这一点很重要：任何拥有所有权的数据总是满足 `'static` 生命周期约束，但对该数据的引用通常不满足：

```
1 use std::fmt::Debug;
2
3 fn print_it(input: impl Debug + 'static) {
4     println!("传入的 'static 值是: {:?}", input);
5 }
6
7 fn main() {
8     // i 拥有所有权且不包含任何引用，因此它是 'static 的：
9     let i = 5;
10    print_it(i);
11
12    // 糟糕，&i 的生命周期仅由 main() 的作用域定义，
13    // 所以它不是 'static 的：
14    print_it(&i);
15 }
```

编译器会提示你：

```
error[E0597]: `i` does not live long enough
--> src/lib.rs:15:15
   |
15 |     print_it(&i);
   |             ^^^
   |             |
   |             borrowed value does not live long enough
   |             argument requires that `i` is borrowed for `'static`
16 | }
   | - `i` dropped here while still borrowed
```

另请参阅：

['static 常量](#)

省略

有些生命周期模式非常常见，因此借用检查器允许省略它们以减少代码量并提高可读性。这被称为省略。Rust 中的省略存在的唯一原因是这些模式很常见。

以下代码展示了一些省略的例子。要更全面地了解省略，请参阅 Rust 程序设计语言中的[生命周期省略](#)章节。

```
1  // `elided_input` 和 `annotated_input` 本质上具有相同的签名
2  // 因为 `elided_input` 的生命周期是由编译器推断的：
3  fn elided_input(x: &i32) {
4      println!("`elided_input`: {}", x);
5  }
6
7  fn annotated_input<'a>(x: &'a i32) {
8      println!("`annotated_input`: {}", x);
9  }
10
11 // 同样，`elided_pass` 和 `annotated_pass` 具有相同的签名
12 // 因为生命周期被隐式地添加到 `elided_pass`：
13 fn elided_pass(x: &i32) -> &i32 { x }
14
15 fn annotated_pass<'a>(x: &'a i32) -> &'a i32 { x }
16
17 fn main() {
18     let x = 3;
19
20     elided_input(&x);
21     annotated_input(&x);
22
23     println!("`elided_pass`: {}", elided_pass(&x));
24     println!("`annotated_pass`: {}", annotated_pass(&x));
25 }
```

另请参阅：

[省略](#)

特质

`trait` 是为未知类型 `Self` 定义的一组方法集合。这些方法可以访问同一 `trait` 中声明的其他方法。

`trait` 可以为任何数据类型实现。在下面的例子中，我们定义了 `Animal`，一组方法的集合。然后为 `Sheep` 数据类型实现 `Animal` `trait`，这样就可以对 `Sheep` 使用 `Animal` 中的方法。

```

1 struct Sheep { naked: bool, name: &'static str }
2
3 trait Animal {
4     // 关联函数签名; `Self` 指代实现者类型。
5     fn new(name: &'static str) -> Self;
6
7     // 方法签名; 这些方法将返回一个字符串。
8     fn name(&self) -> &'static str;
9     fn noise(&self) -> &'static str;
10
11     // trait 可以提供默认的方法实现。
12     fn talk(&self) {
13         println!("{}", self.name(), self.noise());
14     }
15 }
16
17 impl Sheep {
18     fn is_naked(&self) -> bool {
19         self.naked
20     }
21
22     fn shear(&mut self) {
23         if self.is_naked() {
24             // 实现者的方法可以使用实现者的 trait 方法。
25             println!("{}", self.name());
26         } else {
27             println!("{}", self.name());
28
29             self.naked = true;
30         }
31     }
32 }
33
34 // 为 `Sheep` 实现 `Animal` trait
35 impl Animal for Sheep {
36     // `Self` 是实现者类型, 即 `Sheep`
37     fn new(name: &'static str) -> Sheep {
38         Sheep { name: name, naked: false }
39     }
40
41     fn name(&self) -> &'static str {
42         self.name
43     }
44
45     fn noise(&self) -> &'static str {
46         if self.is_naked() {
47             "咩~? "
48         } else {
49             "咩~! "
50         }
51     }
52
53     // 可以重写默认的 trait 方法
54     fn talk(&self) {
55         // 例如, 我们可以添加一些安静的思考
56         println!("{}", self.name, self.noise());
57     }
58 }
59
60 fn main() {

```

```
60     // 在这种情况下需要类型标注
61     // 在这种情况下需要类型标注
62     let mut dolly: Sheep = Animal::new("多莉");
63     // TODO ^ 尝试移除类型标注
64
65     dolly.talk();
66     dolly.shear();
67     dolly.talk();
68 }
```

派生

编译器可以通过 `#[derive]` 属性为某些 trait 提供基本实现。如果需要更复杂的行为，这些 trait 仍然可以手动实现。

以下是可派生的 trait 列表：

- 比较 trait： `Eq`、`PartialEq`、`Ord`、`PartialOrd`。
- `Clone`，通过复制 `&T` 创建 `T`。
- `Copy`，使类型具备“复制语义”而不是“移动语义”。
- `Hash`，从 `&T` 计算哈希值。
- `Default`，创建一个数据类型的空实例。
- `Debug`，使用 `{:?}` 格式化器来格式化一个值。

```
1  // `Centimeters`, 一个可比较的元组结构体
2  #[derive(PartialEq, PartialOrd)]
3  struct Centimeters(f64);
4
5  // `Inches`, 一个可打印的元组结构体
6  #[derive(Debug)]
7  struct Inches(i32);
8
9  impl Inches {
10     fn to_centimeters(&self) -> Centimeters {
11         let &Inches(inches) = self;
12
13         Centimeters(inches as f64 * 2.54)
14     }
15 }
16
17 // `Seconds`, 一个没有额外属性的元组结构体
18 struct Seconds(i32);
19
20 fn main() {
21     let _one_second = Seconds(1);
22
23     // 错误: `Seconds` 无法打印, 因为它没有实现 `Debug` trait
24     //println!("One second looks like: {:?}", _one_second);
25     // TODO ^ 尝试取消这行的注释
26
27     // 错误: `Seconds` 无法比较, 因为它没有实现 `PartialEq` trait
28     //let _this_is_true = (_one_second == _one_second);
29     // TODO ^ 尝试取消这行的注释
30
31     let foot = Inches(12);
32
33     println!("一英尺等于 {:?}", foot);
34
35     let meter = Centimeters(100.0);
36
37     let cmp =
38         if foot.to_centimeters() < meter {
39             "更小"
40         } else {
41             "更大"
42         };
43
44     println!("一英尺比一米{}", cmp);
45 }
```

另请参阅:

[derive](#)

使用 dyn 返回 trait

Rust 编译器需要知道每个函数的返回类型所需的内存空间。这意味着所有函数都必须返回一个具体类型。与其他语言不同，如果你有一个像 `Animal` 这样的 trait，你不能编写一个直接返回 `Animal` 的函数，因为它的不同实现可能需要不同大小的内存。

然而，有一个简单的解决方法。我们可以让函数返回一个**包含** `Animal` 的 `Box`，而不是直接返回 trait 对象。`Box` 本质上是一个指向堆内存的引用。由于引用的大小是静态已知的，且编译器可以保证它指向堆上分配的 `Animal`，这样我们就能从函数中返回一个 trait 了！

Rust 在堆上分配内存时力求明确。因此，如果你的函数以这种方式返回一个指向堆上 trait 的指针，你需要在返回类型中使用 `dyn` 关键字，例如 `Box<dyn Animal>`。

```
1 struct Sheep {}
2 struct Cow {}
3
4 trait Animal {
5     // 实例方法签名
6     fn noise(&self) -> &'static str;
7 }
8
9 // 为 `Sheep` 实现 `Animal` trait
10 impl Animal for Sheep {
11     fn noise(&self) -> &'static str {
12         "咩~！"
13     }
14 }
15
16 // 为 `Cow` 实现 `Animal` trait
17 impl Animal for Cow {
18     fn noise(&self) -> &'static str {
19         "哞~哞~哞~"
20     }
21 }
22
23 // 返回某个实现了 `Animal` 的结构体，但在编译时我们并不知道具体是哪一个
24 fn random_animal(random_number: f64) -> Box<dyn Animal> {
25     if random_number < 0.5 {
26         Box::new(Sheep {})
27     } else {
28         Box::new(Cow {})
29     }
30 }
31
32 fn main() {
33     let random_number = 0.234;
34     let animal = random_animal(random_number);
35     println!("你随机选择了一个动物，它说：{}", animal.noise());
36 }
37
```

运算符重载

在 Rust 中，许多运算符可以通过 trait 进行重载。这意味着某些运算符可以根据输入参数执行不同的任务。之所以可能，是因为运算符实际上是方法调用的语法糖。例如，`a + b` 中的 `+` 运算符会调用 `add` 方法（相当于 `a.add(b)`）。这个 `add` 方法是 `Add` trait 的一部分。因此，任何实现了 `Add` trait 的类型都可以使用 `+` 运算符。

可以在 `core::ops` 模块中找到用于重载运算符的 trait 列表，如 `Add` 等。

```
1 use std::ops;
2
3 struct Foo;
4 struct Bar;
5
6 #[derive(Debug)]
7 struct FooBar;
8
9 #[derive(Debug)]
10 struct BarFoo;
11
12 // `std::ops::Add` trait 用于指定 `+` 的功能
13 // 这里我们实现 `Add<Bar>` - 这个 trait 用于与 `Bar` 类型的右操作数相加
14 // 下面的代码块实现了操作: Foo + Bar = FooBar
15 impl ops::Add<Bar> for Foo {
16     type Output = FooBar;
17
18     fn add(self, _rhs: Bar) -> FooBar {
19         println!("> 调用了 Foo.add(Bar)");
20
21         FooBar
22     }
23 }
24
25 // 通过交换类型顺序，我们实现了非交换加法。
26 // 这里我们实现 `Add<Foo>` trait — 用于处理右操作数类型为 `Foo` 的加法。
27 // 此代码块实现了如下操作: Bar + Foo = BarFoo
28 impl ops::Add<Foo> for Bar {
29     type Output = BarFoo;
30
31     fn add(self, _rhs: Foo) -> BarFoo {
32         println!("> 调用了 Bar.add(Foo)");
33
34         BarFoo
35     }
36 }
37
38 fn main() {
39     println!("Foo + Bar = {:?}", Foo + Bar);
40     println!("Bar + Foo = {:?}", Bar + Foo);
41 }
```

参见

[Add](#), [语法索引](#)

Drop

`Drop` trait 只有一个方法：`drop`，它会在对象离开作用域时自动调用。`Drop` trait 的主要用途是释放实现该 trait 的实例所拥有的资源。

`Box`、`Vec`、`String`、`File` 和 `Process` 是一些实现了 `Drop` trait 以释放资源的类型示例。你也可以为任何自定义数据类型手动实现 `Drop` trait。

下面的例子在 `drop` 函数中添加了一个控制台打印，用于宣告它被调用的时机。

```
1 struct Droppable {
2     name: &'static str,
3 }
4
5 // 这个简单的 `drop` 实现添加了一个控制台打印
6 impl Drop for Droppable {
7     fn drop(&mut self) {
8         println!("> 正在释放 {}", self.name);
9     }
10 }
11
12 fn main() {
13     let _a = Droppable { name: "a" };
14
15     // 块 A
16     {
17         let _b = Droppable { name: "b" };
18
19         // 块 B
20         {
21             let _c = Droppable { name: "c" };
22             let _d = Droppable { name: "d" };
23
24             println!("正在退出块 B");
25         }
26         println!("刚刚退出了块 B");
27
28         println!("正在退出块 A");
29     }
30     println!("刚刚退出了块 A");
31
32     // 可以使用 `drop` 函数手动释放变量
33     drop(_a);
34     // TODO ^ 试试注释掉这一行
35
36     println!("main 函数结束");
37
38     // `_a` 在这里**不会**被再次 `drop`，因为它已经
39     // 被（手动）`drop` 过了
40 }
```

下面是一个更实用的例子，展示如何使用 `Drop` 特质在不再需要临时文件时自动清理它们：

```
1 use std::fs::File;
2 use std::path::PathBuf;
3
4 struct TempFile {
5     file: File,
6     path: PathBuf,
7 }
8
9 impl TempFile {
10     fn new(path: PathBuf) -> std::io::Result<Self> {
11         // 注意: File::create() 会覆盖现有文件
12         let file = File::create(&path)?;
13
14         Ok(Self { file, path })
15     }
16 }
17
18 // When TempFile is dropped:
19 // 1. First, our custom drop implementation runs. The file is still open
20 //    but we can remove it from the filesystem by path.
21 // 2. Then, after our drop returns, Rust automatically drops each field,
22 //    so File's drop runs and closes the file handle.
23 impl Drop for TempFile {
24     fn drop(&mut self) {
25         // Note: the File is still open here – field destructors run after
26         if let Err(e) = std::fs::remove_file(&self.path) {
27             eprintln!("删除临时文件失败: {}", e);
28         }
29         println!("> 已丢弃临时文件: {:?}", self.path);
30         // After this method returns, Rust will drop each field (including
31         // which closes the underlying file handle.
32     }
33 }
34
35 fn main() -> std::io::Result<()> {
36     // 创建新作用域来演示丢弃行为
37     {
38         let temp = TempFile::new("test.txt".into())?;
39         println!("已创建临时文件");
40         // 当 temp 超出作用域时, 文件会自动清理
41     }
42     println!("作用域结束 - 文件应该被清理");
43
44     // 如果需要, 我们也可以手动丢弃
45     let temp2 = TempFile::new("another_test.txt".into())?;
46     drop(temp2); // 显式丢弃文件
47     println!("手动丢弃的文件");
48
49     Ok(())
50 }
```

迭代器

`Iterator` trait 用于实现对数组等集合的迭代器。

该 trait 只要求为 `next` 元素定义一个方法，这个方法可以在 `impl` 块中手动定义，也可以自动定义（如在数组和区间中）。

为了在常见情况下提供便利，`for` 结构使用 `.into_iter()` 方法将某些集合转换为迭代器。

```

1 struct Fibonacci {
2     curr: u32,
3     next: u32,
4 }
5
6 // 为 `Fibonacci` 实现 `Iterator`。
7 // `Iterator` 特质只要求为 `next` 元素定义一个方法，
8 // 并需要一个 `关联类型` 来声明迭代器的返回类型。
9 impl Iterator for Fibonacci {
10     // 我们可以使用 `Self::Item` 引用这个类型
11     type Item = u32;
12
13     // 这里，我们使用 `.curr` 和 `.next` 定义序列。
14     // 返回类型是 `Option<T>`；
15     // * 当 `Iterator` 结束时，返回 `None`。
16     // * 否则，将下一个值包装在 `Some` 中并返回。
17     // 我们在返回类型中使用 `Self::Item`，这样可以更改
18     // 类型而无需更新函数签名。
19     fn next(&mut self) -> Option<Self::Item> {
20         let current = self.curr;
21
22         self.curr = self.next;
23         self.next = current + self.next;
24
25         // 由于斐波那契序列没有终点，`Iterator`
26         // 永远不会返回 `None`，始终返回 `Some`。
27         Some(current)
28     }
29 }
30
31 // 返回一个斐波那契序列生成器
32 fn fibonacci() -> Fibonacci {
33     Fibonacci { curr: 0, next: 1 }
34 }
35
36 fn main() {
37     // `0..3` 是一个生成 0、1 和 2 的 `Iterator`。
38     let mut sequence = 0..3;
39
40     println!("对 0..3 连续调用四次 `next`");
41     println!("> {:?}", sequence.next());
42     println!("> {:?}", sequence.next());
43     println!("> {:?}", sequence.next());
44     println!("> {:?}", sequence.next());
45
46     // `for` 遍历 `Iterator` 直到它返回 `None`。
47     // 每个 `Some` 值被解包并绑定到一个变量（这里是 `i`）。
48     println!("使用 `for` 遍历 0..3");
49     for i in 0..3 {
50         println!("> {}", i);
51     }
52
53     // `take(n)` 方法将 `Iterator` 限制为其前 `n` 项。
54     println!("斐波那契序列的前四项是：");
55     for i in fibonacci().take(4) {
56         println!("> {}", i);
57     }
58
59     // `skip(n)` 方法通过跳过前 `n` 项来缩短 `Iterator`。
60     println!("斐波那契序列的接下来四项是：");

```

```
60     println!(// 又做那大数列的孩子不白烦死！);
61     for i in fibonacci().skip(4).take(4) {
62         println!(> {}", i);
63     }
64
65     let array = [1u32, 3, 3, 7];
66
67     // `iter` 方法为数组/切片创建一个 `Iterator`。
68     println!("遍历以下数组 {:?}", &array);
69     for i in array.iter() {
70         println!(> {}", i);
71     }
72 }
```


impl Trait

`impl Trait` 可以在两个位置使用：

1. 作为参数类型
2. 作为返回类型

作为参数类型

如果你的函数对某个 trait 是泛型的，但不关心具体类型，你可以使用 `impl Trait` 作为参数类型来简化函数声明。

例如，考虑以下代码：

```
1 fn parse_csv_document<R: std::io::BufRead>(src: R) -> std::io::Result<Vec<
2     src.lines()
3     .map(|line| {
4         // 遍历数据源中的每一行
5         line.map(|line| {
6             // 如果成功读取行，则处理它；否则，返回错误
7             line.split(',') // 按逗号分割行
8                 .map(|entry| String::from(entry.trim())) // 去除首尾空
9                 .collect() // 将该行的所有字符串收集到 Vec<String> 中
10        })
11    })
12    .collect() // 将所有行收集到 Vec<Vec<String>> 中
13 }
```

`parse_csv_document` 是泛型函数，可以接受任何实现了 `BufRead` 的类型，如 `BufReader<File>` 或 `[u8]`。但具体的 `R` 类型并不重要，`R` 仅用于声明 `src` 的类型。因此，这个函数也可以写成：

```
1 fn parse_csv_document(src: impl std::io::BufRead) -> std::io::Result<Vec<
2     src.lines()
3     .map(|line| {
4         // 遍历数据源中的每一行
5         line.map(|line| {
6             // 如果成功读取行，则处理它；否则，返回错误
7             line.split(',') // 按逗号分割行
8                 .map(|entry| String::from(entry.trim())) // 去除首尾空
9                 .collect() // 将该行的所有字符串收集到 Vec<String> 中
10        })
11    })
12    .collect() // 将所有行收集到 Vec<Vec<String>> 中
13 }
```

注意，使用 `impl Trait` 作为参数类型意味着你无法显式指定使用的函数形式。例如，`parse_csv_document::<std::io::Empty>(std::io::empty())` 在第二个例子中将无法工

作。

作为返回类型

如果函数返回一个实现了 `MyTrait` 的类型，你可以将其返回类型写为 `-> impl MyTrait`。这可以大大简化类型签名！

```

1 use std::iter;
2 use std::vec::IntoIter;
3
4 // 这个函数合并两个 `Vec<i32>` 并返回一个迭代器。
5 // 看看它的返回类型有多复杂！
6 fn combine_vecs_explicit_return_type(
7     v: Vec<i32>,
8     u: Vec<i32>,
9 ) -> iter::Cycle<iter::Chain<IntoIter<i32>, IntoIter<i32>>> {
10     v.into_iter().chain(u.into_iter()).cycle()
11 }
12
13 // 这是完全相同的函数，但它的返回类型使用了 `impl Trait`。
14 // 看看它变得多么简单！
15 fn combine_vecs(
16     v: Vec<i32>,
17     u: Vec<i32>,
18 ) -> impl Iterator<Item=i32> {
19     v.into_iter().chain(u.into_iter()).cycle()
20 }
21
22 fn main() {
23     let v1 = vec![1, 2, 3];
24     let v2 = vec![4, 5];
25     let mut v3 = combine_vecs(v1, v2);
26     assert_eq!(Some(1), v3.next());
27     assert_eq!(Some(2), v3.next());
28     assert_eq!(Some(3), v3.next());
29     assert_eq!(Some(4), v3.next());
30     assert_eq!(Some(5), v3.next());
31     println!("全部完成");
32 }

```

更重要的是，某些 Rust 类型无法直接写出。例如，每个闭包都有自己的未命名具体类型。在 `impl Trait` 语法出现之前，你必须在堆上分配内存才能返回闭包。但现在你可以完全静态地做到这一点，像这样：

```

1 // 返回一个将 `y` 加到输入值上的函数
2 fn make_adder_function(y: i32) -> impl Fn(i32) -> i32 {
3     let closure = move |x: i32| { x + y };
4     closure
5 }
6
7 fn main() {
8     let plus_one = make_adder_function(1);
9     assert_eq!(plus_one(2), 3);
10 }

```

你还可以使用 `impl Trait` 返回一个使用 `map` 或 `filter` 闭包的迭代器！这使得使用 `map` 和 `filter` 更加容易。由于闭包类型没有名称，如果你的函数返回带有闭包的迭代器，你无法写出显式的返回类型。但使用 `impl Trait`，你可以轻松实现：

```
1 fn double_positives<'a>(numbers: &'a Vec<i32>) -> impl Iterator<Item = i32> {
2     numbers
3     .iter()
4     .filter(|x| x > &&0)
5     .map(|x| x * 2)
6 }
7
8 fn main() {
9     let singles = vec![-3, -2, 2, 3];
10    let doubles = double_positives(&singles);
11    assert_eq!(doubles.collect::<Vec<i32>>(), vec![4, 6]);
12 }
```

Clone and Copy

在处理资源时，默认行为是在赋值或函数调用期间转移它们。然而，有时我们也需要复制资源。

`Clone` trait 帮助我们实现这一点。最常见的是，我们可以使用 `Clone` trait 定义的 `.clone()` 方法。

Copy: Implicit Cloning

The `Copy` trait allows a type to be duplicated simply by copying bits, with no additional logic required. When a type implements `Copy`, assignments and function calls will implicitly copy the value instead of moving it.

Important: `Copy` requires `Clone` - any type that implements `Copy` must also implement `Clone`. This is because `Copy` is defined as a subtrait: `trait Copy: Clone {}`. The `Clone` implementation for `Copy` types simply copies the bits.

Not all types can implement `Copy`. A type can only be `Copy` if:

- All of its components are `Copy`
- It doesn't manage external resources (like heap memory, file handles, etc.)

```
1  // A unit struct without resources
2  // Note: Copy requires Clone, so we must derive both
3  #[derive(Debug, Clone, Copy)]
4  struct Unit;
5
6  // A tuple struct with resources that implements the `Clone` trait
7  // This CANNOT be Copy because Box<T> is not Copy
8  #[derive(Clone, Debug)]
9  struct Pair(Box<i32>, Box<i32>);
10
11 fn main() {
12     // 实例化 `Unit`
13     let unit = Unit;
14     // Copy `Unit` - this is an implicit copy, not a move!
15     // Because Unit implements Copy, the value is duplicated automatically
16     let copied_unit = unit;
17
18     // 两个 `Unit` 可以独立使用
19     println!("原始: {:?}", unit);
20     println!("复制: {:?}", copied_unit);
21
22     // 实例化 `Pair`
23     let pair = Pair(Box::new(1), Box::new(2));
24     println!("原始: {:?}", pair);
25
26     // Move `pair` into `moved_pair`, moves resources
27     // Pair does not implement Copy, so this is a move
28     let moved_pair = pair;
29     println!("已移动: {:?}", moved_pair);
30
31     // 错误! `pair` 已失去其资源
32     //println!("原始: {:?}", pair);
33     // TODO ^ 尝试取消注释此行
34
35     // Clone `moved_pair` into `cloned_pair` (resources are included)
36     // Unlike Copy, Clone is explicit - we must call .clone()
37     let cloned_pair = moved_pair.clone();
38     // 使用 std::mem::drop 丢弃已移动的原始对
39     drop(moved_pair);
40
41     // 错误! `moved_pair` 已被丢弃
42     //println!("已移动并丢弃: {:?}", moved_pair);
43     // TODO ^ 尝试取消注释此行
44
45     // .clone() 的结果仍然可以使用!
46     println!("克隆: {:?}", cloned_pair);
47 }
```

父特质

Rust 没有“继承”，但你可以将一个 trait 定义为另一个 trait 的超集。例如：

```
1 trait Person {
2     fn name(&self) -> String;
3 }
4
5 // Person 是 Student 的超 trait。
6 // 实现 Student 需要你同时实现 Person。
7 trait Student: Person {
8     fn university(&self) -> String;
9 }
10
11 trait Programmer {
12     fn fav_language(&self) -> String;
13 }
14
15 // CompSciStudent (计算机科学学生) 是 Programmer 和 Student 的子 trait。
16 // 实现 CompSciStudent 需要你实现两个超 trait。
17 trait CompSciStudent: Programmer + Student {
18     fn git_username(&self) -> String;
19 }
20
21 fn comp_sci_student_greeting(student: &dyn CompSciStudent) -> String {
22     format!(
23         "我的名字是 {}, 我就读于 {}。我最喜欢的语言是 {}。我的 Git 用户名: ",
24         student.name(),
25         student.university(),
26         student.fav_language(),
27         student.git_username()
28     )
29 }
30
31 struct CSStudent {
32     name: String,
33     university: String,
34     fav_language: String,
35     git_username: String
36 }
37
38 impl Programmer for CSStudent {
39     fn fav_language(&self) -> String {
40         self.fav_language.clone()
41     }
42 }
43
44 impl Student for CSStudent {
45     fn university(&self) -> String {
46         self.university.clone()
47     }
48 }
49
50 impl Person for CSStudent {
51     fn name(&self) -> String {
52         self.name.clone()
53     }
54 }
55
56 impl CompSciStudent for CSStudent {
57     fn git_username(&self) -> String {
58         self.git_username.clone()
59     }
60 }
```

```
61
62 fn main() {
63     let student = CSStudent {
64         name: String::from("Alice"),
65         university: String::from("MIT"),
66         fav_language: String::from("Rust"),
67         git_username: String::from("alice_codes"),
68     };
69
70     let greeting = comp_sci_student_greeting(&student);
71     println!("{}", greeting);
72 }
```

另请参阅：

[《Rust 程序设计语言》中关于超 trait 的章节](#)

消除重叠特质的歧义

一个类型可以实现多个不同的 trait。如果两个 trait 都要求函数使用相同的名称，该怎么办？例如，许多 trait 可能都有一个名为 `get()` 的方法，它们甚至可能有不同的返回类型！

好消息是：由于每个 trait 实现都有自己的 `impl` 块，因此很容易分清楚你正在实现哪个 trait 的 `get` 方法。

那么在调用这些方法时又该如何处理呢？为了消除它们之间的歧义，我们必须使用完全限定语法。

```
1 trait UsernameWidget {
2     // 从这个小程序件中获取选定的用户名
3     fn get(&self) -> String;
4 }
5
6 trait AgeWidget {
7     // 从这个小程序件中获取选定的年龄
8     fn get(&self) -> u8;
9 }
10
11 // 一个同时包含 UsernameWidget 和 AgeWidget 的表单
12 struct Form {
13     username: String,
14     age: u8,
15 }
16
17 impl UsernameWidget for Form {
18     fn get(&self) -> String {
19         self.username.clone()
20     }
21 }
22
23 impl AgeWidget for Form {
24     fn get(&self) -> u8 {
25         self.age
26     }
27 }
28
29 fn main() {
30     let form = Form {
31         username: "rustacean".to_owned(),
32         age: 28,
33     };
34
35     // 如果你取消注释这一行，你会得到一个错误，提示
36     // "找到多个 `get`"。这是因为确实存在多个
37     // 名为 `get` 的方法。
38     // println!("{}", form.get());
39
40     let username = <Form as UsernameWidget>::get(&form);
41     assert_eq!("rustacean".to_owned(), username);
42     let age = <Form as AgeWidget>::get(&form);
43     assert_eq!(28, age);
44 }
```

另请参阅：

《Rust 程序设计语言》中关于完全限定语法的章节

macro_rules!

Rust 提供了一个强大的宏系统，支持元编程。正如你在前面章节中所看到的，宏看起来像函数，只是它们的名字以感叹号 `!` 结尾。但与生成函数调用不同，宏会展开成源代码，然后与程序的其余部分一起编译。与 C 和其他语言中的宏不同，Rust 宏展开成抽象语法树，而不是进行字符串预处理，因此你不会遇到意外的优先级错误。

Macros can be created using the `macro_rules!` macro.

```
1 // 这是一个名为 `say_hello` 的简单宏。
2 macro_rules! say_hello {
3     // `()` 表示该宏不接受任何参数。
4     () => {
5         // 宏将展开成这个代码块的内容。
6         println!("Hello!")
7     };
8 }
9
10 fn main() {
11     // 这个调用将展开成 `println!("Hello!")`
12     say_hello!()
13 }
```

那么，为什么宏是有用的呢？

1. 不要重复自己。在许多情况下，你可能需要在多个地方使用相似的功能，但类型不同。通常，编写宏是避免代码重复的有效方法。（稍后会详细介绍）
2. 领域特定语言。宏允许你为特定目的定义专门的语法。（稍后会详细介绍）
3. 可变参数接口。有时你可能想定义一个接受可变数量参数的接口。例如 `println!`，它可以根据格式字符串接受任意数量的参数。（稍后会详细介绍）

语法

在接下来的小节中，我们将展示如何在 Rust 中定义宏。有三个基本概念：

- [模式和指示符](#)
- [重载](#)
- [重复](#)

指示符

宏的参数以美元符号 `$` 为前缀，并用**指示符**来标注类型：

```
1 macro_rules! create_function {
2     // 这个宏接受一个 `ident` 指示符的参数，
3     // 并创建一个名为 `$func_name` 的函数。
4     // `ident` 指示符用于变量/函数名。
5     ($func_name:ident) => {
6         fn $func_name() {
7             // `stringify!` 宏将 `ident` 转换为字符串。
8             println!("你调用了 {:?}()",
9                     stringify!($func_name));
10        }
11    };
12 }
13
14 // 使用上面的宏创建名为 `foo` 和 `bar` 的函数。
15 create_function!(foo);
16 create_function!(bar);
17
18 macro_rules! print_result {
19     // 这个宏接受一个 `expr` 类型的表达式，
20     // 并将其作为字符串打印出来，同时打印其结果。
21     // `expr` 指示符用于表达式。
22     ($expression:expr) => {
23         // `stringify!` 将表达式**原样**转换为字符串。
24         println!("{:?} = {:?}",
25                 stringify!($expression),
26                 $expression);
27     };
28 }
29
30 fn main() {
31     foo();
32     bar();
33
34     print_result!(1u32 + 1);
35
36     // 记住，代码块也是表达式！
37     print_result!({
38         let x = 1u32;
39
40         x * x + 2 * x - 1
41     });
42 }
```

以下是一些可用的指示符：

- `block`
- `expr` 用于表达式
- `ident` 用于变量/函数名
- `item`
- `literal` 用于字面常量

- `pat` (模式 *pattern*)
- `path`
- `stmt` (语句 *statement*)
- `tt` (标记树 *token tree*)
- `ty` (类型 *type*)
- `vis` (可见性限定符 *visibility qualifier*)

完整列表请参阅 [Rust 参考手册](#)。

重载

宏可以被重载以接受不同的参数组合。在这方面，`macro_rules!` 的工作方式类似于 `match` 块：

```
1  // `test!` 将以不同的方式比较 `$left` 和 `$right`
2  // 具体取决于你如何调用它：
3  macro_rules! test {
4      // 参数不需要用逗号分隔。
5      // 可以使用任何模板！
6      ($left:expr; and $right:expr) => {
7          println!("{:?} 和 {:?} 是 {:?}",
8                  stringify!($left),
9                  stringify!($right),
10                 $left && $right)
11      };
12      // ^ 每个分支必须以分号结束。
13      ($left:expr; or $right:expr) => {
14          println!("{:?} 或 {:?} 是 {:?}",
15                  stringify!($left),
16                  stringify!($right),
17                 $left || $right)
18      };
19  }
20
21 fn main() {
22     test!(1i32 + 1 == 2i32; and 2i32 * 2 == 4i32);
23     test!(true; or false);
24 }
```

重复

宏可以在参数列表中使用 `+` 来表示一个参数可能重复至少一次，或使用 `*` 来表示一个参数可能重复零次或多次。

在下面的例子中，用 `$(...),+` 包围匹配器将匹配一个或多个由逗号分隔的表达式。另外请注意，最后一个情况的分号是可选的。

```
1  // `find_min!` 将计算任意数量参数中的最小值。
2  macro_rules! find_min {
3      // 基本情况:
4      ($x:expr) => ($x);
5      // `$x` 后面至少跟着一个 `$y`,`
6      ($x:expr, $($y:expr),+) => (
7          // 对剩余的 `$y` 递归调用 `find_min!`
8          std::cmp::min($x, find_min!($($y),+))
9      )
10 }
11
12 fn main() {
13     println!("{}", find_min!(1));
14     println!("{}", find_min!(1 + 2, 2));
15     println!("{}", find_min!(5, 2 * 3, 4));
16 }
```


DRY(不要重复自己)

宏通过提取函数和/或测试套件的共同部分，使我们能够编写符合 DRY（Don't Repeat Yourself）原则的代码。下面是一个在 `Vec<T>` 上实现并测试 `+=`、`*=` 和 `-=` 运算符的示例：

```

1 use std::ops::{Add, Mul, Sub};
2
3 macro_rules! assert_equal_len {
4     // `tt` (token tree, 标记树) 指示符用于
5     // 运算符和标记。
6     ($a:expr, $b:expr, $func:ident, $op:tt) => {
7         assert!($a.len() == $b.len(),
8             "{:?}: 维度不匹配: {:?} {:?} {:?}",
9             stringify!($func),
10             ($a.len(),),
11             stringify!($op),
12             ($b.len(),));
13     };
14 }
15
16 macro_rules! op {
17     ($func:ident, $bound:ident, $op:tt, $method:ident) => {
18         fn $func<T: $bound<T, Output=T> + Copy>(xs: &mut Vec<T>, ys: &Vec<T>) {
19             assert_equal_len!(xs, ys, $func, $op);
20
21             for (x, y) in xs.iter_mut().zip(ys.iter()) {
22                 *x = $bound::$method(*x, *y);
23                 // *x = x.$method(*y);
24             }
25         }
26     };
27 }
28
29 // 实现 `add_assign`、`mul_assign` 和 `sub_assign` 函数。
30 op!(add_assign, Add, +=, add);
31 op!(mul_assign, Mul, *=, mul);
32 op!(sub_assign, Sub, -=, sub);
33
34 mod test {
35     use std::iter;
36     macro_rules! test {
37         ($func:ident, $x:expr, $y:expr, $z:expr) => {
38             #[test]
39             fn $func() {
40                 for size in 0usize..10 {
41                     let mut x: Vec<_> = iter::repeat($x).take(size).collect();
42                     let y: Vec<_> = iter::repeat($y).take(size).collect();
43                     let z: Vec<_> = iter::repeat($z).take(size).collect();
44
45                     super::$func(&mut x, &y);
46
47                     assert_eq!(x, z);
48                 }
49             }
50         };
51     }
52
53     // 测试 `add_assign`、`mul_assign` 和 `sub_assign`。
54     test!(add_assign, 1u32, 2u32, 3u32);
55     test!(mul_assign, 2u32, 3u32, 6u32);
56     test!(sub_assign, 3u32, 2u32, 1u32);
57 }

```

```
$ rustc --test dry.rs && ./dry
running 3 tests
test test::mul_assign ... ok
test test::add_assign ... ok
test test::sub_assign ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured
```

领域特定语言 (DSL)

DSL 是嵌入在 Rust 宏中的一种微型“语言”。它是完全有效的 Rust 代码，因为宏系统会将其展开为普通的 Rust 结构，但它看起来像一种小型语言。这使你能够为某些特定功能定义简洁或直观的语法（在一定范围内）。

假设我想定义一个简单的计算器 API。我希望提供一个表达式，并将计算结果打印到控制台。

```
1 macro_rules! calculate {
2     (eval $e:expr) => {
3         {
4             let val: usize = $e; // 强制类型为无符号整数
5             println!("{}", stringify!{$e}, val);
6         }
7     };
8 }
9
10 fn main() {
11     calculate! {
12         eval 1 + 2 // 嘿嘿，`eval` 可不是 Rust 的关键字哦！
13     }
14
15     calculate! {
16         eval (1 + 2) * (3 / 4)
17     }
18 }
```

输出：

```
1 + 2 = 3
(1 + 2) * (3 / 4) = 0
```

这个例子非常简单，但已经有很多利用宏开发的复杂接口，比如 `lazy_static` 或 `clap`。

另外，注意宏中的两对大括号。外层的大括号是 `macro_rules!` 语法的一部分，除此之外还可以使用 `()` 或 `[]`。

可变参数接口

可变参数接口可以接受任意数量的参数。例如，`println!` 可以接受任意数量的参数，这由格式字符串决定。

我们可以将上一节的 `calculate!` 宏扩展为可变参数的形式：

```
1 macro_rules! calculate {
2     // 单个 `eval` 的模式
3     (eval $e:expr) => {
4         {
5             let val: usize = $e; // 强制类型为整数
6             println!("{}", stringify!{$e}, val);
7         }
8     };
9
10    // 递归分解多个 `eval`
11    (eval $e:expr, $(eval $es:expr),+) => {{
12        calculate! { eval $e }
13        calculate! { $(eval $es),+ }
14    }};
15 }
16
17 fn main() {
18     calculate! { // 瞧瞧！这是可变参数的 `calculate!`！
19         eval 1 + 2,
20         eval 3 + 4,
21         eval (2 * 3) + 1
22     }
23 }
```

输出：

```
1 + 2 = 3
3 + 4 = 7
(2 * 3) + 1 = 7
```

错误处理

错误处理是处理可能出现失败情况的过程。例如，读取文件失败后继续使用那个**错误的**输入显然会导致问题。注意并明确管理这些错误可以使程序的其他部分避免各种陷阱。

Rust 中有多种处理错误的方法，这些方法将在接下来的小节中详细介绍。它们或多或少都有一些细微的差别和不同的使用场景。一般来说：

显式的 `panic` 主要用于测试和处理不可恢复的错误。在原型开发中，它可能会有用，例如在处理尚未实现的函数时。但在这些情况下，使用更具描述性的 `unimplemented` 会更好。在测试中，`panic` 是一种合理的显式失败方式。

`Option` 类型用于值是可选的情况，或者缺少值不构成错误条件的情况。例如目录的父目录 - / 和 C: 就没有父目录。在处理 `Option` 时，对于原型设计和绝对确定有值的情况，使用 `unwrap` 是可以的。然而，`expect` 更有用，因为它允许你指定一个错误消息，以防万一出错。

当有可能出错且调用者必须处理问题时，请使用 `Result`。你也可以对它们使用 `unwrap` 和 `expect`（除非是测试或快速原型，否则请不要这样做）。

关于错误处理更详尽的讨论，请参阅[官方文档](#)中的错误处理章节。

panic

我们将看到的最简单的错误处理机制是 `panic`。它会打印一条错误消息，开始展开栈，并通常会退出程序。在这里，我们在错误条件下显式调用 `panic`：

```
1 fn drink(beverage: &str) {  
2     // 你不应该喝太多含糖饮料。  
3     if beverage == "柠檬水" { panic!("啊啊啊啊啊！！！！"); }  
4  
5     println!("来点清爽的{}就是我现在需要的。", beverage);  
6 }  
7  
8 fn main() {  
9     drink("水");  
10    drink("柠檬水");  
11    drink("纯净水");  
12 }
```

第一次调用 `drink` 正常执行。第二次调用会引发 `panic`，因此第三次调用永远不会被执行。

abort 和 unwind

上一节介绍了错误处理机制 `panic`。可以根据 `panic` 设置有条件地编译不同的代码路径。当前可用的值有 `unwind` 和 `abort`。

基于之前的柠檬水示例，我们明确使用 `panic` 策略来执行不同的代码行。

```
1 fn drink(beverage: &str) {
2     // 你不应该喝太多含糖饮料。
3     if beverage == "柠檬水" {
4         if cfg!(panic = "中止") {
5             println!("这不是你的派对。快跑！！！！");
6         } else {
7             println!("快吐出来！！！！");
8         }
9     } else {
10        println!("来点清爽的{}就是我现在需要的。", beverage);
11    }
12 }
13
14 fn main() {
15     drink("水");
16     drink("柠檬水");
17 }
```

这里是另一个示例，重点是重写 `drink()` 函数并明确使用 `unwind` 关键字。

```
1 #[cfg(panic = "unwind")]
2 fn ah() {
3     println!("快吐出来！！！！");
4 }
5
6 #[cfg(not(panic = "unwind"))]
7 fn ah() {
8     println!("这不是你的派对。快跑！！！！");
9 }
10
11 fn drink(beverage: &str) {
12     if beverage == "柠檬水" {
13         ah();
14     } else {
15         println!("来点清爽的{}就是我现在需要的。", beverage);
16     }
17 }
18
19 fn main() {
20     drink("水");
21     drink("柠檬水");
22 }
```

可以通过命令行使用 `abort` 或 `unwind` 来设置 `panic` 策略。

```
rustc lemonade.rs -C panic=abort
```


Option 和 unwrap

在上一个例子中，我们展示了如何主动引发程序失败。我们让程序在喝含糖柠檬水时触发 `panic`。但如果我们期望得到**某种**饮料却没有收到呢？这种情况同样糟糕，所以需要处理！

我们**可以**像处理柠檬水那样对空字符串（`""`）进行测试。但既然我们使用的是 Rust，不如让编译器指出没有饮料的情况。

`std` 库中的 `Option<T>` 枚举用于处理可能存在缺失的情况。它表现为两个“选项”之一：

- `Some(T)`：找到了一个 `T` 类型的元素
- `None`：没有找到元素

这些情况可以通过 `match` 显式处理，也可以用 `unwrap` 隐式处理。隐式处理要么返回内部元素，要么触发 `panic`。

注意，可以使用 `expect` 手动自定义 `panic`，但 `unwrap` 相比显式处理会产生一个不太有意义的输出。在下面的例子中，显式处理产生了一个更可控的结果，同时保留了在需要时触发 `panic` 的选项。

```
1  // 成年人见多识广，可以很好地处理任何饮料。
2  // 所有饮料都使用 `match` 显式处理。
3  fn give_adult(drink: Option<&str>) {
4      // 为每种情况指定一个处理方案。
5      match drink {
6          Some("柠檬水") => println!("呸！太甜了。"),
7          Some(inner)    => println!("{}", ? 真不错。", inner),
8          None           => println!("没有饮料？好吧。"),
9      }
10 }
11
12 // 其他人在喝含糖饮料前会触发 `panic`。
13 // 所有饮料都使用 `unwrap` 隐式处理。
14 fn drink(drink: Option<&str>) {
15     // 当 `unwrap` 收到 `None` 时会触发 `panic`。
16     let inside = drink.unwrap();
17     if inside == "柠檬水" { panic!("啊啊啊啊啊！！！！"); }
18
19     println!("我超爱{}！！！！", inside);
20 }
21
22 fn main() {
23     let water   = Some("水");
24     let lemonade = Some("柠檬水");
25     let void    = None;
26
27     give_adult(water);
28     give_adult(lemonade);
29     give_adult(void);
30
31     let coffee = Some("咖啡");
32     let nothing = None;
33
34     drink(coffee);
35     drink(nothing);
36 }
```

使用 ? 解包 Option

你可以使用 `match` 语句来解包 `Option`，但使用 `?` 运算符通常更简便。如果 `x` 是一个 `Option`，那么求值 `x?` 将在 `x` 是 `Some` 时返回其内部值，否则它将终止当前执行的函数并返回 `None`。

```
fn next_birthday(current_age: Option<u8>) -> Option<String> {  
    // 如果 `current_age` 是 `None`，这将返回 `None`。  
    // 如果 `current_age` 是 `Some`，内部的 `u8` 值加 1  
    // 后被赋值给 `next_age`  
    let next_age: u8 = current_age? + 1;  
    Some(format!("明年我将会 {} 岁", next_age))  
}
```

你可以将多个 `?` 链接在一起，使你的代码更易读。

```
1 struct Person {
2     job: Option<Job>,
3 }
4
5 #[derive(Clone, Copy)]
6 struct Job {
7     phone_number: Option<PhoneNumber>,
8 }
9
10 #[derive(Clone, Copy)]
11 #[allow(dead_code)]
12 struct PhoneNumber {
13     area_code: Option<u8>,
14     number: u32,
15 }
16
17 impl Person {
18
19     // 如果存在，获取此人工作电话号码的区号。
20     fn work_phone_area_code(&self) -> Option<u8> {
21         // 如果没有 `?` 运算符，这将需要许多嵌套的 `match` 语句。
22         // 这将需要更多的代码 - 试着自己写一下，看看哪种方式
23         // 更容易。
24         self.job?.phone_number?.area_code
25     }
26 }
27
28 fn main() {
29     let p = Person {
30         job: Some(Job {
31             phone_number: Some(PhoneNumber {
32                 area_code: Some(61),
33                 number: 439222222,
34             }),
35         }),
36     };
37
38     assert_eq!(p.work_phone_area_code(), Some(61));
39 }
```

组合器：map

`match` 是处理 `Option` 的有效方法。然而，频繁使用可能会让人感到繁琐，尤其是在只有输入时才有效的操作中。在这些情况下，可以使用[组合器](#)以模块化的方式管理控制流。

`Option` 有一个内置方法 `map()`，这是一个用于简单映射 `Some -> Some` 和 `None -> None` 的组合器。多个 `map()` 调用可以链式使用，从而提供更大的灵活性。

在下面的例子中，`process()` 函数以简洁的方式替代了之前的所有函数。

```

1  #![allow(dead_code)]
2
3  #[derive(Debug)] enum Food { Apple, Carrot, Potato }
4
5  #[derive(Debug)] struct Peeled(Food);
6  #[derive(Debug)] struct Chopped(Food);
7  #[derive(Debug)] struct Cooked(Food);
8
9  // 剥皮食物。如果没有食物，则返回 `None`。
10 // 否则，返回剥皮后的食物。
11 fn peel(food: Option<Food>) -> Option<Peeled> {
12     match food {
13         Some(food) => Some(Peeled(food)),
14         None      => None,
15     }
16 }
17
18 // 切碎食物。如果没有食物，则返回 `None`。
19 // 否则，返回切碎后的食物。
20 fn chop(peeled: Option<Peeled>) -> Option<Chopped> {
21     match peeled {
22         Some(Peeled(food)) => Some(Chopped(food)),
23         None              => None,
24     }
25 }
26
27 // 烹饪食物。这里我们展示了使用 `map()` 而非 `match` 来处理不同情况。
28 fn cook(chopped: Option<Chopped>) -> Option<Cooked> {
29     chopped.map(|Chopped(food)| Cooked(food))
30 }
31
32 // 一个按顺序剥皮、切碎和烹饪食物的函数。
33 // 我们通过链式调用多个 `map()` 来简化代码。
34 fn process(food: Option<Food>) -> Option<Cooked> {
35     food.map(|f| Peeled(f))
36         .map(|Peeled(f)| Chopped(f))
37         .map(|Chopped(f)| Cooked(f))
38 }
39
40 // 在尝试吃之前，先检查是否有食物！
41 fn eat(food: Option<Cooked>) {
42     match food {
43         Some(food) => println!("嗯，我喜欢 {:?}" , food),
44         None      => println!("哎呀！这不能吃。"),
45     }
46 }
47
48 fn main() {
49     let apple = Some(Food::Apple);
50     let carrot = Some(Food::Carrot);
51     let potato = None;
52
53     let cooked_apple = cook(chop(peel(apple)));
54     let cooked_carrot = cook(chop(peel(carrot)));
55     // 现在让我们试试看起来更简洁的 `process()` 函数。
56     let cooked_potato = process(potato);
57
58     eat(cooked_apple);
59     eat(cooked_carrot);
60     eat(cooked_potato);

```

```
    eat(cooked_potato),  
61 }
```

另请参阅：

[闭包](#)、[Option](#)、[Option::map\(\)](#)

组合器：and_then

`map()` 被描述为一种可链式调用的方式来简化 `match` 语句。然而，在返回 `Option<T>` 的函数上使用 `map()` 会导致嵌套的 `Option<Option<T>>`。链式调用多个这样的函数可能会变得令人困惑。这时，另一个称为 `and_then()` 的组合器（在某些语言中称为 `flatMap`）就派上用场了。

`and_then()` 使用包装的值调用其函数输入并返回结果。如果 `Option` 是 `None`，则直接返回 `None`。

在下面的例子中，`cookable_v3()` 返回一个 `Option<Food>`。如果使用 `map()` 而不是 `and_then()`，将会得到一个 `Option<Option<Food>>`，这对于 `eat()` 函数来说是一个无效的类型。


```
1  #![allow(dead_code)]
2
3  #[derive(Debug)] enum Food { CordonBleu, Steak, Sushi }
4  #[derive(Debug)] enum Day { Monday, Tuesday, Wednesday }
5
6  // 我们没有制作寿司的原料。
7  fn have_ingredients(food: Food) -> Option<Food> {
8      match food {
9          Food::Sushi => None,
10         _          => Some(food),
11     }
12 }
13
14 // 我们有除了蓝带猪排以外所有菜品的食谱。
15 fn have_recipe(food: Food) -> Option<Food> {
16     match food {
17         Food::CordonBleu => None,
18         _                 => Some(food),
19     }
20 }
21
22 // 要做一道菜，我们需要食谱和原料。
23 // 我们可以用一系列的 `match` 来表示这个逻辑：
24 fn cookable_v1(food: Food) -> Option<Food> {
25     match have_recipe(food) {
26         None          => None,
27         Some(food)    => have_ingredients(food),
28     }
29 }
30
31 // 这可以使用 `and_then()` 更简洁地重写：
32 fn cookable_v3(food: Food) -> Option<Food> {
33     have_recipe(food).and_then(have_ingredients)
34 }
35
36 // 否则我们需要对 `Option<Option<Food>>` 使用 `flatten()`
37 // 来获得 `Option<Food>`：
38 fn cookable_v2(food: Food) -> Option<Food> {
39     have_recipe(food).map(have_ingredients).flatten()
40 }
41
42 fn eat(food: Food, day: Day) {
43     match cookable_v3(food) {
44         Some(food) => println!("太好了！在 {:?} 我们可以吃到 {:?}。", day, food),
45         None      => println!("哦不。我们在 {:?} 没有东西吃吗？", day),
46     }
47 }
48
49 fn main() {
50     let (cordon_bleu, steak, sushi) = (Food::CordonBleu, Food::Steak, Food::Sushi);
51
52     eat(cordon_bleu, Day::Monday);
53     eat(steak, Day::Tuesday);
54     eat(sushi, Day::Wednesday);
55 }
```

另请参阅：

闭包、`Option`、`Option::and_then()` 和 `Option::flatten()`

解包 Option 和设置默认值

有多种方法可以解包 Option 并在其为 None 时使用默认值。为了选择满足我们需求的方法，我们需要考虑以下几点：

- 我们需要立即求值还是惰性求值？
- 我们需要保持原始的空值不变，还是就地修改它？

or() 可链式调用，立即求值，保持空值不变

or() 可以链式调用，并且会立即求值其参数，如下例所示。注意，由于 or 的参数是立即求值的，传递给 or 的变量会被移动。

```
1  #[derive(Debug)]
2  enum Fruit { Apple, Orange, Banana, Kiwi, Lemon }
3
4  fn main() {
5      let apple = Some(Fruit::Apple);
6      let orange = Some(Fruit::Orange);
7      let no_fruit: Option<Fruit> = None;
8
9      let first_available_fruit = no_fruit.or(orange).or(apple);
10     println!("第一个可用的水果: {:?}", first_available_fruit);
11     // first_available_fruit: Some(Orange)
12
13     // `or` 会移动其参数。
14     // 在上面的例子中，`or(orange)` 返回了 `Some`，所以 `or(apple)` 没有被
15     // 但是名为 `apple` 的变量无论如何都被移动了，不能再使用。
16     // println!("变量 apple 被移动了，所以这行不会编译: {:?}", apple);
17     // TODO: 取消上面这行的注释来查看编译器错误
18 }
```

or_else() 可以链式调用，惰性求值，保持空值不变

另一种选择是使用 or_else，它同样支持链式调用，并且采用惰性求值。以下是一个示例：

```

1  #[derive(Debug)]
2  enum Fruit { Apple, Orange, Banana, Kiwi, Lemon }
3
4  fn main() {
5      let no_fruit: Option<Fruit> = None;
6      let get_kiwi_as_fallback = || {
7          println!("提供猕猴桃作为备选");
8          Some(Fruit::Kiwi)
9      };
10     let get_lemon_as_fallback = || {
11         println!("提供柠檬作为备选");
12         Some(Fruit::Lemon)
13     };
14
15     let first_available_fruit = no_fruit
16         .or_else(get_kiwi_as_fallback)
17         .or_else(get_lemon_as_fallback);
18     println!("第一个可用的水果: {:?}", first_available_fruit);
19     // 提供猕猴桃作为备选
20     // first_available_fruit: Some(Kiwi)
21 }

```

get_or_insert() 立即求值，原地修改空值

为确保 `Option` 包含一个值，我们可以使用 `get_or_insert` 来原地修改它，提供一个备选值。下面的例子展示了这一点。请注意，`get_or_insert` 会立即求值其参数，因此变量 `apple` 会被移动：

```

1  #[derive(Debug)]
2  enum Fruit { Apple, Orange, Banana, Kiwi, Lemon }
3
4  fn main() {
5      let mut my_fruit: Option<Fruit> = None;
6      let apple = Fruit::Apple;
7      let first_available_fruit = my_fruit.get_or_insert(apple);
8      println!("第一个可用的水果是: {:?}", first_available_fruit);
9      println!("我的水果是: {:?}", my_fruit);
10     // first_available_fruit is: Apple
11     // my_fruit is: Some(Apple)
12     //println!("名为 `apple` 的变量已被移动: {:?}", apple);
13     // TODO: 取消上面这行的注释以查看编译器错误
14 }

```

get_or_insert_with() 惰性求值，原地修改空值

我们可以向 `get_or_insert_with` 传递一个闭包，而不是显式提供一个备选值。示例如下：

```
1  #[derive(Debug)]
2  enum Fruit { Apple, Orange, Banana, Kiwi, Lemon }
3
4  fn main() {
5      let mut my_fruit: Option<Fruit> = None;
6      let get_lemon_as_fallback = || {
7          println!("提供柠檬作为备选");
8          Fruit::Lemon
9      };
10     let first_available_fruit = my_fruit
11         .get_or_insert_with(get_lemon_as_fallback);
12     println!("第一个可用的水果是: {:?}", first_available_fruit);
13     println!("我的水果是: {:?}", my_fruit);
14     // 提供柠檬作为备选
15     // first_available_fruit is: Lemon
16     // my_fruit is: Some(Lemon)
17
18     // 如果 Option 已有值，它将保持不变，闭包不会被调用
19     let mut my_apple = Some(Fruit::Apple);
20     let should_be_apple = my_apple.get_or_insert_with(get_lemon_as_fallback);
21     println!("should_be_apple 的值为: {:?}", should_be_apple);
22     println!("my_apple 保持不变: {:?}", my_apple);
23     // 输出如下。注意闭包 `get_lemon_as_fallback` 并未被调用
24     // should_be_apple is: Apple
25     // my_apple is unchanged: Some(Apple)
26 }
```

另请参阅：

闭包、[get_or_insert](#)、[get_or_insert_with](#)、[变量移动](#)、[or](#)、[or_else](#)

Result

`Result` 是 `Option` 类型的增强版，它描述可能的**错误**而非可能的**缺失**。

也就是说，`Result<T, E>` 可能有两种结果之一：

- `Ok(T)`：找到了一个 `T` 类型的元素
- `Err(E)`：发生了一个 `E` 类型的错误

按照惯例，预期的结果是 `Ok`，而意外的结果是 `Err`。

与 `Option` 类似，`Result` 也有许多关联方法。例如，`unwrap()` 要么返回元素 `T`，要么触发 `panic`。对于情况处理，`Result` 和 `Option` 之间有许多重叠的组合子。

在使用 Rust 时，你可能会遇到返回 `Result` 类型的方法，比如 `parse()` 方法。将字符串解析为其他类型并非总是可行，因此 `parse()` 返回一个 `Result` 来表示可能的失败。

让我们看看成功和失败地 `parse()` 一个字符串会发生什么：

```
1 fn multiply(first_number_str: &str, second_number_str: &str) -> i32 {
2     // 让我们尝试使用 `unwrap()` 来获取数字。这样做会有问题吗？
3     let first_number = first_number_str.parse::<i32>().unwrap();
4     let second_number = second_number_str.parse::<i32>().unwrap();
5     first_number * second_number
6 }
7
8 fn main() {
9     let twenty = multiply("10", "2");
10    println!("double is {}", twenty);
11
12    let tt = multiply("t", "2");
13    println!("double is {}", tt);
14 }
```

在解析失败的情况下，`parse()` 会返回一个错误，导致 `unwrap()` 触发 `panic`。此外，`panic` 会终止程序并输出一条不友好的错误信息。

为了提高错误信息的质量，我们应该更明确地指定返回类型，并考虑显式地处理错误。

在 main 函数中使用 Result

如果显式指定，`Result` 类型也可以作为 `main` 函数的返回类型。通常，`main` 函数的形式如下：

```
fn main() {
    println!("Hello World!");
}
```

然而，`main` 函数也可以返回 `Result` 类型。如果 `main` 函数内发生错误，它将返回一个错误代码并打印该错误的调试表示（使用 `Debug` trait）。以下示例展示了这种情况，并涉及了下一节中讨论的内容。

```
1 use std::num::ParseIntError;
2
3 fn main() -> Result<(), ParseIntError> {
4     let number_str = "10";
5     let number = match number_str.parse::<i32>() {
6         Ok(number) => number,
7         Err(e) => return Err(e),
8     };
9     println!("{}", number);
10    Ok(())
11 }
```

Result 的 map

在前面示例的 `multiply` 函数中使用 `panic` 并不能产生健壮的代码。通常，我们希望将错误返回给调用者，让它决定如何正确地处理错误。

首先，我们需要知道我们正在处理的错误类型。要确定 `Err` 类型，我们可以查看 `parse()` 方法，它是通过 `FromStr` trait 为 `i32` 实现的。因此，`Err` 类型被指定为 `ParseIntError`。

在下面的示例中，直接使用 `match` 语句会导致整体代码更加繁琐。

```
1 use std::num::ParseIntError;
2
3 // 重写返回类型后，我们使用模式匹配而不是 `unwrap()`。
4 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i32, ParseIntError> {
5     match first_number_str.parse::<i32>() {
6         Ok(first_number) => {
7             match second_number_str.parse::<i32>() {
8                 Ok(second_number) => {
9                     Ok(first_number * second_number)
10                },
11                Err(e) => Err(e),
12            }
13        },
14        Err(e) => Err(e),
15    }
16 }
17
18 fn print(result: Result<i32, ParseIntError>) {
19     match result {
20         Ok(n) => println!("n 是 {}", n),
21         Err(e) => println!("错误: {}", e),
22     }
23 }
24
25 fn main() {
26     // 这仍然给出了一个合理的答案。
27     let twenty = multiply("10", "2");
28     print(twenty);
29
30     // 以下代码现在提供了一个更有帮助的错误消息。
31     let tt = multiply("t", "2");
32     print(tt);
33 }
```

幸运的是，`Option` 的 `map`、`and_then` 以及许多其他组合器也为 `Result` 实现了。[Result](#) 文档中包含了完整的列表。


```
1 use std::num::ParseIntError;
2
3 // 与 `Option` 类似，我们可以使用诸如 `map()` 之类的组合器。
4 // 这个函数除此之外与上面的函数相同，其含义为：
5 // 如果两个值都可以从字符串解析，则相乘；否则传递错误。
6 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i32,
7     first_number_str.parse::<i32>().and_then(|first_number| {
8         second_number_str.parse::<i32>().map(|second_number| first_number
9     })
10 }
11
12 fn print(result: Result<i32, ParseIntError>) {
13     match result {
14         Ok(n) => println!("n 是 {}", n),
15         Err(e) => println!("错误: {}", e),
16     }
17 }
18
19 fn main() {
20     // 这仍然给出了一个合理的答案。
21     let twenty = multiply("10", "2");
22     print(twenty);
23
24     // 以下代码现在提供了一个更有帮助的错误消息。
25     let tt = multiply("t", "2");
26     print(tt);
27 }
```

Result 的别名

如果我们想多次重用特定的 `Result` 类型，该怎么办？回想一下，Rust 允许我们创建**别名**。方便的是，我们可以为特定的 `Result` 定义一个别名。

在模块级别，创建别名特别有用。在特定模块中发现的错误通常具有相同的 `Err` 类型，因此单个别名可以简洁地定义**所有**相关的 `Result`。这非常实用，以至于 `std` 库甚至提供了一个：

`io::Result` !

这里有一个简单的例子来展示语法：

```
1 use std::num::ParseIntError;
2
3 // 为错误类型为 `ParseIntError` 的 `Result` 定义一个泛型别名。
4 type AliasedResult<T> = Result<T, ParseIntError>;
5
6 // 使用上面定义的别名来引用我们特定的 `Result` 类型。
7 fn multiply(first_number_str: &str, second_number_str: &str) -> AliasedResult<T> {
8     first_number_str.parse:::<i32>().and_then(|first_number| {
9         second_number_str.parse:::<i32>().map(|second_number| first_number
10     })
11 }
12
13 // 在这里，别名再次让我们节省了一些代码空间。
14 fn print(result: AliasedResult<i32>) {
15     match result {
16         Ok(n) => println!("n 是 {}", n),
17         Err(e) => println!("错误: {}", e),
18     }
19 }
20
21 fn main() {
22     print(multiply("10", "2"));
23     print(multiply("t", "2"));
24 }
```

另请参阅：

`io::Result`

提前返回

在前面的例子中，我们使用组合器显式地处理了错误。处理这种情况分析的另一种方法是使用 `match` 语句和**提前返回**的组合。

也就是说，如果发生错误，我们可以简单地停止执行函数并返回错误。对某些人来说，这种形式的代码可能更容易阅读和编写。考虑使用提前返回重写的前面示例的这个版本：

```
1 use std::num::ParseIntError;
2
3 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i32,
4     let first_number = match first_number_str.parse::<i32>() {
5         Ok(first_number) => first_number,
6         Err(e) => return Err(e),
7     };
8
9     let second_number = match second_number_str.parse::<i32>() {
10         Ok(second_number) => second_number,
11         Err(e) => return Err(e),
12     };
13
14     Ok(first_number * second_number)
15 }
16
17 fn print(result: Result<i32, ParseIntError>) {
18     match result {
19         Ok(n) => println!("n 是 {}", n),
20         Err(e) => println!("错误: {}", e),
21     }
22 }
23
24 fn main() {
25     print(multiply("10", "2"));
26     print(multiply("t", "2"));
27 }
```

到目前为止，我们已经学会了使用组合器和提前返回来显式处理错误。虽然我们通常想避免 `panic`，但显式处理所有错误是很繁琐的。

在下一节中，我们将介绍 `?` 运算符，用于我们只需要 `unwrap` 而不可能引发 `panic` 的情况。

引入 ?

有时我们只想要 `unwrap` 的简单性，而不希望有 `panic` 的可能。到目前为止，当我们真正想要的是获取变量值时，`unwrap` 迫使我们不断地增加嵌套。这正是 `?` 运算符的目的。

当遇到 `Err` 时，有两种可行的处理方式：

1. 使用 `panic!`（我们已经决定尽可能避免这种方式）
2. 使用 `return`（因为 `Err` 表示无法处理该错误）

`?` 运算符¹几乎等同于在遇到 `Err` 时执行 `return` 而非 `panic` 的 `unwrap`。让我们看看如何简化之前使用组合器的例子：

```
1 use std::num::ParseIntError;
2
3 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i32, ParseIntError> {
4     let first_number = first_number_str.parse::<i32>()?;
5     let second_number = second_number_str.parse::<i32>()?;
6
7     Ok(first_number * second_number)
8 }
9
10 fn print(result: Result<i32, ParseIntError>) {
11     match result {
12         Ok(n) => println!("n 是 {}", n),
13         Err(e) => println!("错误: {}", e),
14     }
15 }
16
17 fn main() {
18     print(multiply("10", "2"));
19     print(multiply("t", "2"));
20 }
```

try! 宏

在 `?` 运算符出现之前，相同的功能是通过 `try!` 宏实现的。现在推荐使用 `?` 运算符，但在查看旧代码时，你可能仍会遇到 `try!`。使用 `try!` 宏，前面例子中的 `multiply` 函数会是这样的：

```
1 // 要使用 Cargo 编译并运行此示例而不出错，请将 `Cargo.toml` 文件中
2 // `[package]` 部分的 `edition` 字段值更改为 "2015"。
3
4 use std::num::ParseIntError;
5
6 fn multiply(first_number_str: &str, second_number_str: &str) -> Result<i32> {
7     let first_number = try!(first_number_str.parse::<i32>());
8     let second_number = try!(second_number_str.parse::<i32>());
9
10    Ok(first_number * second_number)
11 }
12
13 fn print(result: Result<i32>, ParseIntError) {
14     match result {
15         Ok(n) => println!("n 是 {}", n),
16         Err(e) => println!("错误: {}", e),
17     }
18 }
19
20 fn main() {
21     print(multiply("10", "2"));
22     print(multiply("t", "2"));
23 }
```

1. 更多详情请参阅[重新认识 ?](#)。 ↩

多种错误类型

前面的例子一直都很方便：`Result` 与 `Result` 交互，`Option` 与 `Option` 交互。

有时，`Option` 需要与 `Result` 交互，或者 `Result<T, Error1>` 需要与 `Result<T, Error2>` 交互。在这些情况下，我们希望以一种使不同错误类型可组合且易于交互的方式来管理它们。

在下面的代码中，两个 `unwrap` 实例生成了不同的错误类型。`Vec::first` 返回一个 `Option`，而 `parse::<i32>` 返回一个 `Result<i32, ParseIntError>`：

```
1 fn double_first(vec: Vec<&str>) -> i32 {
2     let first = vec.first().unwrap(); // 生成错误 1
3     2 * first.parse::<i32>().unwrap() // 生成错误 2
4 }
5
6 fn main() {
7     let numbers = vec!["42", "93", "18"];
8     let empty = vec![];
9     let strings = vec!["tofu", "93", "18"];
10
11     println!("第一个数的两倍是 {}", double_first(numbers));
12
13     println!("第一个数的两倍是 {}", double_first(empty));
14     // 错误 1: 输入向量为空
15
16     println!("第一个数的两倍是 {}", double_first(strings));
17     // 错误 2: 元素无法解析为数字
18 }
```

在接下来的章节中，我们将探讨几种处理此类问题的策略。

从 Option 中提取 Result

处理混合错误类型最基本的方法是将它们相互嵌套。

```
1 use std::num::ParseIntError;
2
3 fn double_first(vec: Vec<&str>) -> Option<Result<i32, ParseIntError>> {
4     vec.first().map(|first| {
5         first.parse::<i32>().map(|n| 2 * n)
6     })
7 }
8
9 fn main() {
10     let numbers = vec!["42", "93", "18"];
11     let empty = vec![];
12     let strings = vec!["tofu", "93", "18"];
13
14     println!("第一个数的两倍是 {:?}", double_first(numbers));
15
16     println!("第一个数的两倍是 {:?}", double_first(empty));
17     // 错误 1: 输入向量为空
18
19     println!("第一个数的两倍是 {:?}", double_first(strings));
20     // 错误 2: 元素无法解析为数字
21 }
```

有时我们希望在遇到错误时停止处理（例如使用 `?`），但在 `Option` 为 `None` 时继续执行。这时 `transpose` 函数就派上用场了，它可以方便地交换 `Result` 和 `Option`。

```
1 use std::num::ParseIntError;
2
3 fn double_first(vec: Vec<&str>) -> Result<Option<i32>, ParseIntError> {
4     let opt = vec.first().map(|first| {
5         first.parse::<i32>().map(|n| 2 * n)
6     });
7
8     opt.transpose()
9 }
10
11 fn main() {
12     let numbers = vec!["42", "93", "18"];
13     let empty = vec![];
14     let strings = vec!["tofu", "93", "18"];
15
16     println!("第一个数的两倍是 {:?}", double_first(numbers));
17     println!("第一个数的两倍是 {:?}", double_first(empty));
18     println!("第一个数的两倍是 {:?}", double_first(strings));
19 }
```

定义错误类型

有时，用单一类型的错误来掩盖所有不同的错误可以简化代码。我们将通过自定义错误来演示这一点。

Rust 允许我们定义自己的错误类型。通常，一个“好的”错误类型应该：

- 用同一类型表示不同的错误
- 向用户展示友好的错误消息
- 易于与其他类型进行比较
 - 好的示例： `Err(EmptyVec)`
 - 不好的示例： `Err("请使用至少包含一个元素的向量".to_owned())`
- 能够保存错误的相关信息
 - 好的示例： `Err(BadChar(c, position))`
 - 不好的示例： `Err("此处不能使用 +".to_owned())`
- 能够与其他错误很好地组合


```
1 use std::fmt;
2
3 type Result<T> = std::result::Result<T, DoubleError>;
4
5 // 定义我们的错误类型。这些可以根据我们的错误处理情况进行自定义。
6 // 现在我们可以编写自己的错误，依赖底层的错误实现，
7 // 或者在两者之间做一些处理。
8 #[derive(Debug, Clone)]
9 struct DoubleError;
10
11 // 错误的生成与其显示方式是完全分离的。
12 // 无需担心显示样式会使复杂的逻辑变得混乱。
13 //
14 // 注意，我们没有存储关于错误的任何额外信息。这意味着如果不修改类型
15 // 来携带相关信息，我们就无法指出具体是哪个字符串解析失败了。
16 impl fmt::Display for DoubleError {
17     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
18         write!(f, "无效的待加倍项")
19     }
20 }
21
22 fn double_first(vec: Vec<&str>) -> Result<i32> {
23     vec.first()
24         // 将错误更改为我们的新类型。
25         .ok_or(DoubleError)
26         .and_then(|s| {
27             s.parse::<i32>()
28                 // 这里也更新为新的错误类型。
29                 .map_err(|_| DoubleError)
30                 .map(|i| 2 * i)
31         })
32 }
33
34 fn print(result: Result<i32>) {
35     match result {
36         Ok(n) => println!("第一个数的两倍是 {}", n),
37         Err(e) => println!("错误: {}", e),
38     }
39 }
40
41 fn main() {
42     let numbers = vec!["42", "93", "18"];
43     let empty = vec![];
44     let strings = vec!["tofu", "93", "18"];
45
46     print(double_first(numbers));
47     print(double_first(empty));
48     print(double_first(strings));
49 }
```

使用 Box 将错误装箱

一种既能编写简洁代码又能保留原始错误信息的方法是使用 `Box` 将它们装箱。这种方法的缺点是底层错误类型只能在运行时确定，而不是静态确定的。

标准库通过让 `Box` 实现从任何实现了 `Error` trait 的类型到 trait 对象 `Box<Error>` 的转换来帮助我们装箱错误，这是通过 `From` 实现的。

```
1 use std::error;
2 use std::fmt;
3
4 // 将别名改为使用 `Box<dyn error::Error>`。
5 type Result<T> = std::result::Result<T, Box<dyn error::Error>>;
6
7 #[derive(Debug, Clone)]
8 struct EmptyVec;
9
10 impl fmt::Display for EmptyVec {
11     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
12         write!(f, "无效的 第一个待加倍项")
13     }
14 }
15
16 impl error::Error for EmptyVec {}
17
18 fn double_first(vec: Vec<&str>) -> Result<i32> {
19     vec.first()
20         .ok_or_else(|| EmptyVec.into()) // Converts to Box using Into tra
21         .and_then(|s| {
22             s.parse::<i32>()
23                 .map_err(From::from) // Converts to Box using From::from
24                 .map(|i| 2 * i)
25         })
26 }
27
28 fn print(result: Result<i32>) {
29     match result {
30         Ok(n) => println!("第一个数的两倍是 {}", n),
31         Err(e) => println!("错误: {}", e),
32     }
33 }
34
35 fn main() {
36     let numbers = vec!["42", "93", "18"];
37     let empty = vec![];
38     let strings = vec!["tofu", "93", "18"];
39
40     print(double_first(numbers));
41     print(double_first(empty));
42     print(double_first(strings));
43 }
```

另请参阅：

[动态分发](#)和 [Error trait](#)

? 的其他用途

注意在前面的例子中，我们对调用 `parse` 的直接反应是将库错误通过 `map` 转换为一个装箱的错误：

```
.and_then(|s| s.parse::<i32>())  
    .map_err(|e| e.into())
```

由于这是一个简单且常见的操作，如果能够省略就会很方便。可惜的是，由于 `and_then` 不够灵活，所以无法实现这一点。不过，我们可以使用 `?` 来替代。

之前我们将 `?` 解释为 `unwrap` 或 `return Err(err)`。这只是大致正确。实际上，它的含义是 `unwrap` 或 `return Err(From::from(err))`。由于 `From::from` 是不同类型之间的转换工具，这意味着如果你在错误可转换为返回类型的地方使用 `?`，它将自动进行转换。

在这里，我们使用 `?` 重写了前面的例子。当为我们的错误类型实现 `From::from` 后，`map_err` 就不再需要了：

```

1 use std::error;
2 use std::fmt;
3
4 // 将别名改为使用 `Box<dyn error::Error>`。
5 type Result<T> = std::result::Result<T, Box<dyn error::Error>>;
6
7 #[derive(Debug)]
8 struct EmptyVec;
9
10 impl fmt::Display for EmptyVec {
11     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
12         write!(f, "无效的待加倍项")
13     }
14 }
15
16 impl error::Error for EmptyVec {}
17
18 // 结构与之前相同，但不再链式处理所有的 `Result` 和 `Option`，
19 // 而是使用 `?` 立即获取内部值。
20 fn double_first(vec: Vec<&str>) -> Result<i32> {
21     let first = vec.first().ok_or(EmptyVec)?;
22     let parsed = first.parse::<i32>()?;
23     Ok(2 * parsed)
24 }
25
26 fn print(result: Result<i32>) {
27     match result {
28         Ok(n) => println!("第一个数的两倍是 {}", n),
29         Err(e) => println!("错误: {}", e),
30     }
31 }
32
33 fn main() {
34     let numbers = vec!["42", "93", "18"];
35     let empty = vec![];
36     let strings = vec!["tofu", "93", "18"];
37
38     print(double_first(numbers));
39     print(double_first(empty));
40     print(double_first(strings));
41 }

```

现在代码变得相当简洁了。与原来使用 `panic` 的版本相比，这种方法非常类似于用 `?` 替换 `unwrap` 调用，只是返回类型变成了 `Result`。因此，需要在顶层对结果进行解构。

另请参阅：

[From::from](#) 和 [?](#) 运算符

包装错误

除了将错误装箱外，另一种方法是将它们包装在你自定义的错误类型中。

```

1 use std::error;
2 use std::error::Error;
3 use std::num::ParseIntError;
4 use std::fmt;
5
6 type Result<T> = std::result::Result<T, DoubleError>;
7
8 #[derive(Debug)]
9 enum DoubleError {
10     EmptyVec,
11     // 我们将使用解析错误的实现来处理它们的错误。
12     // 提供额外信息需要向类型添加更多数据。
13     Parse(ParseIntError),
14 }
15
16 impl fmt::Display for DoubleError {
17     fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
18         match *self {
19             DoubleError::EmptyVec =>
20                 write!(f, "请使用至少包含一个元素的向量"),
21             // 包装的错误包含额外信息, 可通过 source() 方法获取。
22             DoubleError::Parse(..) =>
23                 write!(f, "提供的字符串无法解析为整数"),
24         }
25     }
26 }
27
28 impl error::Error for DoubleError {
29     fn source(&self) -> Option<&(dyn error::Error + 'static)> {
30         match *self {
31             DoubleError::EmptyVec => None,
32             // 错误原因是底层实现的错误类型。它被隐式地
33             // 转换为 trait 对象 `&error::Error`。这是可行的, 因为
34             // 底层类型已经实现了 `Error` trait。
35             DoubleError::Parse(ref e) => Some(e),
36         }
37     }
38 }
39
40 // 实现从 `ParseIntError` 到 `DoubleError` 的转换。
41 // 当需要将 `ParseIntError` 转换为 `DoubleError` 时,
42 // `?` 运算符会自动调用这个转换。
43 impl From<ParseIntError> for DoubleError {
44     fn from(err: ParseIntError) -> DoubleError {
45         DoubleError::Parse(err)
46     }
47 }
48
49 fn double_first(vec: Vec<&str>) -> Result<i32> {
50     let first = vec.first().ok_or(DoubleError::EmptyVec)?;
51     // 这里我们隐式使用了 `ParseIntError` 的 `From` 实现 (我们在上面定义的
52     // 来创建一个 `DoubleError`。
53     let parsed = first.parse::<i32>()?;
54
55     Ok(2 * parsed)
56 }
57
58 fn print(result: Result<i32>) {
59     match result {
60         Ok(n) => println!("第一个数的两位是 {}", n)

```

```
60         Ok(n) => println!("第 {} 个数的两倍是 {}", n, 2 * n);
61     Err(e) => {
62         println!("错误: {}", e);
63         if let Some(source) = e.source() {
64             println!("  错误原因: {}", source);
65         }
66     },
67 }
68 }
69
70 fn main() {
71     let numbers = vec!["42", "93", "18"];
72     let empty = vec![];
73     let strings = vec!["tofu", "93", "18"];
74
75     print(double_first(numbers));
76     print(double_first(empty));
77     print(double_first(strings));
78 }
```

这种方法增加了一些处理错误的样板代码，可能并非所有应用程序都需要。有一些库可以帮你处理这些样板代码。

另请参阅：

[From::from](#) 和 [枚举](#)

[处理错误的 crate](#)

遍历 Result

`Iter::map` 操作可能会失败，例如：

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let numbers: Vec<_> = strings
4         .into_iter()
5         .map(|s| s.parse::<i32>())
6         .collect();
7     println!("结果: {:?}", numbers);
8 }
```

让我们逐步介绍处理这种情况的策略。

使用 `filter_map()` 忽略失败的项

`filter_map` 调用一个函数并过滤掉结果为 `None` 的项。

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let numbers: Vec<_> = strings
4         .into_iter()
5         .filter_map(|s| s.parse::<i32>().ok())
6         .collect();
7     println!("结果: {:?}", numbers);
8 }
```

使用 `map_err()` 和 `filter_map()` 收集失败的项

`map_err` 会对错误调用一个函数，因此将其添加到之前的 `filter_map` 解决方案中，我们可以在迭代时将错误项保存到一旁。

```
1 fn main() {
2     let strings = vec!["42", "tofu", "93", "999", "18"];
3     let mut errors = vec![];
4     let numbers: Vec<_> = strings
5         .into_iter()
6         .map(|s| s.parse::<u8>())
7         .filter_map(|r| r.map_err(|e| errors.push(e)).ok())
8         .collect();
9     println!("数字: {:?}", numbers);
10    println!("错误: {:?}", errors);
11 }
```

使用 collect() 使整个操作失败

`Result` 实现了 `FromIterator` trait, 因此结果的向量 (`Vec<Result<T, E>>`) 可以转换为包含向量的结果 (`Result<Vec<T>, E>`)。一旦遇到 `Result::Err`, 迭代就会终止。

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let numbers: Result<Vec<_>, _> = strings
4         .into_iter()
5         .map(|s| s.parse::<i32>())
6         .collect();
7     println!("结果: {:?}", numbers);
8 }
```

这种技巧同样适用于 `Option`。

使用 partition() 收集所有有效值和失败项

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let (numbers, errors): (Vec<_>, Vec<_>) = strings
4         .into_iter()
5         .map(|s| s.parse::<i32>())
6         .partition(Result::is_ok);
7     println!("数字: {:?}", numbers);
8     println!("错误: {:?}", errors);
9 }
```

当你查看结果时, 你会注意到所有内容仍然被包装在 `Result` 中。这需要一些额外的样板代码。

```
1 fn main() {
2     let strings = vec!["tofu", "93", "18"];
3     let (numbers, errors): (Vec<_>, Vec<_>) = strings
4         .into_iter()
5         .map(|s| s.parse::<i32>())
6         .partition(Result::is_ok);
7     let numbers: Vec<_> = numbers.into_iter().map(Result::unwrap).collect();
8     let errors: Vec<_> = errors.into_iter().map(Result::unwrap_err).collect();
9     println!("数字: {:?}", numbers);
10    println!("错误: {:?}", errors);
11 }
```

标准库类型

`std` 库提供了许多自定义类型，大大扩展了“原生类型”的功能。其中包括：

- 可增长的 `String`，例如： `"hello world"`
- 可增长的向量（vector）： `[1, 2, 3]`
- 可选类型： `Option<i32>`
- 错误处理类型： `Result<i32, i32>`
- 堆分配的指针： `Box<i32>`

另请参阅：

[原生类型](#)和[标准库](#)

Box、栈和堆

在 Rust 中，所有值默认都是栈分配的。通过创建 `Box<T>`，可以将值**装箱**（在堆上分配）。`Box` 是指向堆分配的 `T` 类型值的智能指针。当 `Box` 离开作用域时，会调用其析构函数，内部对象被销毁，堆上的内存被释放。

可以使用 `*` 运算符解引用装箱的值，这会移除一层间接引用。

```
1 use std::mem;
2
3 #[allow(dead_code)]
4 #[derive(Debug, Clone, Copy)]
5 struct Point {
6     x: f64,
7     y: f64,
8 }
9
10 // 可以通过指定左上角和右下角在空间中的位置来定义矩形
11 #[allow(dead_code)]
12 struct Rectangle {
13     top_left: Point,
14     bottom_right: Point,
15 }
16
17 fn origin() -> Point {
18     Point { x: 0.0, y: 0.0 }
19 }
20
21 fn boxed_origin() -> Box<Point> {
22     // 在堆上分配这个点，并返回指向它的指针
23     Box::new(Point { x: 0.0, y: 0.0 })
24 }
25
26 fn main() {
27     // （所有的类型标注都不是必须的）
28     // 栈分配的变量
29     let point: Point = origin();
30     let rectangle: Rectangle = Rectangle {
31         top_left: origin(),
32         bottom_right: Point { x: 3.0, y: -4.0 }
33     };
34
35     // 堆分配的矩形
36     let boxed_rectangle: Box<Rectangle> = Box::new(Rectangle {
37         top_left: origin(),
38         bottom_right: Point { x: 3.0, y: -4.0 },
39     });
40
41     // 函数的输出可以被装箱
42     let boxed_point: Box<Point> = Box::new(origin());
43
44     // 双重间接引用
45     let box_in_a_box: Box<Box<Point>> = Box::new(boxed_origin());
46
47     println!("Point 在栈上占用 {} 字节",
48             mem::size_of_val(&point));
49     println!("Rectangle 在栈上占用 {} 字节",
50             mem::size_of_val(&rectangle));
51
52     // box 大小 == 指针大小
53     println!("装箱的 point 在栈上占用 {} 字节",
54             mem::size_of_val(&boxed_point));
55     println!("装箱的 rectangle 在栈上占用 {} 字节",
56             mem::size_of_val(&boxed_rectangle));
57     println!("装箱的 box 在栈上占用 {} 字节",
58             mem::size_of_val(&box_in_a_box));
59
60     // 将 `boxed_point` 中的数据复制到 `unboxed_point`
```

```
60 // 将 boxed_point 中的数据复制到 unboxed_point
61 let unboxed_point: Point = *boxed_point;
62 println!("未装箱的 point 在栈上占用 {} 字节",
63         mem::size_of_val(&unboxed_point));
64 }
```

Vectors

向量（Vector）是可调整大小的数组。与切片（Slice）类似，它们的大小在编译时是未知的，但可以随时增长或缩小。向量由 3 个参数表示：

- 指向数据的指针
- 长度
- 容量

容量表示为向量预留的内存量。只要长度小于容量，向量就可以增长。当需要超过这个阈值时，向量会被重新分配更大的容量。

```
1 fn main() {
2     // 迭代器可以被收集到向量中
3     let collected_iterator: Vec<i32> = (0..10).collect();
4     println!("将 (0..10) 收集到: {:?}", collected_iterator);
5
6     // 可以使用 `vec!` 宏初始化向量
7     let mut xs = vec![1i32, 2, 3];
8     println!("初始向量: {:?}", xs);
9
10    // 在向量末尾插入新元素
11    println!("将 4 推入向量");
12    xs.push(4);
13    println!("向量: {:?}", xs);
14
15    // 错误! 不可变向量无法增长
16    collected_iterator.push(0);
17    // 修复: ^ 注释掉此行
18
19    // `len` 方法返回当前存储在向量中的元素数量
20    println!("向量长度: {}", xs.len());
21
22    // 使用方括号进行索引 (索引从 0 开始)
23    println!("第二个元素: {}", xs[1]);
24
25    // `pop` 移除并返回向量的最后一个元素
26    println!("弹出最后一个元素: {:?}", xs.pop());
27
28    // 越界索引会导致 panic
29    println!("第四个元素: {}", xs[3]);
30    // 修复: ^ 注释掉此行
31
32    // 可以轻松遍历 `Vector`
33    println!("xs 的内容: ");
34    for x in xs.iter() {
35        println!("> {}", x);
36    }
37
38    // 遍历 `Vector` 时, 可以同时用一个单独的变量 (`i`)
39    // 来枚举迭代计数
40    for (i, x) in xs.iter().enumerate() {
41        println!("在位置 {} 的值是 {}", i, x);
42    }
43
44    // 借助 `iter_mut`, 可变的 `Vector` 也可以被遍历,
45    // 并且允许修改每个值
46    for x in xs.iter_mut() {
47        *x *= 3;
48    }
49    println!("更新后的向量: {:?}", xs);
50 }
```

更多 Vec 方法可以在 [std::vec](#) 模块中找到

字符串

Rust 中最常用的两种字符串类型是 `String` 和 `&str`。

`String` 存储为字节向量 (`Vec<u8>`)，但保证始终是有效的 UTF-8 序列。`String` 在堆上分配，可增长，且不以 `null` 结尾。

`&str` 是一个切片 (`&[u8]`)，它始终指向一个有效的 UTF-8 序列，可以用来查看 `String`，就像 `&[T]` 是 `Vec<T>` 的一个视图一样。

```
1 fn main() {
2     // (所有的类型注解都不是必须的)
3     // 一个指向只读内存中分配的字符串的引用
4     let pangram: &'static str = "敏捷的棕色狐狸跳过懒惰的狗";
5     println!("全字母句: {}", pangram);
6
7     // 反向遍历单词，不会分配新的字符串
8     println!("单词逆序");
9     for word in pangram.split_whitespace().rev() {
10         println!("> {}", word);
11     }
12
13     // 将字符复制到向量中，排序并去重
14     let mut chars: Vec<char> = pangram.chars().collect();
15     chars.sort();
16     chars.dedup();
17
18     // 创建一个空的、可增长的 `String`
19     let mut string = String::new();
20     for c in chars {
21         // 在字符串末尾插入一个字符
22         string.push(c);
23         // 在字符串末尾追加另一个字符串
24         string.push_str(", ");
25     }
26
27     // 修剪后的字符串是原始字符串的一个切片，因此不会进行新的内存分配
28     let chars_to_trim: &[char] = &[' ', ','];
29     let trimmed_str: &str = string.trim_matches(chars_to_trim);
30     println!("使用的字符: {}", trimmed_str);
31
32     // 在堆上分配一个字符串
33     let alice = String::from("我喜欢狗");
34     // 分配新的内存并在其中存储修改后的字符串
35     let bob: String = alice.replace("狗", "猫");
36
37     println!("爱丽丝说: {}", alice);
38     println!("鲍勃说: {}", bob);
39 }
```

更多 `str` 和 `String` 的方法可以在 `std::str` 和 `std::string` 模块中找到

字面值和转义字符

有多种方式可以编写包含特殊字符的字符串面值。所有方式都会产生类似的 `&str`，因此最好使用最方便编写的形式。同样，也有多种方式可以编写字节字符串面值，它们都会产生 `&[u8; N]` 类型。

通常，特殊字符用反斜杠（`\`）进行转义。这样你可以在字符串中添加任何字符，包括不可打印的字符和你不知道如何输入的字符。如果你想要一个字面的反斜杠，用另一个反斜杠转义它：`\\`

出现在面值内的字符串或字符面值分隔符必须被转义：`"\"` 和 `'\''`。

```
1 fn main() {
2     // 你可以使用转义来通过十六进制值写入字节...
3     let byte_escape = "我正在写 \x52\x75\x73\x74! ";
4     println!("你在做什么 \x3F (\\x3F 表示 ?) {}", byte_escape);
5
6     // ...或 Unicode 码点。
7     let unicode_codepoint = "\u{211D}";
8     let character_name = "\"双线大写 R\"";
9
10    println!("Unicode 字符 {} (U+211D) 被称为 {}",
11             unicode_codepoint, character_name );
12
13
14    let long_string = "字符串面值
15                      可以跨越多行。
16                      这里的换行和缩进 -> \
17                      <- 也可以被转义! ";
18    println!("{}", long_string);
19 }
```

有时需要转义的字符太多，或者直接按原样写出字符串会更方便。这时就可以使用原始字符串面值。

```
1 fn main() {
2     let raw_str = r"转义在这里不起作用： \x3F \u{211D}";
3     println!("{}", raw_str);
4
5     // 如果你需要在原始字符串中使用引号，可以添加一对 #
6     let quotes = r#"然后我说： "无处可逃！" "#;
7     println!("{}", quotes);
8
9     // 如果你需要在字符串中使用 "#，只需在分隔符中使用更多的 #。
10    // 你最多可以使用 255 个 #。
11    let longer_delimiter = r####"一个包含 "# 的字符串。甚至还有 "##! "###;
12    println!("{}", longer_delimiter);
13 }
```

想要一个非 UTF-8 的字符串吗？（请记住，`str` 和 `String` 必须是有效的 UTF-8）。或者你想要一个主要是文本的字节数组？字节字符串来帮忙！

```
1 use std::str;
2
3 fn main() {
4     // 注意，这实际上不是一个 `&str`
5     let bytestring: &[u8; 21] = b"这是一个字节字符串";
6
7     // 字节数组没有 `Display` trait, 所以打印它们有一些限制
8     println!("一个字节字符串: {:?}", bytestring);
9
10    // 字节字符串可以包含字节转义...
11    let escaped = b"\x52\x75\x73\x74 作为字节";
12    // ...但不允许 Unicode 转义
13    // let escaped = b"\u{211D}" 是不允许的";
14    println!("一些转义的字节: {:?}", escaped);
15
16
17    // 原始字节字符串的工作方式与原始字符串相同
18    let raw_bytestring = br"\u{211D} 在这里不会被转义";
19    println!("{:?}", raw_bytestring);
20
21    // 将字节数组转换为 `str` 可能会失败
22    if let Ok(my_str) = str::from_utf8(raw_bytestring) {
23        println!("作为文本的相同内容: '{}'", my_str);
24    }
25
26    let _quotes = br#"你也可以使用"更花哨的"格式, \
27                  就像普通的原始字符串一样"#;
28
29    // 字节字符串不必是 UTF-8 编码
30    let shift_jis = b"\x82\xe6\x82\xa8\x82\xb1\x82\xbb"; // SHIFT-JIS 编码
31
32    // 但它们并不总是能被转换为 `str`
33    match str::from_utf8(shift_jis) {
34        Ok(my_str) => println!("转换成功: '{}'", my_str),
35        Err(e) => println!("转换失败: {:?}", e),
36    };
37 }
```

对于字符编码之间的转换，请查看 [encoding crate](#)。

关于编写字符串字面值和转义字符的更详细说明，请参阅 Rust 参考手册的「[标记](#)」章节。

Option

有时我们希望捕获程序某些部分的失败，而不是调用 `panic!`。这可以通过使用 `Option` 枚举来实现。

`Option<T>` 枚举有两个变体：

- `None`：表示失败或缺少值，以及
- `Some(value)`：一个元组结构体，包装了类型为 `T` 的 `value`。

```
1 // 一个不会触发 `panic!` 的整数除法
2 fn checked_division(dividend: i32, divisor: i32) -> Option<i32> {
3     if divisor == 0 {
4         // 失败用 `None` 变体表示
5         None
6     } else {
7         // 结果被包装在 `Some` 变体中
8         Some(dividend / divisor)
9     }
10 }
11
12 // 这个函数处理可能不成功的除法运算
13 fn try_division(dividend: i32, divisor: i32) {
14     // `Option` 值可以进行模式匹配，就像其他枚举一样
15     match checked_division(dividend, divisor) {
16         None => println!("{}", / {} 失败!", dividend, divisor),
17         Some(quotient) => {
18             println!("{}", / {} = {}", dividend, divisor, quotient)
19         },
20     }
21 }
22
23 fn main() {
24     try_division(4, 2);
25     try_division(1, 0);
26
27     // 将 `None` 绑定到变量时需要进行类型注解
28     let none: Option<i32> = None;
29     let _equivalent_none = None::<i32>;
30
31     let optional_float = Some(0f32);
32
33     // 解包 `Some` 变体将提取其中包装的值。
34     println!("{}", 解包后得到 {:?}", optional_float, optional_float.unwrap());
35
36     // 解包 `None` 变体将触发 `panic!`
37     println!("{}", 解包后得到 {:?}", none, none.unwrap());
38 }
```

Result

我们已经看到 `Option` 枚举可以用作可能失败的函数的返回值，其中 `None` 用于表示失败。然而，有时表达操作失败的原因很重要。为此，我们有 `Result` 枚举。

`Result<T, E>` 枚举有两个变体：

- `Ok(value)`：表示操作成功，并包装了操作返回的 `value`。（`value` 的类型为 `T`）
- `Err(why)`：表示操作失败，并包装了 `why`，它（希望）解释了失败的原因。（`why` 的类型为 `E`）

```

1  mod checked {
2      // 我们要捕获的数学"错误"
3      #[derive(Debug)]
4      pub enum MathError {
5          DivisionByZero,
6          NonPositiveLogarithm,
7          NegativeSquareRoot,
8      }
9
10     pub type MathResult = Result<f64, MathError>;
11
12     pub fn div(x: f64, y: f64) -> MathResult {
13         if y == 0.0 {
14             // 这个操作会"失败", 所以我们返回包装在 `Err` 中的失败原因
15             Err(MathError::DivisionByZero)
16         } else {
17             // 这个操作是有效的, 返回包装在 `Ok` 中的结果
18             Ok(x / y)
19         }
20     }
21
22     pub fn sqrt(x: f64) -> MathResult {
23         if x < 0.0 {
24             Err(MathError::NegativeSquareRoot)
25         } else {
26             Ok(x.sqrt())
27         }
28     }
29
30     pub fn ln(x: f64) -> MathResult {
31         if x <= 0.0 {
32             Err(MathError::NonPositiveLogarithm)
33         } else {
34             Ok(x.ln())
35         }
36     }
37 }
38
39 // `op(x, y)` == `sqrt(ln(x / y))`
40 fn op(x: f64, y: f64) -> f64 {
41     // 这是一个三层的匹配金字塔!
42     match checked::div(x, y) {
43         Err(why) => panic!("{:?}", why),
44         Ok(ratio) => match checked::ln(ratio) {
45             Err(why) => panic!("{:?}", why),
46             Ok(ln) => match checked::sqrt(ln) {
47                 Err(why) => panic!("{:?}", why),
48                 Ok(sqrt) => sqrt,
49             },
50         },
51     }
52 }
53
54 fn main() {
55     // 这会失败吗?
56     println!("{}", op(1.0, 10.0));
57 }

```

?

使用 `match` 链式处理结果可能会变得相当混乱；幸运的是，我们可以使用 `?` 运算符来让代码变得整洁。`?` 运算符用在返回 `Result` 的表达式末尾，等效于一个 `match` 表达式。在这个表达式中，`Err(err)` 分支会展开为提前返回的 `return Err(From::from(err))`，而 `Ok(ok)` 分支则展开为 `ok` 表达式。

```
1 mod checked {
2     #[derive(Debug)]
3     enum MathError {
4         DivisionByZero,
5         NonPositiveLogarithm,
6         NegativeSquareRoot,
7     }
8
9     type MathResult = Result<f64, MathError>;
10
11     fn div(x: f64, y: f64) -> MathResult {
12         if y == 0.0 {
13             Err(MathError::DivisionByZero)
14         } else {
15             Ok(x / y)
16         }
17     }
18
19     fn sqrt(x: f64) -> MathResult {
20         if x < 0.0 {
21             Err(MathError::NegativeSquareRoot)
22         } else {
23             Ok(x.sqrt())
24         }
25     }
26
27     fn ln(x: f64) -> MathResult {
28         if x <= 0.0 {
29             Err(MathError::NonPositiveLogarithm)
30         } else {
31             Ok(x.ln())
32         }
33     }
34
35     // 中间函数
36     fn op_(x: f64, y: f64) -> MathResult {
37         // 如果 `div` "失败", 则会 `return` `DivisionByZero`
38         let ratio = div(x, y)?;
39
40         // 如果 `ln` "失败", 则会 `return` `NonPositiveLogarithm`
41         let ln = ln(ratio)?;
42
43         sqrt(ln)
44     }
45
46     pub fn op(x: f64, y: f64) {
47         match op_(x, y) {
48             Err(why) => panic!("{}", match why {
49                 MathError::NonPositiveLogarithm
50                     => "非正数的对数",
51                 MathError::DivisionByZero
52                     => "除以零",
53                 MathError::NegativeSquareRoot
54                     => "负数的平方根",
55             }),
56             Ok(value) => println!("{}", value),
57         }
58     }
59 }
```



```
61 fn main() {  
62     checked::op(1.0, 10.0);  
63 }
```

请务必查阅 [文档](#)，其中包含了许多用于映射和组合 `Result` 的方法。

panic!

`panic!` 宏可用于生成一个 panic 并开始展开其栈。在展开过程中，运行时会通过调用该线程所有对象的析构函数来释放线程**拥有**的所有资源。

由于我们处理的是只有一个线程的程序，`panic!` 会导致程序报告 panic 消息并退出。

```

1  // 重新实现整数除法 (/)
2  fn division(dividend: i32, divisor: i32) -> i32 {
3      if divisor == 0 {
4          // 除以零会触发 panic
5          panic!("除以零");
6      } else {
7          dividend / divisor
8      }
9  }
10
11 // `main` 任务
12 fn main() {
13     // 堆分配的整数
14     let _x = Box::new(0i32);
15
16     // 这个操作将触发任务失败
17     division(3, 0);
18
19     println!("这个点不会被执行到!");
20
21     // `_x` 应该在此处被销毁
22 }
```

让我们验证 `panic!` 不会导致内存泄漏。

```

$ rustc panic.rs && valgrind ./panic
==4401== Memcheck, a memory error detector
==4401== Copyright (C) 2002-2013, and GNU GPL'd, by Julian Seward et al.
==4401== Using Valgrind-3.10.0.SVN and LibVEX; rerun with -h for copyright
info
==4401== Command: ./panic
==4401==
thread '<main>' panicked at 'division by zero', panic.rs:5
==4401==
==4401== HEAP SUMMARY:
==4401==     in use at exit: 0 bytes in 0 blocks
==4401==   total heap usage: 18 allocs, 18 frees, 1,648 bytes allocated
==4401==
==4401== All heap blocks were freed -- no leaks are possible
==4401==
==4401== For counts of detected and suppressed errors, rerun with: -v
==4401== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

HashMap

向量 (Vector) 通过整数索引存储值，而 `HashMap` 则通过键存储值。`HashMap` 的键可以是布尔值、整数、字符串，或任何其他实现了 `Eq` 和 `Hash trait` 的类型。下一节将详细介绍这一点。

与向量类似，`HashMap` 也可以增长，但当有多余空间时，`HashMap` 还能自动收缩。你可以使用 `HashMap::with_capacity(uint)` 创建一个具有指定初始容量的 `HashMap`，或使用 `HashMap::new()` 来获得一个具有默认初始容量的 `HashMap` (推荐)。

```
1 use std::collections::HashMap;
2
3 fn call(number: &str) -> &str {
4     match number {
5         "798-1364" => "很抱歉，无法接通您拨打的电话。
6             请挂机后重试。",
7         "645-7689" => "您好，这里是 Awesome 先生的披萨店。我是 Fred。
8             请问今天您想点些什么？",
9         _ => "嗨！请问是哪位？"
10    }
11 }
12
13 fn main() {
14     let mut contacts = HashMap::new();
15
16     contacts.insert("Daniel", "798-1364");
17     contacts.insert("Ashley", "645-7689");
18     contacts.insert("Katie", "435-8291");
19     contacts.insert("Robert", "956-1745");
20
21     // 接受一个引用并返回 Option<&V>
22     match contacts.get(&"Daniel") {
23         Some(&number) => println!("正在呼叫 Daniel: {}", call(number)),
24         _ => println!("没有 Daniel 的电话号码。"),
25     }
26
27     // 如果插入的值是新的，`HashMap::insert()` 返回 `None`
28     // 否则返回 `Some(value)`
29     contacts.insert("Daniel", "164-6743");
30
31     match contacts.get(&"Ashley") {
32         Some(&number) => println!("正在呼叫 Ashley: {}", call(number)),
33         _ => println!("没有 Ashley 的电话号码。"),
34     }
35
36     contacts.remove(&"Ashley");
37
38     // `HashMap::iter()` 返回一个迭代器，该迭代器以任意顺序生成
39     // (&'a key, &'a value) 键值对。
40     for (contact, &number) in contacts.iter() {
41         println!("正在呼叫 {}: {}", contact, call(number));
42     }
43 }
```

要了解更多关于哈希和哈希映射（有时称为哈希表）的工作原理，请参阅[哈希表的维基百科页面](#)

更改或自定义键类型

任何实现了 `Eq` 和 `Hash trait` 的类型都可以作为 `HashMap` 的键。这包括：

- `bool`（虽然用处不大，因为只有两个可能的键值）
- `int`、`uint` 及其所有变体
- `String` 和 `&str`（专业提示：你可以使用 `String` 作为 `HashMap` 的键，并用 `&str` 调用 `.get()` 方法）

注意，`f32` 和 `f64` **并未**实现 `Hash trait`，这很可能是因为[浮点数精度误差](#)会导致将它们用作哈希映射的键时极易出错。

如果集合类中包含的类型分别实现了 `Eq` 和 `Hash`，那么这些集合类也会实现 `Eq` 和 `Hash`。例如，如果 `T` 实现了 `Hash`，那么 `Vec<T>` 也会实现 `Hash`。

你可以通过一行代码轻松地为自定义类型实现 `Eq` 和 `Hash`：`#[derive(PartialEq, Eq, Hash)]`

编译器会完成剩余的工作。如果你想对细节有更多控制，可以自己实现 `Eq` 和/或 `Hash`。本指南不会涉及实现 `Hash` 的具体细节。

为了尝试在 `HashMap` 中使用 `struct`，让我们来创建一个非常简单的用户登录系统：

```
1 use std::collections::HashMap;
2
3 // Eq 要求你在类型上派生 PartialEq。
4 #[derive(PartialEq, Eq, Hash)]
5 struct Account<'a>{
6     username: &'a str,
7     password: &'a str,
8 }
9
10 struct AccountInfo<'a>{
11     name: &'a str,
12     email: &'a str,
13 }
14
15 type Accounts<'a> = HashMap<Account<'a>, AccountInfo<'a>>;
16
17 fn try_logon<'a>(accounts: &Accounts<'a>,
18     username: &'a str, password: &'a str){
19     println!("用户名: {}", username);
20     println!("密码: {}", password);
21     println!("正在尝试登录...");
22
23     let logon = Account {
24         username,
25         password,
26     };
27
28     match accounts.get(&logon) {
29         Some(account_info) => {
30             println!("登录成功!");
31             println!("姓名: {}", account_info.name);
32             println!("电子邮箱: {}", account_info.email);
33         },
34         _ => println!("登录失败!"),
35     }
36 }
37
38 fn main(){
39     let mut accounts: Accounts = HashMap::new();
40
41     let account = Account {
42         username: "j.everyman",
43         password: "password123",
44     };
45
46     let account_info = AccountInfo {
47         name: "John Everyman",
48         email: "j.everyman@email.com",
49     };
50
51     accounts.insert(account, account_info);
52
53     try_logon(&accounts, "j.everyman", "psasword123");
54
55     try_logon(&accounts, "j.everyman", "password123");
56 }
```

HashSet

可以将 `HashSet` 视为一个只关心键的 `HashMap`（实际上，`HashSet<T>` 只是 `HashMap<T, ()>` 的封装）。

你可能会问：“这有什么意义？我可以直接把键存储在 `Vec` 中啊。”

`HashSet` 的独特之处在于它保证不会有重复元素。这是所有集合类型都应满足的约定。
`HashSet` 只是其中一种实现。（另请参阅：[BTreeSet](#)）

如果你插入一个已存在于 `HashSet` 中的值（即新值等于现有值，且它们的哈希值相同），那么新值将替换旧值。

当你不希望某个元素出现多次，或者想知道是否已经拥有某个元素时，这非常有用。

但是集合的功能不仅限于此。

集合有 4 种主要操作（以下所有调用都返回一个迭代器）：

- 并集（`union`）：获取两个集合中的所有唯一元素。
- 差集（`difference`）：获取存在于第一个集合但不在第二个集合中的所有元素。
- 交集（`intersection`）：获取**同时**存在于两个集合中的所有元素。
- 对称差（`symmetric_difference`）：获取存在于其中一个集合中，但**不**同时存在于两个集合中的所有元素。

在下面的例子中尝试这些操作：

```
1 use std::collections::HashSet;
2
3 fn main() {
4     let mut a: HashSet<i32> = vec![1i32, 2, 3].into_iter().collect();
5     let mut b: HashSet<i32> = vec![2i32, 3, 4].into_iter().collect();
6
7     assert!(a.insert(4));
8     assert!(a.contains(&4));
9
10    // 如果集合中已存在该值, `HashSet::insert()` 将返回 false。
11    assert!(b.insert(4), "值 4 已存在于集合 B 中!");
12    // 修复: ^ 注释掉此行
13
14    b.insert(5);
15
16    // 如果集合的元素类型实现了 `Debug` 特征,
17    // 那么该集合也会实现 `Debug` 特征。
18    // 通常会以 `[elem1, elem2, ...]` 的格式打印其元素
19    println!("A: {:?}", a);
20    println!("B: {:?}", b);
21
22    // 以任意顺序打印 [1, 2, 3, 4, 5]
23    println!("并集: {:?}", a.union(&b).collect::<Vec<&i32>>());
24
25    // 这里应该打印 [1]
26    println!("差集: {:?}", a.difference(&b).collect::<Vec<&i32>>());
27
28    // 以任意顺序打印 [2, 3, 4]
29    println!("交集: {:?}", a.intersection(&b).collect::<Vec<&i32>>());
30
31    // 打印 [1, 5]
32    println!("对称差: {:?}",
33             a.symmetric_difference(&b).collect::<Vec<&i32>>());
34 }
```

(示例改编自[官方文档](#))

Rc

当需要多重所有权时，可以使用 `Rc`（引用计数，Reference Counting）。`Rc` 会跟踪引用的数量，即包裹在 `Rc` 内部的值的所有者数量。

每当克隆一个 `Rc` 时，其引用计数就会增加 1；每当一个克隆的 `Rc` 离开作用域时，引用计数就会减少 1。当 `Rc` 的引用计数变为零（意味着没有剩余的所有者）时，`Rc` 及其包含的值都会被丢弃。

克隆 `Rc` 从不执行深拷贝。克隆只是创建另一个指向被包裹值的指针，并增加计数。

```
1 use std::rc::Rc;
2
3 fn main() {
4     let rc_examples = "Rc 示例".to_string();
5     {
6         println!("--- rc_a 已创建 ---");
7
8         let rc_a: Rc<String> = Rc::new(rc_examples);
9         println!("rc_a 的引用计数: {}", Rc::strong_count(&rc_a));
10
11         {
12             println!("--- rc_a 被克隆为 rc_b ---");
13
14             let rc_b: Rc<String> = Rc::clone(&rc_a);
15             println!("rc_b 的引用计数: {}", Rc::strong_count(&rc_b));
16             println!("rc_a 的引用计数: {}", Rc::strong_count(&rc_a));
17
18             // 两个 `Rc` 如果内部值相等，则它们相等
19             println!("rc_a 和 rc_b 是否相等: {}", rc_a.eq(&rc_b));
20
21             // 我们可以直接使用值的方法
22             println!("rc_a 内部值的长度: {}", rc_a.len());
23             println!("rc_b 的值: {}", rc_b);
24
25             println!("--- rc_b 超出作用域被释放 ---");
26         }
27
28         println!("rc_a 的引用计数: {}", Rc::strong_count(&rc_a));
29
30         println!("--- rc_a 超出作用域被释放 ---");
31     }
32
33     // 错误! `rc_examples` 已经被移动到 `rc_a` 中
34     // 当 `rc_a` 被释放时，`rc_examples` 也会一同被释放
35     // println!("rc_examples: {}", rc_examples);
36     // TODO: 尝试取消注释上面这行
37 }
```

另请参阅：

`std::rc` 和 `std::sync::arc`。

Arc

当需要在线程间共享所有权时，可以使用 `Arc`（原子引用计数，Atomically Reference Counted）。通过 `clone` 实现，这个结构体可以为堆内存中值的位置创建一个引用指针，同时增加引用计数。由于它在线程间共享所有权，当指向某个值的最后一个引用指针超出作用域时，该变量就会被释放。

```
1 use std::time::Duration;
2 use std::sync::Arc;
3 use std::thread;
4
5 fn main() {
6     // 在这个变量声明中指定了它的值。
7     let apple = Arc::new("同一个苹果");
8
9     for _ in 0..10 {
10        // 这里没有指定值，因为它是指向堆内存中引用的指针。
11        let apple = Arc::clone(&apple);
12
13        thread::spawn(move || {
14            // 由于使用了 Arc，可以使用 Arc 变量指针所指向的值来生成线程。
15            println!("{:?}", apple);
16        });
17    }
18
19    // 确保所有 Arc 实例都从生成的线程中打印出来。
20    thread::sleep(Duration::from_secs(1));
21 }
```

标准库中的其他内容

标准库提供了许多其他类型来支持各种功能，例如：

- 线程
- 信道
- 文件 I/O

这些类型扩展了[基本类型](#)所提供的功能。

另请参阅：

[原生类型](#)和[标准库](#)

线程

Rust 通过 `spawn` 函数提供了一种生成原生操作系统线程的机制，该函数的参数是一个移动闭包。

```
1 use std::thread;
2
3 const NTHREADS: u32 = 10;
4
5 // 这是主线程
6 fn main() {
7     // 创建一个向量来存储生成的子线程
8     let mut children = vec![];
9
10    for i in 0..NTHREADS {
11        // 启动一个新线程
12        children.push(thread::spawn(move || {
13            println!("这是第 {} 号线程", i);
14        }));
15    }
16
17    for child in children {
18        // 等待线程完成，返回一个结果
19        let _ = child.join();
20    }
21 }
```

这些线程将由操作系统进行调度。

测试实例：map-reduce

Rust 使并行数据处理变得非常简单，避免了传统并行处理中常见的诸多问题。

标准库提供了开箱即用的优秀线程原语。这些原语结合 Rust 的所有权概念和别名规则，自动防止了数据竞争。

别名规则（一个可写引用或多个可读引用）自动防止你操作对其他线程可见的状态。（在需要同步的情况下，可以使用 `Mutex` 或 `Channel` 等同步原语。）

在这个例子中，我们将计算一个数字块中所有数字的总和。我们通过将数字块分成小块并分配给不同的线程来完成这个任务。每个线程将计算其小块数字的总和，随后我们将汇总每个线程产生的中间结果。

注意，尽管我们在线程间传递引用，但 Rust 理解我们只是传递只读引用，因此不会发生不安全操作或数据竞争。此外，由于我们传递的引用具有 `'static` 生命周期，Rust 确保这些线程运行时数据不会被销毁。（当需要在线程间共享非 `static` 数据时，可以使用 `Arc` 等智能指针来保持数据存活并避免非 `static` 生命周期。）

```

1 use std::thread;
2
3 // 这是主线程
4 fn main() {
5
6     // 这是我们要处理的数据
7     // 我们将通过一个线程化的 map-reduce 算法计算所有数字的总和
8     // 每个由空格分隔的块将在不同的线程中处理
9     //
10    // TODO: 试试插入空格会对输出有什么影响!
11    let data = "86967897737416471853297327050364959
1211861322575564723963297542624962850
1370856234701860851907960690014725639
1438397966707106094172783238747669219
1552380795257888236525459303330302837
1658495327135744041048897885734297812
1769920216438980873548808413720956532
1816278424637452589860345374828574668";
19
20    // 创建一个向量来存储我们将要生成的子线程。
21    let mut children = vec![];
22
23    /*****
24     * "Map"阶段
25     *
26     * 将数据分割成多个段，并进行初步处理
27     *****/
28
29    // 将数据分割成多个段以进行单独计算
30    // 每个数据块都是指向实际数据的引用 (&str)
31    let chunked_data = data.split_whitespace();
32
33    // 遍历数据段
34    // .enumerate() 为迭代的每个元素添加当前循环索引
35    // 生成的元组"(索引, 元素)"随后立即通过
36    // "解构赋值"被"解构"为两个变量: "i"和"data_segment"
37    //
38    for (i, data_segment) in chunked_data.enumerate() {
39        println!("数据段 {} 是\"{}\"", i, data_segment);
40
41        // 在单独的线程中处理每个数据段
42        //
43        // spawn() 返回新线程的句柄,
44        // 我们必须保留该句柄以访问返回值
45        //
46        // 'move || -> u32' 是一个闭包的语法, 它:
47        // * 不接受参数 ('||')
48        // * 获取其捕获变量的所有权 ('move')
49        // * 返回一个无符号 32 位整数 ('-> u32')
50        //
51        // Rust 足够智能, 能从闭包本身推断出 '-> u32',
52        // 所以我们可以省略它。
53        //
54        // TODO: 尝试移除 'move' 并观察结果
55        children.push(thread::spawn(move || -> u32 {
56            // 计算此段的中间和:
57            let result = data_segment
58                // 遍历此段中的字符...
59                .chars()
60                // 将每个字符转换为对应的数字

```

```

60 // ... 将每个子块对应的数字...
61 .map(|c| c.to_digit(10).expect("应该是一个数字"));
62 // ... 并对结果数字迭代器求和
63 .sum();
64
65 // println! 会锁定标准输出，因此不会出现文本交错
66 println!("已处理段 {}, 结果={}", i, result);
67
68 // 不需要使用 "return"，因为 Rust 是一种"表达式语言"，
69 // 每个代码块中最后求值的表达式会自动成为该块的返回值。
70 result
71
72 });
73 }
74
75
76 /*****
77  * "归约"阶段
78  *
79  * 收集中间结果，并将它们合并成最终结果
80  *****/
81
82 // 将每个线程的中间结果合并成一个最终总和。
83 //
84 // 我们使用 "turbofish" ::<> 为 sum() 提供类型提示。
85 //
86 // TODO: 尝试不使用 turbofish，而是显式
87 // 指定 final_result 的类型
88 let final_result = children.into_iter().map(|c| c.join().unwrap()).sum();
89
90 println!("最终求和结果: {}", final_result);
91 }

```

练习

让线程数量依赖于用户输入的数据并不明智。如果用户决定插入大量空格，我们真的想要创建 2,000 个线程吗？修改程序，使数据始终被分割成固定数量的块，这个数量应由程序开头定义的静态常量来确定。

另请参阅：

- [线程](#)
- [向量和迭代器](#)
- [闭包、move 语义和 move 闭包](#)
- [解构赋值](#)
- [涡轮鱼符号用于辅助类型推断](#)
- [unwrap 与 expect](#)
- [enumerate](#)

信道

Rust 提供异步通道（`channels`）用于线程间通信。通道允许信息在两个端点之间单向流动：发送端（`Sender`）和接收端（`Receiver`）。

```
1 use std::sync::mpsc::{Sender, Receiver};
2 use std::sync::mpsc;
3 use std::thread;
4
5 static NTHREADS: i32 = 3;
6
7 fn main() {
8     // 通道有两个端点: `Sender<T>` 和 `Receiver<T>`,
9     // 其中 `T` 是要传输的消息类型
10    // （此处的类型注解是多余的）
11    let (tx, rx): (Sender<i32>, Receiver<i32>) = mpsc::channel();
12    let mut children = Vec::new();
13
14    for id in 0..NTHREADS {
15        // 发送端可以被复制
16        let thread_tx = tx.clone();
17
18        // 每个线程将通过通道发送其 ID
19        let child = thread::spawn(move || {
20            // 线程获取 `thread_tx` 的所有权
21            // 每个线程在通道中排队一条消息
22            thread_tx.send(id).unwrap();
23
24            // 发送是非阻塞操作，线程在发送消息后
25            // 会立即继续执行
26            println!("线程 {} 已完成", id);
27        });
28
29        children.push(child);
30    }
31
32    // 在这里收集所有消息
33    let mut ids = Vec::with_capacity(NTHREADS as usize);
34    for _ in 0..NTHREADS {
35        // `recv` 方法从通道中获取一条消息
36        // 如果没有可用消息，`recv` 将阻塞当前线程
37        ids.push(rx.recv());
38    }
39
40    // 等待线程完成所有剩余工作
41    for child in children {
42        child.join().expect("糟糕！子线程发生了 panic");
43    }
44
45    // 显示消息发送的顺序
46    println!("{:?}", ids);
47 }
```


路径

The `Path` type represents file paths in the underlying filesystem. Across all platforms there is a single `std::path::Path` that abstracts over platform-specific path semantics and separators. Bring it into scope with `use std::path::Path;` when needed.

`Path` 可以从 `OsStr` 创建，并提供多种方法来获取路径所指向的文件或目录的信息。

`Path` 是不可变的。`Path` 的所有权版本是 `PathBuf`。`Path` 和 `PathBuf` 之间的关系类似于 `str` 和 `String`：`PathBuf` 可以原地修改，并且可以解引用为 `Path`。

注意，`Path` 在内部**并非**表示为 UTF-8 字符串，而是存储为 `OsString`。因此，将 `Path` 转换为 `&str` **并非**零开销操作，且可能失败（返回一个 `Option`）。然而，`Path` 可以自由地转换为 `OsString` 或 `&OsStr`，分别使用 `into_os_string` 和 `as_os_str` 方法。

```
1 use std::path::Path;
2
3 fn main() {
4     // 从 `&'static str` 创建一个 `Path`
5     let path = Path::new(".");
6
7     // `display` 方法返回一个可用于显示的结构体
8     let _display = path.display();
9
10    // `join` 使用操作系统特定的分隔符将路径与字节容器合并，
11    // 并返回一个 `PathBuf`
12    let mut new_path = path.join("a").join("b");
13
14    // `push` 使用 `&Path` 扩展 `PathBuf`
15    new_path.push("c");
16    new_path.push("myfile.tar.gz");
17
18    // `set_file_name` 更新 `PathBuf` 的文件名
19    new_path.set_file_name("package.tgz");
20
21    // 将 `PathBuf` 转换为字符串切片
22    match new_path.to_str() {
23        None => panic!("新路径不是有效的 UTF-8 序列"),
24        Some(s) => println!("新路径是 {}", s),
25    }
26 }
```

Be sure to check other `Path` methods and the `Metadata` struct.

另请参阅：

[OsStr](#) 和 [Metadata](#)。

文件 I/O

`File` 结构体表示一个已打开的文件（它封装了一个文件描述符），并提供对底层文件的读取和/或写入访问。

由于文件 I/O 操作可能会出现多种错误，所有 `File` 方法都返回 `io::Result<T>` 类型，这是 `Result<T, io::Error>` 的别名。

这使得所有 I/O 操作的失败都变得**显式**。得益于此，程序员可以看到所有可能的失败路径，并被鼓励主动处理这些情况。

open

`open` 函数可用于以只读模式打开文件。

`File` 拥有一个资源（即文件描述符），并在被 `drop` 时负责关闭文件。

```
1 use std::fs::File;
2 use std::io::prelude::*;
3 use std::path::Path;
4
5 fn main() {
6     // 创建指向目标文件的路径
7     let path = Path::new("hello.txt");
8     let display = path.display();
9
10    // 以只读模式打开路径，返回 `io::Result<File>`
11    let mut file = match File::open(&path) {
12        Err(why) => panic!("无法打开 {}: {}", display, why),
13        Ok(file) => file,
14    };
15
16    // 将文件内容读入字符串，返回 `io::Result<usize>`
17    let mut s = String::new();
18    match file.read_to_string(&mut s) {
19        Err(why) => panic!("无法读取 {}: {}", display, why),
20        Ok(_) => print!("{}", " {} 的内容: \n{}", display, s),
21    }
22
23    // `file` 超出作用域，"hello.txt" 文件随之关闭
24 }
```

以下是预期的成功输出：

```
$ echo "Hello World!" > hello.txt
$ rustc open.rs && ./open
hello.txt 的内容:
Hello World!
```

（建议您在不同的失败情况下测试上述示例：例如 `hello.txt` 不存在，或 `hello.txt` 不可读等。）

create

`create` 函数以只写模式打开文件。如果文件已存在，旧内容会被清除；否则，会创建一个新文件。

```
static LOREM_IPSUM: &str =
    "Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
    tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
    quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
    consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
    cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
    proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
    ";

use std::fs::File;
use std::io::prelude::*;
use std::path::Path;

fn main() {
    let path = Path::new("lorem_ipsum.txt");
    let display = path.display();

    // 以只写模式打开文件，返回 `io::Result<File>`
    let mut file = match File::create(&path) {
        Err(why) => panic!("无法创建 {}: {}", display, why),
        Ok(file) => file,
    };

    // 将 `LOREM_IPSUM` 字符串写入 `file`，返回 `io::Result<()>`
    match file.write_all(LOREM_IPSUM.as_bytes()) {
        Err(why) => panic!("无法写入 {}: {}", display, why),
        Ok(_) => println!("成功写入 {}", display),
    }
}
```

以下是预期的成功输出：

```
$ rustc create.rs && ./create
successfully wrote to lorem_ipsum.txt
```

```
$ cat lorem_ipsum.txt
Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod
tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non
proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
```

(与前面的示例类似，建议您在失败情况下测试此示例。)

[OpenOptions](#) 结构体可用于配置文件的打开方式。

read_lines

一种简单的方法

对于初学者来说，这可能是从文件中读取行的一个合理的初步尝试。

```
use std::fs::read_to_string;

fn read_lines(filename: &str) -> Vec<String> {
    let mut result = Vec::new();

    for line in read_to_string(filename).unwrap().lines() {
        result.push(line.to_string())
    }

    result
}
```

由于 `lines()` 方法返回文件中各行的迭代器，我们可以内联执行 `map` 并收集结果，从而得到一个更简洁流畅的表达式。

```
use std::fs::read_to_string;

fn read_lines(filename: &str) -> Vec<String> {
    read_to_string(filename)
        .unwrap() // 遇到可能的文件读取错误时 panic
        .lines()  // 将字符串分割成字符串切片的迭代器
        .map(String::from) // 将每个切片转换为字符串
        .collect() // 将它们收集到一个向量中
}
```

注意，在上述两个示例中，我们都必须将 `lines()` 返回的 `&str` 引用转换为拥有所有权的 `String` 类型，分别使用 `.to_string()` 和 `String::from`。

一种更高效的方法

在这里，我们将打开的 `File` 的所有权传递给 `BufReader` 结构体。`BufReader` 使用内部缓冲区来减少中间分配。

我们还对 `read_lines` 函数进行了改进，使其返回一个迭代器，而不是为每行内容在内存中分配新的 `String` 对象。

```
use std::fs::File;
use std::io::{self, BufRead};
use std::path::Path;

fn main() {
    // 文件 hosts.txt 必须存在于当前路径下
    if let Ok(lines) = read_lines("./hosts.txt") {
        // 消耗迭代器，返回一个（可选的）String
        for line in lines.map_while(Result::ok) {
            println!("{}", line);
        }
    }
}

// 输出被包装在 Result 中以便于错误匹配。
// 返回一个指向文件行读取器的迭代器。
fn read_lines<P>(filename: P) -> io::Result<io::Lines<io::BufReader<File>>>
where P: AsRef<Path>, {
    let file = File::open(filename)?;
    Ok(io::BufReader::new(file).lines())
}
```

运行此程序将逐行打印文件内容。

```
$ echo -e "127.0.0.1\n192.168.0.1\n" > hosts.txt
$ rustc read_lines.rs && ./read_lines
127.0.0.1
192.168.0.1
```

（注意，由于 `File::open` 需要一个泛型 `AsRef<Path>` 作为参数，我们使用 `where` 关键字为 `read_lines()` 方法定义了相同的泛型约束。）

这种方法比在内存中创建包含整个文件内容的 `String` 更加高效。特别是在处理大文件时，后者可能会导致性能问题。

子进程

`process::Output` 结构体表示已完成子进程的输出，而 `process::Command` 结构体是一个进程构建器。

```
1 use std::process::Command;
2
3 fn main() {
4     let output = Command::new("rustc")
5         .arg("--version")
6         .output().unwrap_or_else(|e| {
7             panic!("执行进程失败: {}", e)
8         });
9
10    if output.status.success() {
11        let s = String::from_utf8_lossy(&output.stdout);
12
13        print!("rustc 执行成功，标准输出为: \n{}", s);
14    } else {
15        let s = String::from_utf8_lossy(&output.stderr);
16
17        print!("rustc 执行失败，标准错误输出为: \n{}", s);
18    }
19 }
```

(建议您尝试在上述示例中向 `rustc` 传递一个错误的标志)

管道

`std::process::Child` 结构体代表一个子进程，它暴露了 `stdin`、`stdout` 和 `stderr` 句柄，允许通过管道与底层进程进行交互。

```
use std::io::prelude::*;
use std::process::{Command, Stdio};

static PANGRAM: &'static str =
    "the quick brown fox jumps over the lazy dog\n";

fn main() {
    // 启动 `wc` 命令
    let mut cmd = if cfg!(target_family = "windows") {
        let mut cmd = Command::new("powershell");
        cmd.arg("-Command").arg("$input | Measure-Object -Line -Word -
Character");
        cmd
    } else {
        Command::new("wc")
    };
    let process = match cmd
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn() {
        Err(why) => panic!("无法启动 wc: {}", why),
        Ok(process) => process,
    };

    // 向 `wc` 的 `stdin` 写入字符串。
    //
    // `stdin` 的类型是 `Option<ChildStdin>`，但我们知道这个实例
    // 必定存在，所以可以直接 `unwrap` 它。
    match process.stdin.unwrap().write_all(PANGRAM.as_bytes()) {
        Err(why) => panic!("无法写入 wc 的标准输入: {}", why),
        Ok(_) => println!("已将 pangram 发送给 wc"),
    }

    // 由于 `stdin` 在上述调用后不再存活，它会被 `drop`，
    // 管道随之关闭。
    //
    // 这一点非常重要，否则 `wc` 不会开始处理
    // 我们刚刚发送的输入。

    // `stdout` 字段的类型也是 `Option<ChildStdout>`，因此必须解包。
    let mut s = String::new();
    match process.stdout.unwrap().read_to_string(&mut s) {
        Err(why) => panic!("无法读取 wc 的标准输出: {}", why),
        Ok(_) => print!("wc 的响应为: \n{}", s),
    }
}
```


等待

如果你想等待一个 `process::Child` 完成，你必须调用 `Child::wait`，它会返回一个 `process::ExitStatus`。

```
use std::process::Command;

fn main() {
    let mut child = Command::new("sleep").arg("5").spawn().unwrap();
    let _result = child.wait().unwrap();

    println!("到达 main 函数末尾");
}

$ rustc wait.rs && ./wait
# `wait` 会持续运行 5 秒，直到 `sleep 5` 命令执行完毕
reached end of main
```

文件系统操作

`std::fs` 模块包含多个用于处理文件系统的函数。

```

use std::fs;
use std::fs::{File, OpenOptions};
use std::io;
use std::io::prelude::*;
#[cfg(target_family = "unix")]
use std::os::unix;
#[cfg(target_family = "windows")]
use std::os::windows;
use std::path::Path;

// `% cat path` 命令的简单实现
fn cat(path: &Path) -> io::Result<String> {
    let mut f = File::open(path)?;
    let mut s = String::new();
    match f.read_to_string(&mut s) {
        Ok(_) => Ok(s),
        Err(e) => Err(e),
    }
}

// `% echo s > path` 命令的简单实现
fn echo(s: &str, path: &Path) -> io::Result<()> {
    let mut f = File::create(path)?;

    f.write_all(s.as_bytes())
}

// `% touch path` 命令的简单实现（忽略已存在的文件）
fn touch(path: &Path) -> io::Result<()> {
    match OpenOptions::new().create(true).write(true).open(path) {
        Ok(_) => Ok(()),
        Err(e) => Err(e),
    }
}

fn main() {
    println!("`mkdir a`");
    // 创建目录, 返回 `io::Result<()>`
    match fs::create_dir("a") {
        Err(why) => println!("! {:?}", why.kind()),
        Ok(_) => {},
    }

    println!("`echo hello > a/b.txt`");
    // 可以使用 `unwrap_or_else` 方法简化之前的匹配
    echo("hello", &Path::new("a/b.txt")).unwrap_or_else(|why| {
        println!("! {:?}", why.kind());
    });

    println!("`mkdir -p a/c/d`");
    // 递归创建目录, 返回 `io::Result<()>`
    fs::create_dir_all("a/c/d").unwrap_or_else(|why| {
        println!("! {:?}", why.kind());
    });

    println!("`touch a/c/e.txt`");
    touch(&Path::new("a/c/e.txt")).unwrap_or_else(|why| {

```

```

        println!("! {:?}", why.kind());
    });

    println!("`ln -s ../b.txt a/c/b.txt`");
    // 创建符号链接, 返回 `io::Result<()>`
    #[cfg(target_family = "unix")] {
        unix::fs::symlink("../b.txt", "a/c/b.txt").unwrap_or_else(|why| {
            println!("! {:?}", why.kind());
        });
    }
    #[cfg(target_family = "windows")] {
        windows::fs::symlink_file("../b.txt",
"a/c/b.txt").unwrap_or_else(|why| {
            println!("! {:?}", why.to_string());
        });
    }

    println!("`cat a/c/b.txt`");
    match cat(&Path::new("a/c/b.txt")) {
        Err(why) => println!("! {:?}", why.kind()),
        Ok(s) => println!("> {}", s),
    }

    println!("`ls a`");
    // 读取目录内容, 返回 `io::Result<Vec<Path>>`
    match fs::read_dir("a") {
        Err(why) => println!("! {:?}", why.kind()),
        Ok(paths) => for path in paths {
            println!("> {:?}", path.unwrap().path());
        },
    }

    println!("`rm a/c/e.txt`");
    // 删除文件, 返回 `io::Result<()>`
    fs::remove_file("a/c/e.txt").unwrap_or_else(|why| {
        println!("! {:?}", why.kind());
    });

    println!("`rmdir a/c/d`");
    // 删除空目录, 返回 `io::Result<()>`
    fs::remove_dir("a/c/d").unwrap_or_else(|why| {
        println!("! {:?}", why.kind());
    });
}

```

以下是预期的成功输出：

```
$ rustc fs.rs && ./fs
`mkdir a`
`echo hello > a/b.txt`
`mkdir -p a/c/d`
`touch a/c/e.txt`
`ln -s ../b.txt a/c/b.txt`
`cat a/c/b.txt`
> hello
`ls a`
> "a/b.txt"
> "a/c"
`rm a/c/e.txt`
`rmdir a/c/d`
```

最终 `a` 目录的状态如下：

```
$ tree a
a
|-- b.txt
`-- c
    |-- b.txt -> ../b.txt
```

1 directory, 2 files

另一种定义 `cat` 函数的方法是使用 `?` 运算符：

```
fn cat(path: &Path) -> io::Result<String> {
    let mut f = File::open(path)?;
    let mut s = String::new();
    f.read_to_string(&mut s)?;
    Ok(s)
}
```

另请参阅：

[cfg!](#)

程序参数

标准库

可以使用 `std::env::args` 访问命令行参数，它返回一个迭代器，为每个参数生成一个 `String`：

```
1 use std::env;
2
3 fn main() {
4     let args: Vec<String> = env::args().collect();
5
6     // 第一个参数是用于调用程序的路径
7     println!("我的路径是 {}。", args[0]);
8
9     // 其余参数是传递的命令行参数
10    // 像这样调用程序：
11    // $ ./args arg1 arg2
12    println!("我获得了 {:?} 个参数：{:?}。", args.len() - 1, &args[1..]);
13 }
```

```
$ ./args 1 2 3
程序路径: ./args
接收到 3 个参数: ["1"、"2"、"3"]
```

Crates

此外，在开发命令行应用程序时，还有许多 crate 可以提供额外的功能。其中，`clap` 是一个广受欢迎的命令行参数处理 crate。

参数解析

可以使用模式匹配来解析简单的参数：

```

use std::env;

fn increase(number: i32) {
    println!("{}", number + 1);
}

fn decrease(number: i32) {
    println!("{}", number - 1);
}

fn help() {
    println!("用法:
match_args <字符串>
    检查给定的字符串是否为正确答案。
match_args {{increase|decrease}} <整数>
    将给定的整数增加或减少 1。");
}

fn main() {
    let args: Vec<String> = env::args().collect();

    match args.len() {
        // 未传递参数
        1 => {
            println!("我的名字是 'match_args'。试试传递一些参数吧！");
        },
        // 传递了一个参数
        2 => {
            match args[1].parse() {
                Ok(42) => println!("这就是正确答案！"),
                _ => println!("这不是正确答案。"),
            }
        },
        // 传递了一个命令和一个参数
        3 => {
            let cmd = &args[1];
            let num = &args[2];
            // 解析数字
            let number: i32 = match num.parse() {
                Ok(n) => {
                    n
                },
                Err(_) => {
                    eprintln!("错误: 第二个参数不是整数");
                    help();
                    return;
                },
            };
            // 解析命令
            match &cmd[..] {
                "increase" => increase(number),
                "decrease" => decrease(number),
                _ => {
                    eprintln!("错误: 无效的命令");
                    help();
                },
            }
        }
    }
}

```



```
    },  
    // 所有其他情况  
    _ => {  
        // 显示帮助信息  
        help();  
    }  
}  
}
```

如果你将程序命名为 `match_args.rs` 并使用 `rustc match_args.rs` 编译它，你可以按以下方式执行：

```
$ ./match_args Rust  
This is not the answer.  
$ ./match_args 42  
This is the answer!  
$ ./match_args do something  
error: second argument not an integer  
usage:  
match_args <string>  
    Check whether given string is the answer.  
match_args {increase|decrease} <integer>  
    Increase or decrease given integer by one.  
$ ./match_args do 42  
error: invalid command  
usage:  
match_args <string>  
    Check whether given string is the answer.  
match_args {increase|decrease} <integer>  
    Increase or decrease given integer by one.  
$ ./match_args increase 42  
43
```

外部函数接口

Rust 提供了与 C 库交互的外部函数接口（FFI）。外部函数必须在 `extern` 块内声明，并使用 `#[link]` 属性标注外部库的名称。

```

use std::fmt;

// 此 extern 块链接到 libm 库
#[cfg(target_family = "windows")]
#[link(name = "msvcrt")]
extern {
    // 这是一个外部函数
    // 用于计算单精度复数的平方根
    fn csqrtf(z: Complex) -> Complex;

    fn ccosf(z: Complex) -> Complex;
}
#[cfg(target_family = "unix")]
#[link(name = "m")]
extern {
    // 这是一个外部函数
    // 用于计算单精度复数的平方根
    fn csqrtf(z: Complex) -> Complex;

    fn ccosf(z: Complex) -> Complex;
}

// 由于调用外部函数被认为是不安全的,
// 通常会为它们编写安全的包装函数。
fn cos(z: Complex) -> Complex {
    unsafe { ccosf(z) }
}

fn main() {
    // z = -1 + 0i
    let z = Complex { re: -1., im: 0. };

    // 调用外部函数是一个不安全的操作
    let z_sqrt = unsafe { csqrtf(z) };

    println!("{:?} 的平方根是 {:?}", z, z_sqrt);

    // 调用封装了不安全操作的安全 API
    println!("cos({:?}) = {:?}", z, cos(z));
}

// 单精度复数的最小实现
#[repr(C)]
#[derive(Clone, Copy)]
struct Complex {
    re: f32,
    im: f32,
}

impl fmt::Debug for Complex {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        if self.im < 0. {
            write!(f, "{}-{}i", self.re, -self.im)
        } else {
            write!(f, "{}+{}i", self.re, self.im)
        }
    }
}

```

```
}  
}
```



测试

Rust 是一种非常注重正确性的编程语言，它内置了编写软件测试的支持。

测试分为三种类型：

- [单元测试](#)
- [文档测试](#)
- [集成测试](#)

此外，Rust 还支持为测试指定额外的依赖项：

- [开发依赖](#)

参见

- 《Rust 程序设计语言》中[关于测试的章节](#)
- API 指南中[关于文档测试的内容](#)

单元测试

测试是 Rust 函数，用于验证非测试代码是否按预期方式运行。测试函数的主体通常包括一些准备工作，运行待测试的代码，然后断言结果是否符合预期。

大多数单元测试都放在带有 `#[cfg(test)]` 属性的 `tests` 模块中。测试函数用 `#[test]` 属性标记。

当测试函数中出现恐慌 (panic) 时，测试就会失败。以下是一些辅助宏：

- `assert!(expression)` - 如果表达式求值为 `false`，则会触发 panic。
- `assert_eq!(left, right)` 和 `assert_ne!(left, right)` - 分别用于测试左右表达式的相等性和不相等性。

```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

// 这是一个非常糟糕的加法函数，它的目的是在这个例子中失败。
#[allow(dead_code)]
fn bad_add(a: i32, b: i32) -> i32 {
    a - b
}

#[cfg(test)]
mod tests {
    // 注意这个有用的惯用法：从外部作用域（对于 mod tests 而言）导入名称。
    use super::*;

    #[test]
    fn test_add() {
        assert_eq!(add(1, 2), 3);
    }

    #[test]
    fn test_bad_add() {
        // 这个断言会触发，测试将失败。
        // 请注意，私有函数也可以被测试！
        assert_eq!(bad_add(1, 2), 3);
    }
}
```

可以使用 `cargo test` 命令运行测试。

```
$ cargo test
```

```
running 2 tests
test tests::test_bad_add ... FAILED
test tests::test_add ... ok
```

```
failures:
```

```
---- tests::test_bad_add stdout ----
thread 'tests::test_bad_add' panicked at 'assertion failed: `(left ==
right)`
  left: `-1`,
 right: `3`', src/lib.rs:21:8
note: Run with `RUST_BACKTRACE=1` for a backtrace.
```

```
failures:
```

```
tests::test_bad_add
```

```
test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered
out
```

测试与 ? 运算符

之前的单元测试示例都没有返回类型。但在 Rust 2018 版本中，你的单元测试可以返回 `Result<(), >`，这使得你可以在测试中使用 `?` 运算符！这可以使测试代码更加简洁。

```
1 fn sqrt(number: f64) -> Result<f64, String> {
2     if number >= 0.0 {
3         Ok(number.powf(0.5))
4     } else {
5         Err("负浮点数没有平方根".to_owned())
6     }
7 }
8
9 #[cfg(test)]
10 mod tests {
11     use super::*;
12
13     #[test]
14     fn test_sqrt() -> Result<(), String> {
15         let x = 4.0;
16         assert_eq!(sqrt(x)?.powf(2.0), x);
17         Ok(())
18     }
19 }
```

更多详情请参阅 [《版本指南》](#)。

测试 panic

要检查在某些情况下应该触发恐慌的函数，可以使用 `#[should_panic]` 属性。这个属性接受可选参数 `expected =` ，用于指定预期的恐慌消息文本。如果你的函数可能以多种方式触发 panic，这有助于确保你的测试正在检查正确的 panic 情况。

注意：Rust 还允许使用简写形式 `#[should_panic = "message"]`，其功能与 `#[should_panic(expected = "message")]` 完全相同。两种形式都有效；后者更常用，被认为更明确。

```
pub fn divide_non_zero_result(a: u32, b: u32) -> u32 {
    if b == 0 {
        panic!("除以零错误");
    } else if a < b {
        panic!("除法结果为零");
    }
    a / b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_divide() {
        assert_eq!(divide_non_zero_result(10, 2), 5);
    }

    #[test]
    #[should_panic]
    fn test_any_panic() {
        divide_non_zero_result(1, 0);
    }

    #[test]
    #[should_panic(expected = "除法结果为零")]
    fn test_specific_panic() {
        divide_non_zero_result(1, 10);
    }

    #[test]
    #[should_panic = "除法结果为零"] // 这样也可以
    fn test_specific_panic_shorthand() {
        divide_non_zero_result(1, 10);
    }
}
```

运行这些测试会得到以下结果：


```
$ cargo test

running 4 tests
test tests::test_any_panic ... ok
test tests::test_divide ... ok
test tests::test_specific_panic ... ok
test tests::test_specific_panic_shorthand ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Doc-tests tmp-test-should-panic

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

运行特定测试

要运行特定的测试，可以在 `cargo test` 命令中指定测试名称。

```
$ cargo test test_any_panic
running 1 test
test tests::test_any_panic ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 3 filtered out

Doc-tests tmp-test-should-panic

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

要运行多个测试，可以指定测试名称的一部分，该部分匹配所有应该运行的测试。

```
$ cargo test panic
running 3 tests
test tests::test_any_panic ... ok
test tests::test_specific_panic ... ok
test tests::test_specific_panic_shorthand ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out

Doc-tests tmp-test-should-panic

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

忽略测试

可以使用 `#[ignore]` 属性标记测试以排除某些测试。或者使用命令 `cargo test -- --ignored` 来运行这些被忽略的测试。

```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_add() {
        assert_eq!(add(2, 2), 4);
    }

    #[test]
    fn test_add_hundred() {
        assert_eq!(add(100, 2), 102);
        assert_eq!(add(2, 100), 102);
    }

    #[test]
    #[ignore]
    fn ignored_test() {
        assert_eq!(add(0, 0), 0);
    }
}
```

```
$ cargo test
running 3 tests
test tests::ignored_test ... ignored
test tests::test_add ... ok
test tests::test_add_hundred ... ok

test result: ok. 2 passed; 0 failed; 1 ignored; 0 measured; 0 filtered out

Doc-tests tmp-ignore

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

$ cargo test -- --ignored
running 1 test
test tests::ignored_test ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Doc-tests tmp-ignore

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

文档测试

Rust 项目的主要文档编写方式是通过在源代码中添加注释。文档注释使用 [CommonMark Markdown 规范](#) 编写，并支持其中的代码块。Rust 注重正确性，因此这些代码块会被编译并用作文档测试。

```
/// 第一行是函数的简要描述。
///
/// 接下来的几行是详细文档。代码块以三个反引号开始，
/// 并隐含了 `fn main()` 函数和 `extern crate <cratename>` 声明。
/// 假设我们正在测试 `playground` crate:
///
/// ```
/// let result = playground::add(2, 3);
/// assert_eq!(result, 5);
/// ```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

/// 文档注释通常包含"示例"、"异常"和"错误"等部分。
///
/// 下面的函数用于两数相除。
///
/// # 示例
///
/// ```
/// let result = playground::div(10, 2);
/// assert_eq!(result, 5);
/// ```
///
/// # 异常
///
/// 当第二个参数为零时，函数会触发异常。
///
/// ```rust,should_panic
/// // 除以零会触发异常
/// playground::div(10, 0);
/// ```
pub fn div(a: i32, b: i32) -> i32 {
    if b == 0 {
        panic!("除以零错误");
    }

    a / b
}
```

运行常规的 `cargo test` 命令时，文档中的代码块会自动进行测试：

```
$ cargo test
running 0 tests
```

```
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

```
Doc-tests playground
```

```
running 3 tests
test src/lib.rs - add (line 7) ... ok
test src/lib.rs - div (line 21) ... ok
test src/lib.rs - div (line 31) ... ok
```

```
test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

文档测试的动机

文档测试的主要目的是提供功能演示的示例，这是最重要的[指导原则](#)之一。它允许将文档中的示例作为完整的代码片段使用。但是使用 `?` 会导致编译失败，因为 `main` 函数返回 `unit` 类型。这时，隐藏文档中的某些源代码行就派上用场了：可以编写 `fn try_main() ->`

`Result<(), ErrorType>`，将其隐藏，并在隐藏的 `main` 函数中 `unwrap` 它。听起来很复杂？这里有一个例子：

```
/// 在文档测试中使用隐藏的 `try_main` 函数。
///
/// ```
/// # // 以 `#` 开头的行在文档中是隐藏的，但它们仍然可以编译！
/// # fn try_main() -> Result<(), String> { // 这行包装了文档中显示的函数体
/// # let res = playground::try_div(10, 2)?;
/// # Ok(()) // 从 try_main 返回
/// # }
/// # fn main() { // 开始会调用 unwrap() 的 main 函数
/// #     try_main().unwrap(); // 调用 try_main 并解包
/// # } // 这样在出错时测试会触发 panic
/// # }
/// ```
pub fn try_div(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        Err(String::from("除以零错误"))
    } else {
        Ok(a / b)
    }
}
```

参见

- 关于文档风格的 [RFC505](#)
- 关于文档指南的 [API 指南](#)

集成测试

单元测试每次只隔离测试一个模块：它们规模小，可以测试私有代码。集成测试则位于 crate 外部，仅使用其公共接口，就像其他代码一样。集成测试的目的是验证库的多个部分能否正确协同工作。

Cargo 在 `src` 目录旁的 `tests` 目录中查找集成测试。

文件 `src/lib.rs`：

```
// 在名为 `adder` 的 crate 中定义此内容。
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

测试文件： `tests/integration_test.rs`：

```
#[test]
fn test_add() {
    assert_eq!(adder::add(3, 2), 5);
}
```

使用 `cargo test` 命令运行测试：

```
$ cargo test
running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

    Running target/debug/deps/integration_test-bcd60824f5fbfe19

running 1 test
test test_add ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

`tests` 目录中的每个 Rust 源文件都被编译为独立的 crate。为了在集成测试之间共享代码，我们可以创建一个包含公共函数的模块，然后在测试中导入并使用它。

文件 `tests/common/mod.rs`：

```
pub fn setup() {  
    // 一些设置代码，如创建必要的文件/目录，启动  
    // 服务器等。  
}
```

测试文件: `tests/integration_test.rs`

```
// 导入 common 模块。  
mod common;  
  
#[test]  
fn test_add() {  
    // 使用公共代码。  
    common::setup();  
    assert_eq!(adder::add(3, 2), 5);  
}
```

将模块创建为 `tests/common.rs` 也可行，但不推荐，因为测试运行器会将该文件视为测试 crate 并尝试运行其中的测试。

开发依赖

有时我们需要仅用于测试（或示例、基准测试）的依赖项。这些依赖项添加在 `Cargo.toml` 的 `[dev-dependencies]` 部分。这些依赖项不会传递给依赖于本包的其他包。

例如 `pretty_assertions`，它扩展了标准的 `assert_eq!` 和 `assert_ne!` 宏，提供彩色差异对比。

文件 `Cargo.toml`：

```
# 省略标准的 crate 数据
[dev-dependencies]
pretty_assertions = "1"
```

文件 `src/lib.rs`：

```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;
    use pretty_assertions::assert_eq; // 仅用于测试的 crate。不能在非测试代码中使用。

    #[test]
    fn test_add() {
        assert_eq!(add(2, 3), 5);
    }
}
```

参见

[Cargo 文档](#)中关于指定依赖项的说明。

不安全操作

作为本节的引言，借用[官方文档](#)的话说：“应该尽量减少代码库中不安全代码的数量。”牢记这一点，让我们开始吧！Rust 中的不安全标注用于绕过编译器设置的保护机制。具体来说，不安全主要用于以下四个方面：

- 解引用裸指针
- 调用被标记为 `unsafe` 的函数或方法（包括通过 FFI 调用函数，参见本书[前面的章节](#)）
- 访问或修改静态可变变量
- 实现不安全特征

裸指针

裸指针 `*` 和引用 `&T` 的功能类似，但引用总是安全的，因为借用检查器保证它们指向有效数据。解引用裸指针只能在 `unsafe` 块中进行。

```
1 fn main() {
2     let raw_p: *const u32 = &10;
3
4     unsafe {
5         assert!(*raw_p == 10);
6     }
7 }
```

调用不安全函数

某些函数可以被声明为 `unsafe`，这意味着确保其正确性是程序员的责任，而不是编译器的责任。一个例子是 `std::slice::from_raw_parts`，它根据指向第一个元素的指针和长度创建一个切片。

```
1 use std::slice;
2
3 fn main() {
4     let some_vector = vec![1, 2, 3, 4];
5
6     let pointer = some_vector.as_ptr();
7     let length = some_vector.len();
8
9     unsafe {
10         let my_slice: &[u32] = slice::from_raw_parts(pointer, length);
11
12         assert_eq!(some_vector.as_slice(), my_slice);
13     }
14 }
```

对于 `slice::from_raw_parts`，**必须遵守**的一个假设是：传入的指针指向有效内存，且指向的内存类型正确。如果这些不变量未被遵守，那么程序的行为将是未定义的，无法预知会发生什



内联汇编

Rust 通过 `asm!` 宏提供了内联汇编支持。它可以用于在编译器生成的汇编输出中嵌入手写的汇编代码。通常这不是必需的，但在无法通过其他方式实现所需性能或时序要求时可能会用到。访问底层硬件原语（例如在内核代码中）也可能需要这个功能。

注意：这里的示例使用 x86/x86-64 汇编，但也支持其他架构。

目前支持内联汇编的架构包括：

- x86 和 x86-64
- ARM
- AArch64
- RISC-V

基本用法

让我们从最简单的例子开始：

```
use std::arch::asm;

unsafe {
    asm!("nop");
}
```

这将在编译器生成的汇编代码中插入一条 NOP（无操作）指令。请注意，所有 `asm!` 调用都必须放在 `unsafe` 块内，因为它们可能插入任意指令并破坏各种不变量。要插入的指令以字符串字面量的形式列在 `asm!` 宏的第一个参数中。

输入和输出

插入一个什么都不做的指令相当无聊。让我们来做些实际操作数据的事情：

```
use std::arch::asm;

let x: u64;
unsafe {
    asm!("mov {}, 5", out(reg) x);
}
assert_eq!(x, 5);
```

这将会把值 5 写入 u64 类型的变量 x。你可以看到，我们用来指定指令的字符串字面量实际上是一个模板字符串。它遵循与 Rust [格式化字符串](#) 相同的规则。然而，插入到模板中的参数看起来可能与你熟悉的有些不同。首先，我们需要指定变量是内联汇编的输入还是输出。在这个例子中，它是一个输出。我们通过写 `out` 来声明这一点。我们还需要指定汇编期望变量在什么类型的寄存器中。这里我们通过指定 `reg` 将其放在任意通用寄存器中。编译器将选择一个合适的寄存器插入到模板中，并在内联汇编执行完成后从该寄存器读取变量的值。

让我们再看一个使用输入的例子：

```
use std::arch::asm;

let i: u64 = 3;
let o: u64;
unsafe {
    asm!(
        "mov {0}, {1}",
        "add {0}, 5",
        out(reg) o,
        in(reg) i,
    );
}
assert_eq!(o, 8);
```

这段代码会将 5 加到变量 i 的值上，然后将结果写入变量 o。具体的汇编实现是先将 i 的值复制到输出寄存器，然后再加上 5。

这个例子展示了几个要点：

`asm!` 宏支持多个模板字符串参数，每个参数都被视为独立的汇编代码行，就像它们之间用换行符连接一样。这使得格式化汇编代码变得简单。

其次，我们可以看到输入参数使用 `in` 声明，而不是 `out`。

第三，我们可以像在任何格式字符串中一样指定参数编号或名称。这在内联汇编模板中特别有用，因为参数通常会被多次使用。对于更复杂的内联汇编，建议使用这种方式，因为它提高了可读性，并且允许在不改变参数顺序的情况下重新排列指令。

我们可以进一步优化上面的例子，避免使用 `mov` 指令：

```
use std::arch::asm;

let mut x: u64 = 3;
unsafe {
    asm!("add {0}, 5", inout(reg) x);
}
assert_eq!(x, 8);
```

我们可以看到 `inout` 用于指定既作为输入又作为输出的参数。这与分别指定输入和输出不同，它保证将两者分配到同一个寄存器。

也可以为 `inout` 操作数的输入和输出部分指定不同的变量：

```
use std::arch::asm;

let x: u64 = 3;
let y: u64;
unsafe {
    asm!("add {0}, 5", inout(reg) x => y);
}
assert_eq!(y, 8);
```

延迟输出操作数

Rust 编译器在分配操作数时采取保守策略。它假设 `out` 可以在任何时候被写入，因此不能与其他参数共享位置。然而，为了保证最佳性能，使用尽可能少的寄存器很重要，这样就不必在内联汇编块前后保存和重新加载寄存器。为此，Rust 提供了 `lateout` 说明符。这可以用于任何在所有输入被消耗后才写入的输出。此外还有一个 `inlateout` 变体。

以下是一个在 `release` 模式或其他优化情况下 不能使用 `inlateout` 的例子：

```
use std::arch::asm;

let mut a: u64 = 4;
let b: u64 = 4;
let c: u64 = 4;
unsafe {
    asm!(
        "add {0}, {1}",
        "add {0}, {2}",
        inout(reg) a,
        in(reg) b,
        in(reg) c,
    );
}
assert_eq!(a, 12);
```

在未优化的情况下（如 `Debug` 模式），将上述例子中的 `inout(reg) a` 替换为 `inlateout(reg) a` 仍能得到预期结果。但在 `release` 模式或其他优化情况下，使用 `inlateout(reg) a` 可能导致最终值 `a = 16`，使断言失败。

这是因为在优化情况下，编译器可以为输入 `b` 和 `c` 分配相同的寄存器，因为它知道它们具有相同的值。此外，当使用 `inlateout` 时，`a` 和 `c` 可能被分配到同一个寄存器，这种情况下，第一条 `add` 指令会覆盖从变量 `c` 初始加载的值。相比之下，使用 `inout(reg) a` 可以确保为 `a` 分配一个单独的寄存器。

然而，以下示例可以使用 `inlateout`，因为输出仅在读取所有输入寄存器后才被修改：

```
use std::arch::asm;

let mut a: u64 = 4;
let b: u64 = 4;
unsafe {
    asm!("add {0}, {1}", inlateout(reg) a, in(reg) b);
}
assert_eq!(a, 8);
```

如你所见，即使 `a` 和 `b` 被分配到同一个寄存器，这段汇编代码片段仍能正确运行。

显式寄存器操作数

某些指令要求操作数必须位于特定寄存器中。因此，Rust 内联汇编提供了一些更具体的约束说明符。虽然 `reg` 通常适用于任何架构，但显式寄存器高度依赖于特定架构。例如，对于 x86 架构，通用寄存器如 `eax`、`ebx`、`ecx`、`edx`、`ebp`、`esi` 和 `edi` 等可以直接通过名称进行寻址。

```
use std::arch::asm;

let cmd = 0xd1;
unsafe {
    asm!("out 0x64, eax", in("eax") cmd);
}
```

在这个例子中，我们调用 `out` 指令将 `cmd` 变量的内容输出到端口 `0x64`。由于 `out` 指令只接受 `eax`（及其子寄存器）作为操作数，我们必须使用 `eax` 约束说明符。

注意：与其他操作数类型不同，显式寄存器操作数不能在模板字符串中使用。你不能使用 `{}`，而应直接写入寄存器名称。此外，它们必须出现在操作数列表的末尾，位于所有其他操作数类型之后。

考虑以下使用 x86 `mul` 指令的例子：

```
use std::arch::asm;

fn mul(a: u64, b: u64) -> u128 {
    let lo: u64;
    let hi: u64;

    unsafe {
        asm!(
            // x86 的 mul 指令将 rax 作为隐式输入，
            // 并将乘法的 128 位结果写入 rax:rdx。
            "mul {}",
            in(reg) a,
            inlateout("rax") b => lo,
            lateout("rdx") hi
        );
    }

    ((hi as u128) << 64) + lo as u128
}
```

这里使用 `mul` 指令将两个 64 位输入相乘，得到一个 128 位的结果。唯一的显式操作数是一个寄存器，我们用变量 `a` 填充它。第二个操作数是隐式的，必须是 `rax` 寄存器，我们用变量 `b` 填充它。结果的低 64 位存储在 `rax` 中，用于填充变量 `lo`。高 64 位存储在 `rdx` 中，用于填充变量 `hi`。

被破坏的寄存器

在许多情况下，内联汇编会修改不需要作为输出的状态。这通常是因为我们必须在汇编中使用临时寄存器，或者因为指令修改了我们不需要进一步检查的状态。这种状态通常被称为“被破坏”。我们需要告知编译器这一点，因为它可能需要在内联汇编块前后保存和恢复这种状态。

```

use std::arch::asm;

fn main() {
    // 三个条目，每个四字节
    let mut name_buf = [0_u8; 12];
    // 字符串按顺序以 ASCII 格式存储在 ebx、edx、ecx 中
    // 由于 ebx 是保留寄存器，汇编需要保留其值
    // 因此我们在主要汇编代码前后执行 push 和 pop 操作
    // 64 位处理器的 64 位模式不允许对 32 位寄存器（如 ebx）进行 push/pop 操作
    // 所以我们必须使用扩展的 rbx 寄存器

    unsafe {
        asm!(
            "push rbx",
            "cpuid",
            "mov [rdi], ebx",
            "mov [rdi + 4], edx",
            "mov [rdi + 8], ecx",
            "pop rbx",
            // 我们使用指向数组的指针来存储值，以简化 Rust 代码
            // 虽然这会增加几条汇编指令，但更清晰地展示了汇编的工作方式
            // 相比于使用显式寄存器输出（如 `out("ecx") val`）
            // *指针本身*只是一个输入，尽管它在背后被写入
            in("rdi") name_buf.as_mut_ptr(),
            // 选择 cpuid 0，同时指定 eax 为被修改寄存器
            inout("eax") 0 => _,
            // cpuid 也会修改这些寄存器
            out("ecx") _,
            out("edx") _,
        );
    }

    let name = core::str::from_utf8(&name_buf).unwrap();
    println!("CPU 制造商 ID: {}", name);
}

```

在上面的示例中，我们使用 `cpuid` 指令读取 CPU 制造商 ID。该指令将最大支持的 `cpuid` 参数写入 `eax`，并按顺序将 CPU 制造商 ID 的 ASCII 字节写入 `ebx`、`edx` 和 `ecx`。

尽管 `eax` 从未被读取，我们仍需要告知编译器该寄存器已被修改，这样编译器就可以保存汇编前这些寄存器中的任何值。我们通过将其声明为输出来实现这一点，但使用 `_` 而非变量名，表示输出值将被丢弃。

这段代码还解决了 LLVM 将 `ebx` 视为保留寄存器的限制。这意味着 LLVM 假定它对该寄存器拥有完全控制权，并且必须在退出汇编块之前将其恢复到原始状态。因此，`ebx` 不能用作输入或输出，**除非**编译器将其用于满足通用寄存器类（如 `in(reg)`）。这使得在使用保留寄存器时，`reg` 操作数变得危险，因为我们可能会在不知情的情况下破坏输入或输出，原因是它们共享同一个寄存器。

为了解决这个问题，我们采用以下策略：使用 `rdi` 存储输出数组的指针；通过 `push` 保存 `ebx`；在汇编块内从 `ebx` 读取数据到数组中；然后通过 `pop` 将 `ebx` 恢复到原始状态。`push`

和 `pop` 操作使用完整的 64 位 `rbx` 寄存器版本，以确保整个寄存器被保存。在 32 位目标上，代码会在 `push / pop` 操作中使用 `ebx`。

这种技术还可以与通用寄存器类一起使用，以获得一个临时寄存器在汇编代码内使用：

```
use std::arch::asm;

// 使用移位和加法将 x 乘以 6
let mut x: u64 = 4;
unsafe {
    asm!(
        "mov {tmp}, {x}",
        "shl {tmp}, 1",
        "shl {x}, 2",
        "add {x}, {tmp}",
        x = inout(reg) x,
        tmp = out(reg) _,
    );
}
assert_eq!(x, 4 * 6);
```

符号操作数和 ABI 破坏

默认情况下，`asm!` 假定汇编代码会保留所有未指定为输出的寄存器的内容。`asm!` 的 `clobber_abi` 参数告诉编译器根据给定的调用约定 ABI 自动插入必要的破坏操作数：任何在该 ABI 中未完全保留的寄存器都将被视为被破坏。可以提供多个 `clobber_abi` 参数，所有指定 ABI 的破坏都将被插入。

```

use std::arch::asm;

extern "C" fn foo(arg: i32) -> i32 {
    println!("arg = {}", arg);
    arg * 2
}

fn call_foo(arg: i32) -> i32 {
    unsafe {
        let result;
        asm!(
            "call {}",
            // 要调用的函数指针
            in(reg) foo,
            // 第一个参数在 rdi 中
            in("rdi") arg,
            // 返回值在 rax 中
            out("rax") result,
            // 将所有不被 "C" 调用约定保留的寄存器
            // 标记为被破坏
            clobber_abi("C"),
        );
        result
    }
}

```

寄存器模板修饰符

在某些情况下，需要对寄存器名称插入模板字符串时的格式进行精细控制。当一个架构的汇编语言对同一个寄存器有多个名称时，这种控制尤为必要。每个名称通常代表寄存器的一个子集“视图”（例如，64 位寄存器的低 32 位）。

默认情况下，编译器总是会选择引用完整寄存器大小的名称（例如，在 x86-64 上是 `rax`，在 x86 上是 `eax` 等）。

可以通过在模板字符串操作数上使用修饰符来覆盖这个默认设置，类似于格式字符串的用法：

```

use std::arch::asm;

let mut x: u16 = 0xab;

unsafe {
    asm!("mov {0:h}, {0:l}", inout(reg_abcd) x);
}

assert_eq!(x, 0xabab);

```

在这个例子中，我们使用 `reg_abcd` 寄存器类来限制寄存器分配器只使用 4 个传统的 x86 寄存器（`ax`、`bx`、`cx`、`dx`）。这些寄存器的前两个字节可以独立寻址。

假设寄存器分配器选择将 `x` 分配到 `ax` 寄存器。`h` 修饰符将生成该寄存器高字节的名称，而 `l` 修饰符将生成低字节的名称。因此，汇编代码将被展开为 `mov ah, al`，这条指令将值的低字节复制到高字节。

如果你对操作数使用较小的数据类型（例如 `u16`）并忘记使用模板修饰符，编译器将发出警告并建议使用正确的修饰符。

内存地址操作数

有时汇编指令需要通过内存地址或内存位置传递操作数。你必须手动使用目标架构指定的内存地址语法。例如，在使用 Intel 汇编语法的 x86/x86_64 架构上，你应该用 `[]` 包裹输入/输出，以表明它们是内存操作数：

```
use std::arch::asm;

fn load_fpu_control_word(control: u16) {
    unsafe {
        asm!("fldcw [{}]", in(reg) &control, options(nostack));
    }
}
```

标签

重复使用命名标签（无论是局部的还是其他类型的）可能导致汇编器或链接器错误，或引起其他异常行为。命名标签的重用可能以多种方式发生，包括：

- 显式重用：在一个 `asm!` 块中多次使用同一标签，或在多个块之间重复使用。
- 通过内联隐式重用：编译器可能会创建 `asm!` 块的多个副本，例如当包含该块的函数在多处被内联时。
- 通过 LTO 隐式重用：链接时优化（LTO）可能导致**其他 crate** 的代码被放置在同一代码生成单元中，从而可能引入任意标签。

因此，你应该只在内联汇编代码中使用 GNU 汇编器的**数字局部标签**。在汇编代码中定义符号可能会由于重复的符号定义而导致汇编器和/或链接器错误。

此外，在 x86 架构上使用默认的 Intel 语法时，由于一个 [LLVM 的 bug](#)，你不应使用仅由 `0` 和 `1` 组成的标签，如 `0`、`11` 或 `101010`，因为它们可能被误解为二进制值。使用 `options(att_syntax)` 可以避免这种歧义，但这会影响整个 `asm!` 块的语法。（关于 `options` 的更多信息，请参见下文的[选项](#)。）

```
use std::arch::asm;

let mut a = 0;
unsafe {
    asm!(
        "mov {0}, 10",
        "2:",
        "sub {0}, 1",
        "cmp {0}, 3",
        "jle 2f",
        "jmp 2b",
        "2:",
        "add {0}, 2",
        out(reg) a
    );
}
assert_eq!(a, 5);
```

这段代码会将 `{0}` 寄存器的值从 10 递减到 3，然后加 2 并将结果存储在 `a` 中。

这个例子展示了几个要点：

- 首先，同一个数字可以在同一个内联块中多次用作标签。
- 其次，当数字标签被用作引用（例如作为指令操作数）时，应在数字标签后添加后缀 “b” (“backward”，向后) 或 “f” (“forward”，向前)。这样它将引用该方向上由这个数字定义的最近的标签。

选项

默认情况下，内联汇编块的处理方式与具有自定义调用约定的外部 FFI 函数调用相同：它可能读写内存，产生可观察的副作用等。然而，在许多情况下，我们希望向编译器提供更多关于汇编代码实际行为的信息，以便编译器能够进行更好的优化。

让我们回顾一下之前 `add` 指令的例子：

```
use std::arch::asm;

let mut a: u64 = 4;
let b: u64 = 4;
unsafe {
    asm!(
        "add {0}, {1}",
        inlateout(reg) a, in(reg) b,
        options(pure, nomem, nostack),
    );
}
assert_eq!(a, 8);
```

可以将选项作为可选的最后一个参数传递给 `asm!` 宏。在这个例子中，我们指定了三个选项：

- `pure`：表示汇编代码没有可观察的副作用，其输出仅依赖于输入。这使得编译器优化器能够减少内联汇编的调用次数，甚至完全消除它。
- `nomem`：表示汇编代码不读取或写入内存。默认情况下，编译器会假设内联汇编可以读写任何它可访问的内存地址（例如通过作为操作数传递的指针或全局变量）。
- `nostack`：表示汇编代码不会向栈中压入任何数据。这允许编译器使用诸如 x86-64 上的栈红区等优化技术，以避免栈指针调整。

这些选项使编译器能够更好地优化使用 `asm!` 的代码，例如消除那些输出未被使用的纯 `asm!` 块。

有关可用选项的完整列表及其效果，请参阅[参考文档](#)。

兼容性

Rust 语言正在快速发展，因此某些兼容性问题可能会出现，尽管我们努力确保尽可能的向前兼容。

- [原始标识符](#)

原始标识符

Rust 和许多编程语言一样，有“关键字”的概念。这些标识符在语言中具有特殊含义，因此你不能在变量名、函数名等地方使用它们。原始标识符允许你在通常不允许使用关键字的地方使用它们。这在 Rust 引入新关键字，而使用旧版本 Rust 的库中有与新版本引入的关键字同名的变量或函数时特别有用。

例如，假设有一个使用 Rust 2015 版编译的 crate `foo`，它导出了一个名为 `try` 的函数。这个关键字在 2018 版中被保留用于新特性，如果没有原始标识符，我们就无法命名这个函数。

```
extern crate foo;

fn main() {
    foo::try();
}
```

你会得到这个错误：

```
error: expected identifier, found keyword `try`
--> src/main.rs:4:4
   |
4  | foo::try();
   |      ^^^ expected identifier, found keyword
```

你可以使用原始标识符这样写：

```
extern crate foo;

fn main() {
    foo::r#try();
}
```

补充

Some topics aren't exactly relevant to how your program runs but provide you tooling or infrastructure support which just makes things better for everyone. These topics include:

- [文档](#): 使用内置的 `rustdoc` 为用户生成库文档。
- [Playground](#): 在文档中集成 Rust Playground。

文档

使用 `cargo doc` 在 `target/doc` 目录下构建文档。运行 `cargo doc --open` 将自动在浏览器中打开文档。

使用 `cargo test` 运行所有测试（包括文档测试）。如果只想运行文档测试，请使用 `cargo test --doc`。

这些命令会根据需要适当地调用 `rustdoc`（和 `rustc`）。

文档注释

文档注释对需要文档的大型项目非常有用。运行 `rustdoc` 时，这些注释会被编译成文档。文档注释以 `///` 开头，并支持 [Markdown](#) 语法。